



Lab Manual

Artificial Intelligence

Faculty of Computing & Information Technology (FCIT)

University of the Punjab, Lahore.

www.pucit.edu.pk

Artificial Intelligence

Lab: 01

Lab 1

Introduction to AI and Its Application Using Python

Python is widely used for **artificial intelligence**, with packages for a number of applications including General **AI**, Machine Learning, Natural **Language** Processing and Neural Networks. Haskell is also a very good programming **language** for **AI**. Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language. It was created by Guido van Rossum during 1985- 1990. Like Perl, Python source code is also available under the GNU General Public License (GPL).

Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.

Python is Interactive – You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python is Object-Oriented – Python supports Object-Oriented style or technique of programming that encapsulates code within objects.

Python is a Beginner's Language – Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

Programming syntax

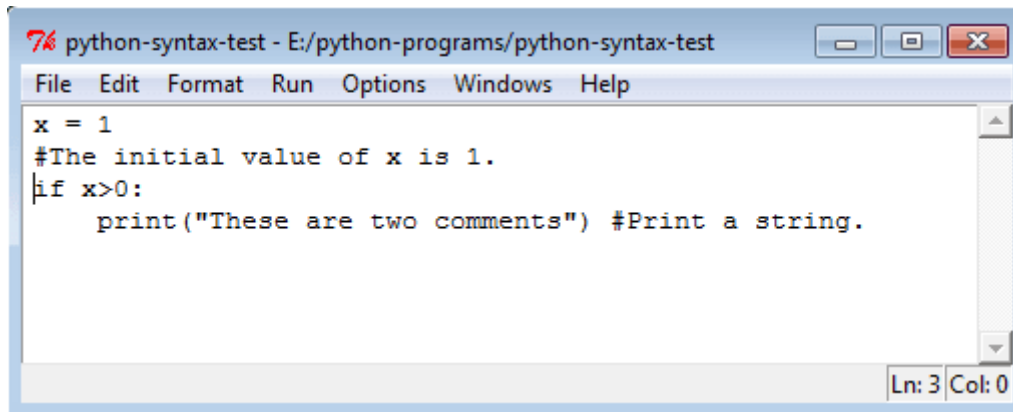
Execute Python Syntax

- As we learned in the previous page, Python syntax can be executed by writing directly in the Command Line:
- ```
>>> print("Hello, World!")
Hello, World!
```
- Or by creating a python file on the server, using the .py file extension, and running it in the Command Line:
- ```
C:\Users\Your Name>python myfile.py
```

A Python program is read by a parser. Python was designed to be a highly readable language. The syntax of the Python programming language is the set of rules which defines how a Python program will be written.

Comments in Python:

A comment begins with a hash character(#) which is not a part of the string literal and ends at the end of the physical line. All characters after the # character up to the end of the line are part of the comment and the Python interpreter ignores them. See the following example. It should be noted that Python has no multi-lines or block comments facility.

A screenshot of a Python IDE window titled "python-syntax-test - E:/python-programs/python-syntax-test". The window has a menu bar with "File", "Edit", "Format", "Run", "Options", "Windows", and "Help". The main text area contains the following Python code:

```
x = 1
#The initial value of x is 1.
if x>0:
    print("These are two comments") #Print a string.
```

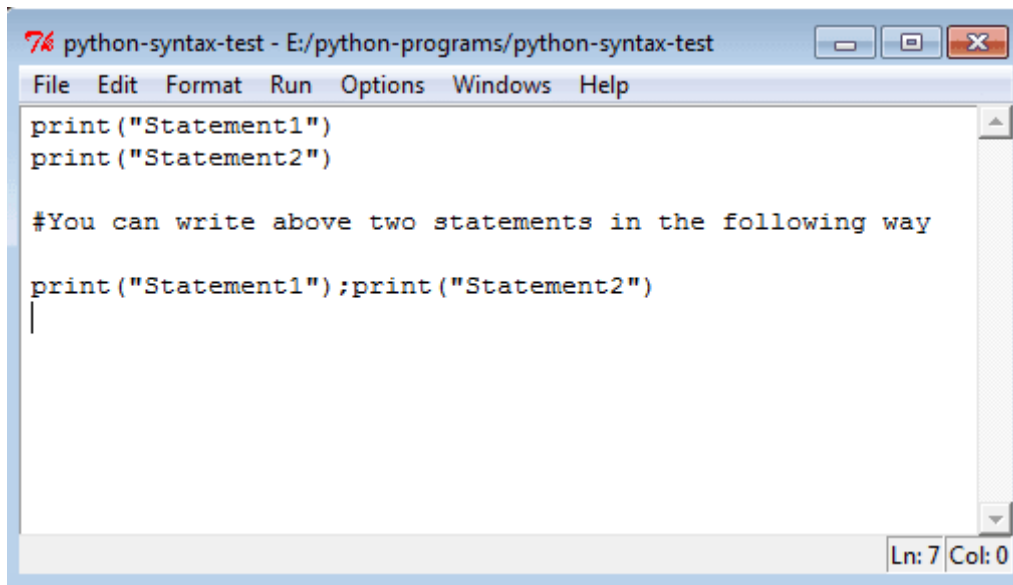
The status bar at the bottom right shows "Ln: 3 Col: 0".

Input / output

```
txt = input("Type something to test this out: ")
print(txt)
```

Multiple Statements on a Single Line:

You can write two separate statements into a single line using a semicolon (;) character between two line.



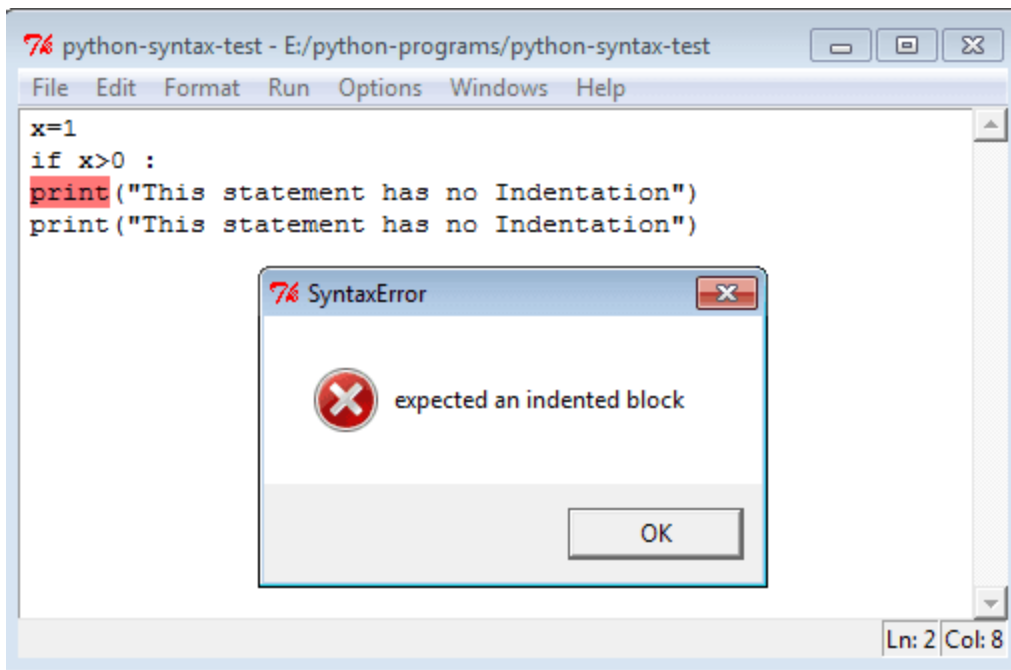
```
python-syntax-test - E:/python-programs/python-syntax-test
File Edit Format Run Options Windows Help
print("Statement1")
print("Statement2")

#You can write above two statements in the following way
print("Statement1");print("Statement2")
|
```

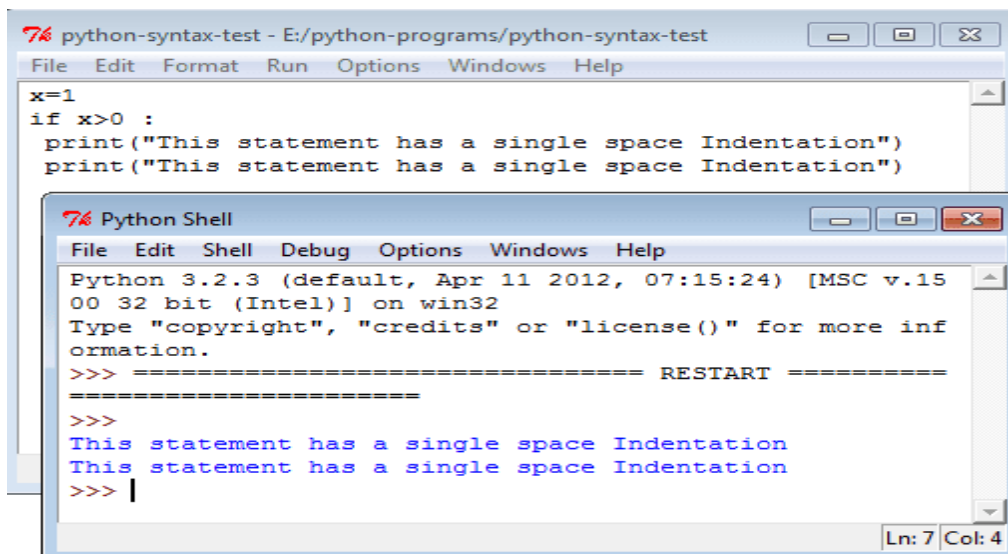
Ln: 7 Col: 0

Indentation:

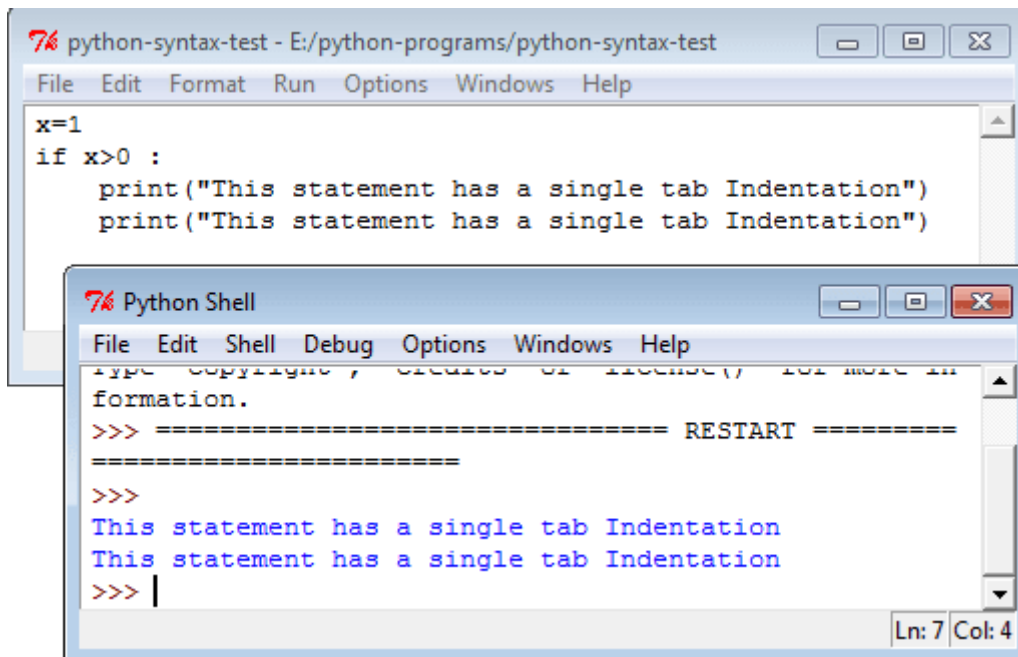
Python uses whitespace (spaces and tabs) to define program blocks whereas other languages like C, C++ use braces ({}) to indicate blocks of codes for class, functions or flow control. The number of whitespaces (spaces and tabs) in the indentation is not fixed, but all statements within the block must be the indented same amount. In the following program, the block statements have no indentation.



This is a program with single space indentation.



This is a program with single tab indentation.



The screenshot shows a Python IDE window titled "python-syntax-test - E:/python-programs/python-syntax-test". The code in the editor is:

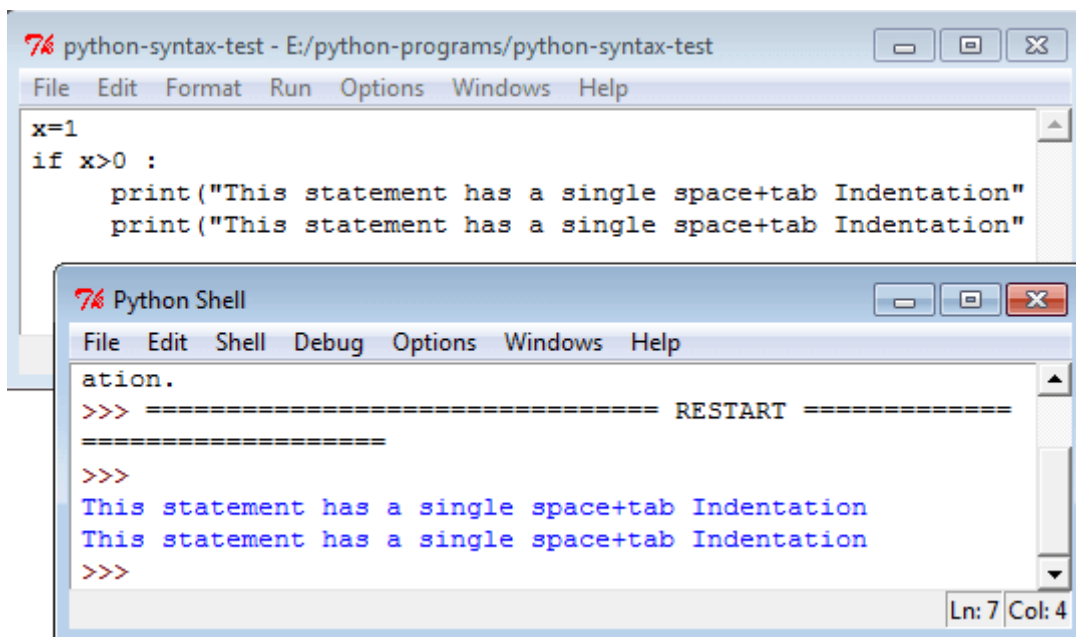
```
x=1
if x>0 :
    print("This statement has a single tab Indentation")
    print("This statement has a single tab Indentation")
```

Below the editor is a "Python Shell" window. It shows the execution of the code, with the output:

```
>>> ===== RESTART =====
>>>
This statement has a single tab Indentation
This statement has a single tab Indentation
>>> |
```

The status bar at the bottom right of the shell indicates "Ln: 7 Col: 4".

Here is an another program with an indentation of a single space + a single tab.



The screenshot shows a Python IDE window titled "python-syntax-test - E:/python-programs/python-syntax-test". The code in the editor is:

```
x=1
if x>0 :
    print("This statement has a single space+tab Indentation")
    print("This statement has a single space+tab Indentation")
```

Below the editor is a "Python Shell" window. It shows the execution of the code, with the output:

```
>>> ===== RESTART =====
>>>
This statement has a single space+tab Indentation
This statement has a single space+tab Indentation
>>>
```

The status bar at the bottom right of the shell indicates "Ln: 7 Col: 4".

Python Coding Style:

- Use 4 spaces per indentation and no tabs.

- Do not mix tabs and spaces. Tabs create confusion and it is recommended to use only spaces.
- Maximum line length : 79 characters which help users with a small display.

Python Reserve words:

The following identifiers are used as reserved words of the language, and cannot be used as ordinary identifiers.

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	el	if	or	yield
assert	else	import	pass	
break	except	in	raise	

- Use blank lines to separate top-level function and class definitions and single blank line to separate methods definitions inside a class and larger blocks of code inside functions.
- When possible, put inline comments (should be complete sentences).
- Use spaces around expressions and statements.

Data types and Type casting

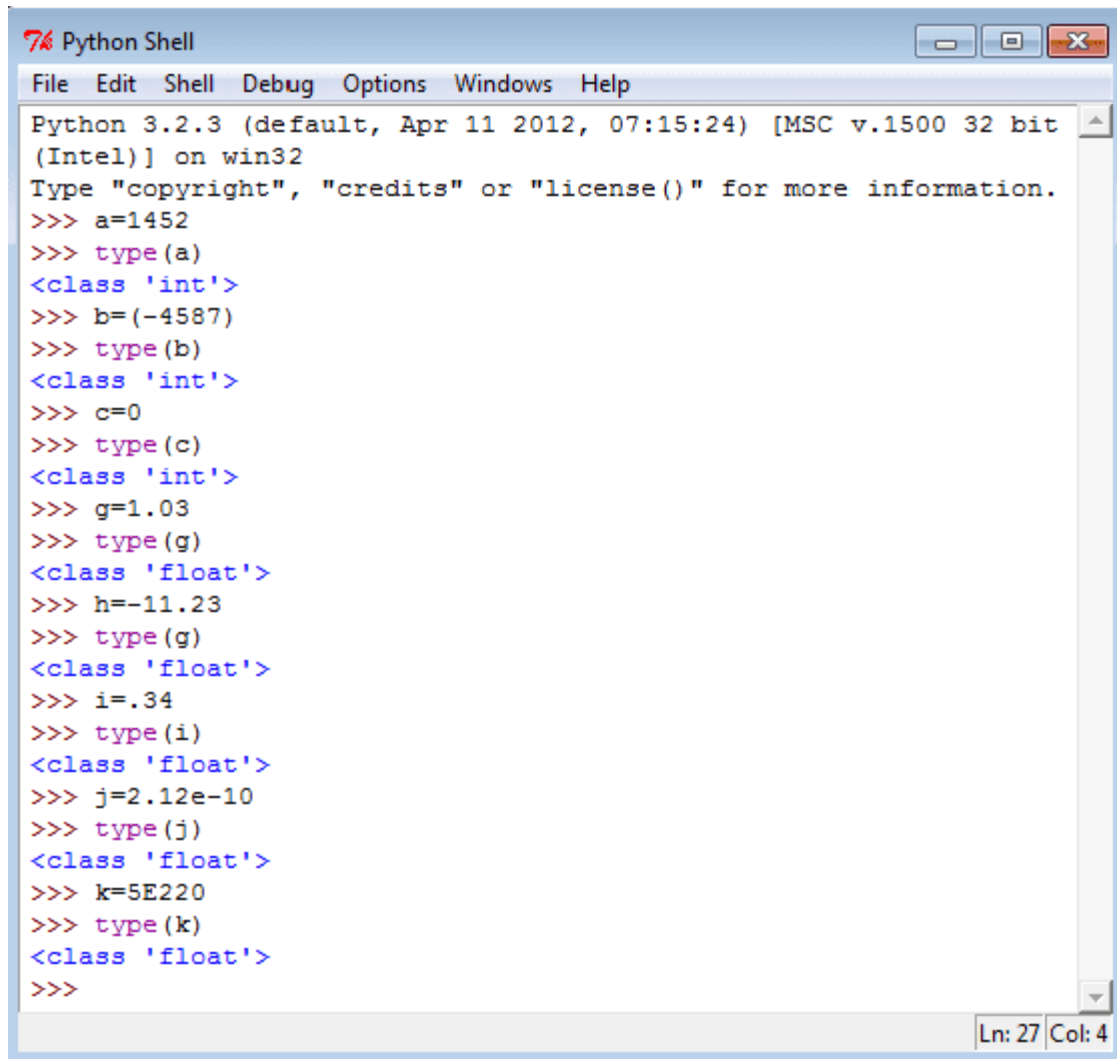
Type represents the kind of value and determines how the value can be used. All data values in Python are encapsulated in relevant object classes. Everything in Python is an object and every object has an identity, a type, and a value. Like another object-oriented language such as Java or C++, there are several data types which are built into Python. Extension modules which are written in C, Java, or other languages can define additional types.

To determine a variable's type in Python you can use the `type()` function. The value of some objects can be changed. Objects whose value can be changed are called mutable and objects whose value is unchangeable (once they are created) are called immutable. Here are the details of Python data types

Numbers

Numbers are created by numeric literals. Numeric objects are immutable, which means when an object is created its value cannot be changed.

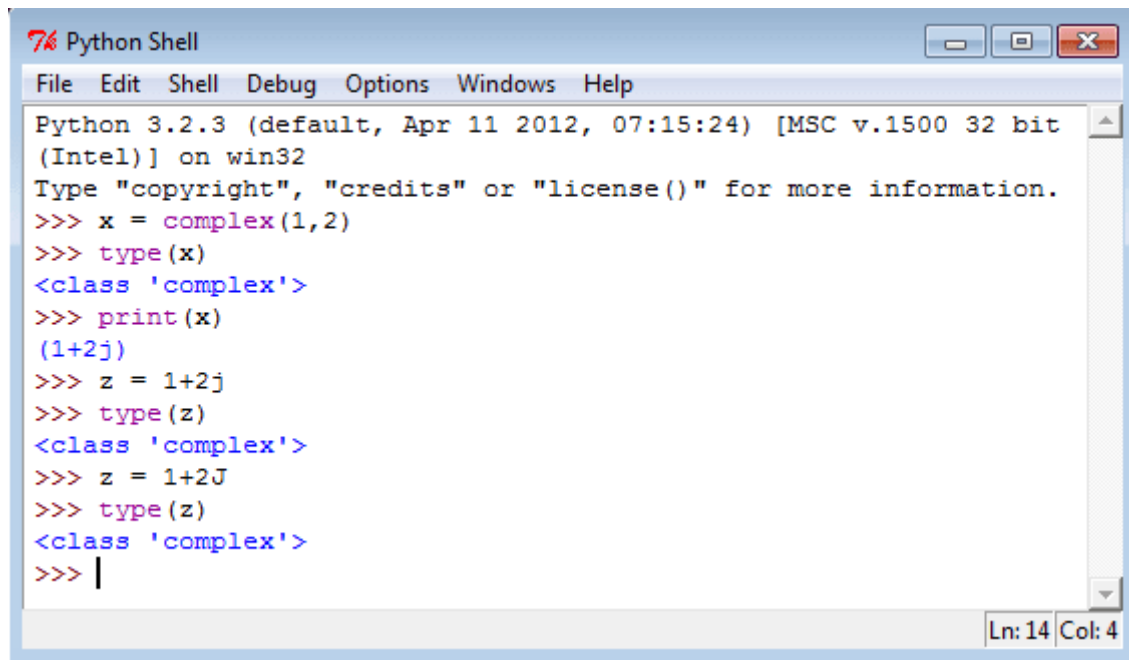
Python has three distinct numeric types: ***integers, floating point numbers, and complex numbers***. Integers represent negative and positive integers without fractional parts whereas floating point numbers represents negative and positive numbers with fractional parts. In addition, Booleans are a subtype of plain integers. See the following statements in Python shell.



```
Python Shell
File Edit Shell Debug Options Windows Help
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit
(Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> a=1452
>>> type(a)
<class 'int'>
>>> b=(-4587)
>>> type(b)
<class 'int'>
>>> c=0
>>> type(c)
<class 'int'>
>>> g=1.03
>>> type(g)
<class 'float'>
>>> h=-11.23
>>> type(h)
<class 'float'>
>>> i=.34
>>> type(i)
<class 'float'>
>>> j=2.12e-10
>>> type(j)
<class 'float'>
>>> k=5E220
>>> type(k)
<class 'float'>
>>>
```

Ln: 27 Col: 4

Mathematically, a complex number (generally used in engineering) is a number of the form $A+Bi$ where i is the imaginary number. Complex numbers have a real and imaginary part. Python supports complex numbers either by specifying the number in (real + imag J) or (real + imag j) form or using a built-in method `complex(x, y)`. See the following statements in Python Shell.

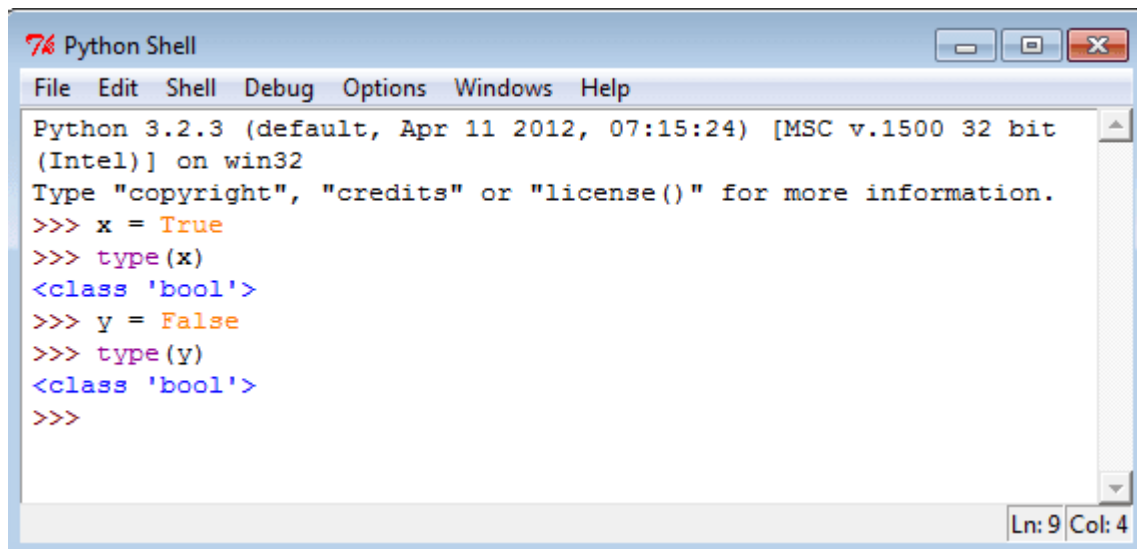
A screenshot of a Python Shell window titled "Python Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Windows", and "Help". The main text area shows the following code and output:

```
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> x = complex(1,2)
>>> type(x)
<class 'complex'>
>>> print(x)
(1+2j)
>>> z = 1+2j
>>> type(z)
<class 'complex'>
>>> z = 1+2J
>>> type(z)
<class 'complex'>
>>> |
```

The status bar at the bottom right indicates "Ln: 14 Col: 4".

Boolean (bool)

The simplest build-in type in Python is the bool type, it represents the truth values False and True. See the following statements in Python shell.

A screenshot of a Python Shell window titled "Python Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Windows", and "Help". The main text area shows the following code and output:

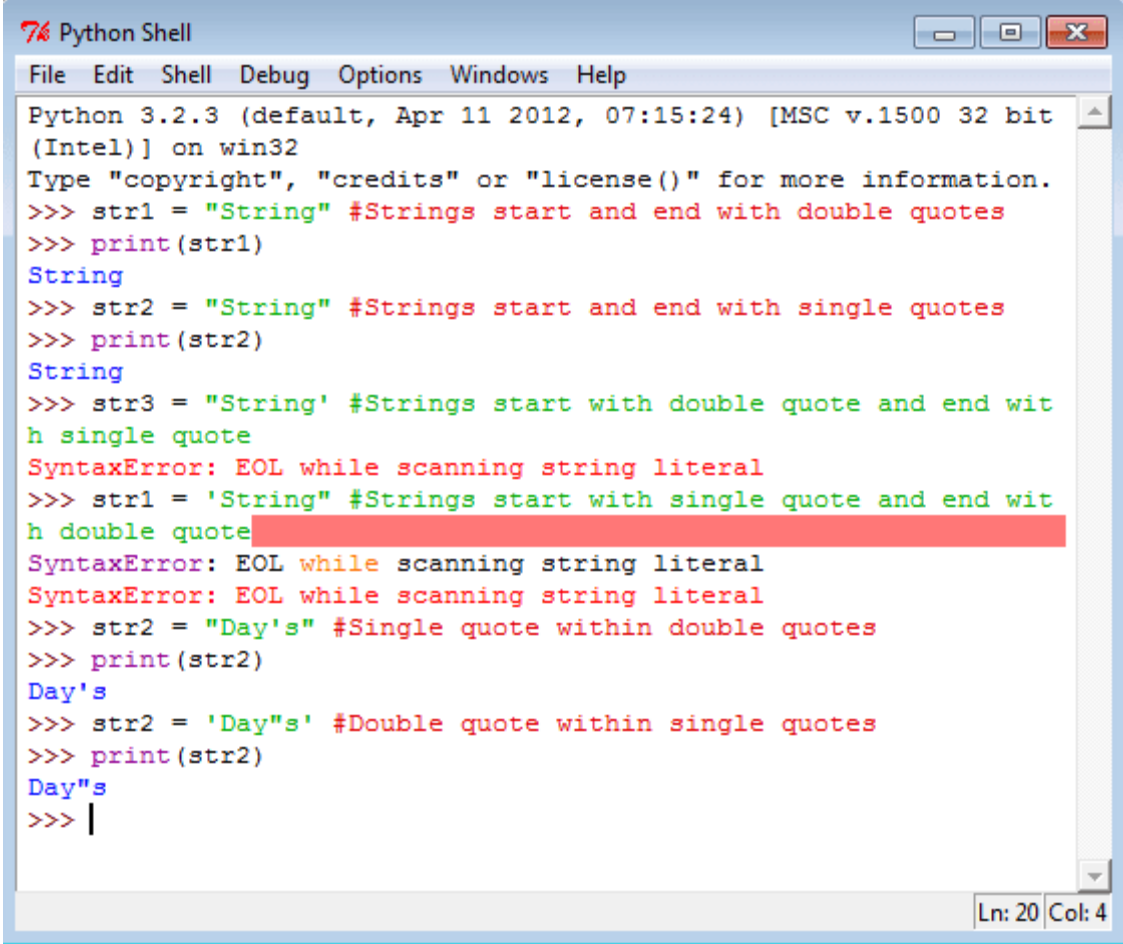
```
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> x = True
>>> type(x)
<class 'bool'>
>>> y = False
>>> type(y)
<class 'bool'>
>>>
```

The status bar at the bottom right indicates "Ln: 9 Col: 4".

Strings

In Python, a string type object is a sequence (left-to- right order) of characters. Strings start and end with single or double quotes Python strings are immutable. Single and double

quoted strings are same and you can use a single quote within a string when it is surrounded by double quote and vice versa. Declaring a string is simple, see the following statements.

A screenshot of a Python Shell window titled "Python Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Windows", and "Help". The main text area shows the following code and output:

```
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> str1 = "String" #Strings start and end with double quotes
>>> print(str1)
String
>>> str2 = "String" #Strings start and end with single quotes
>>> print(str2)
String
>>> str3 = "String' #Strings start with double quote and end with single quote
SyntaxError: EOL while scanning string literal
>>> str1 = 'String" #Strings start with single quote and end with double quote
SyntaxError: EOL while scanning string literal
SyntaxError: EOL while scanning string literal
>>> str2 = "Day's" #Single quote within double quotes
>>> print(str2)
Day's
>>> str2 = 'Day"s' #Double quote within single quotes
>>> print(str2)
Day"s
>>> |
```

The status bar at the bottom right shows "Ln: 20 Col: 4".

Special characters in strings

The backslash (\) character is used to introduce a special character. See the following table.

Escape sequence	Meaning
\n	Newline
\t	Horizontal Tab
\\	Backslash

\'	Single Quote
\"	Double Quote

See the following statements on special characters.

```

Python Shell
File Edit Shell Debug Options Windows Help
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit
(Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> print("The is a backslash (\\) mark.")
The is a backslash (\) mark.
>>> print("This is tab \t key")
This is tab      key
>>> print("These are \'single quotes\'")
These are 'single quotes'
>>> print("These are \"double quotes\"")
These are "double quotes"
>>> print("This is a new line\nNew line")
This is a new line
New line
>>>
Ln: 14 Col: 4

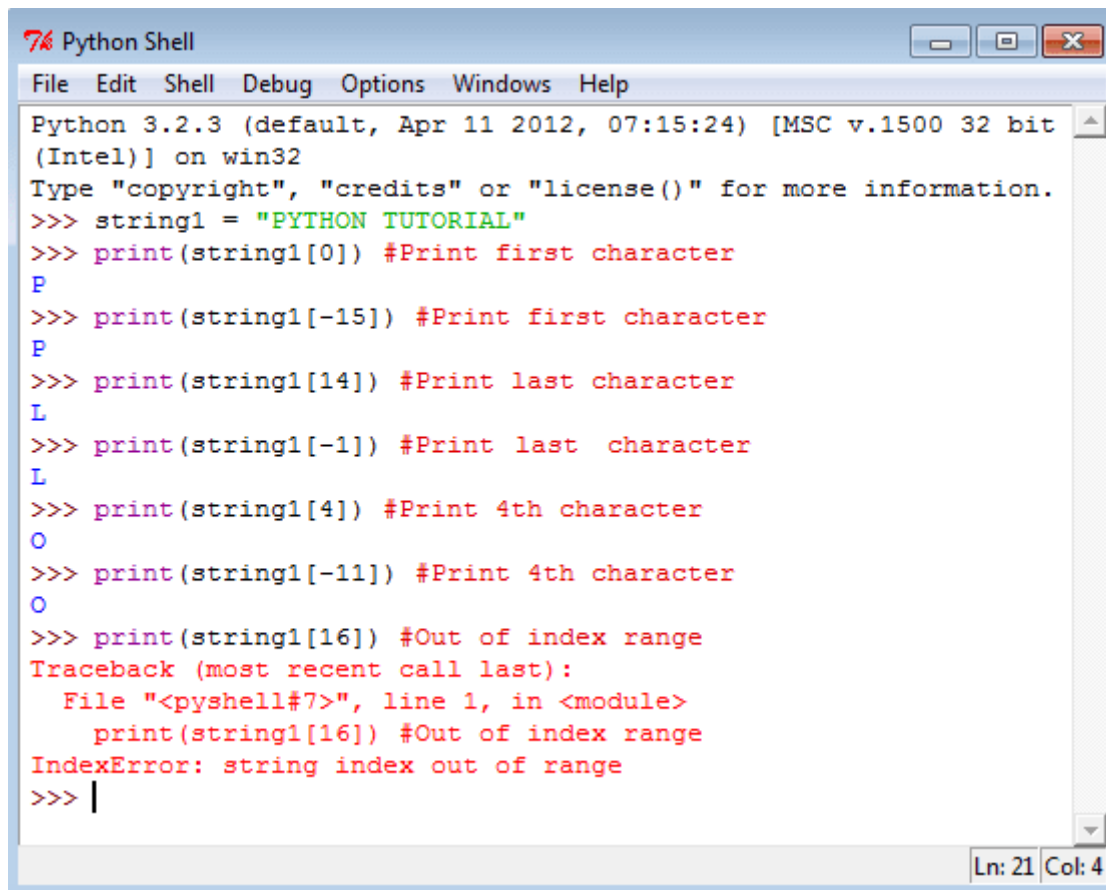
```

String indices and accessing string elements

- Strings are arrays of characters and elements of an array can be accessed using indexing. Indices start with 0 from left side and -1 when starting from right side.
- `string1 = "PYTHON TUTORIAL"`

Character	P	Y	T	H	O	N		T	U	T	O	R	I	A	L
Index (from left)	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Index (from right)	-15	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

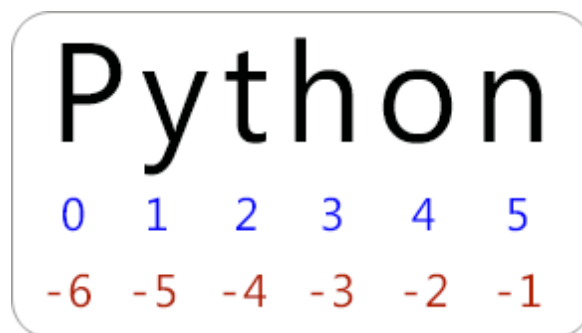
- See the following statements to access single character from various positions.



```
Python Shell
File Edit Shell Debug Options Windows Help
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit
(Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> string1 = "PYTHON TUTORIAL"
>>> print(string1[0]) #Print first character
P
>>> print(string1[-15]) #Print first character
P
>>> print(string1[14]) #Print last character
L
>>> print(string1[-1]) #Print last character
L
>>> print(string1[4]) #Print 4th character
O
>>> print(string1[-11]) #Print 4th character
O
>>> print(string1[16]) #Out of index range
Traceback (most recent call last):
  File "<pyshell#7>", line 1, in <module>
    print(string1[16]) #Out of index range
IndexError: string index out of range
>>> |
```

String Slicing

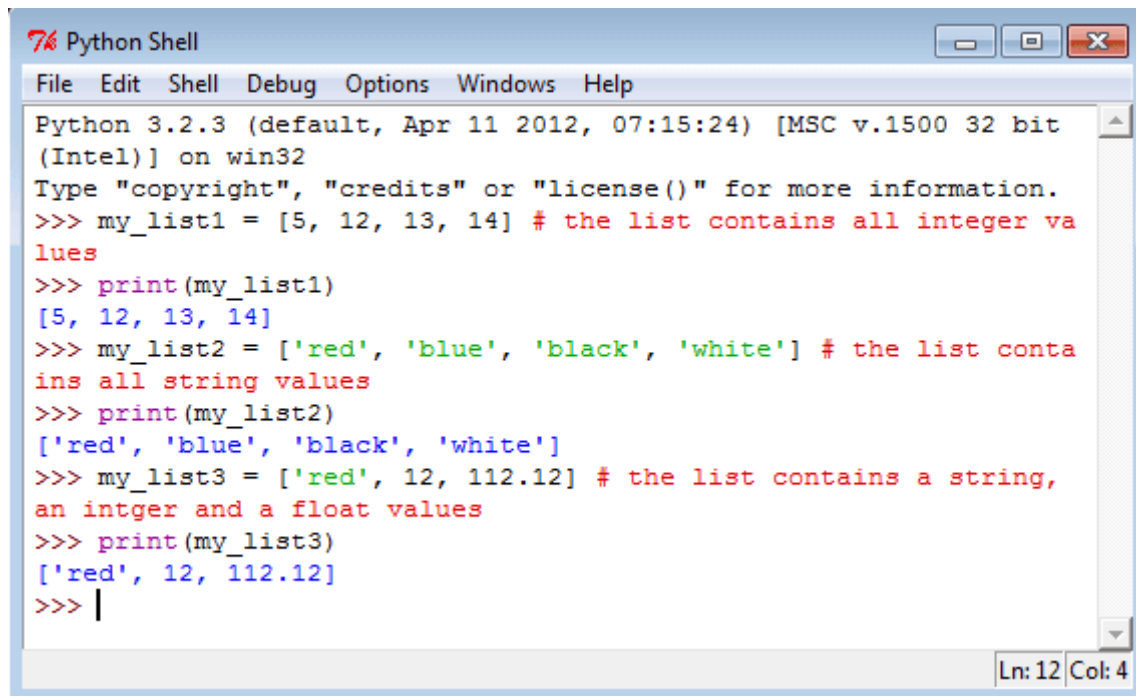
- To cut a substring from a string is called string slicing. Here two indices are used separated by a colon (:). A slice 3:7 means indices characters of 3rd, 4th, 5th and 6th positions. The second integer index i.e. 7 is not included. You can use negative indices for slicing. See the following statements.



Lists

A list is a container which holds comma-separated values (items or elements) between square brackets where Items or elements need not all have the same type.

Creating Lists

A screenshot of a Python Shell window titled "Python Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Windows", and "Help". The main text area shows the following code:

```
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> my_list1 = [5, 12, 13, 14] # the list contains all integer values
>>> print(my_list1)
[5, 12, 13, 14]
>>> my_list2 = ['red', 'blue', 'black', 'white'] # the list contains all string values
>>> print(my_list2)
['red', 'blue', 'black', 'white']
>>> my_list3 = ['red', 12, 112.12] # the list contains a string, an integer and a float values
>>> print(my_list3)
['red', 12, 112.12]
>>> |
```

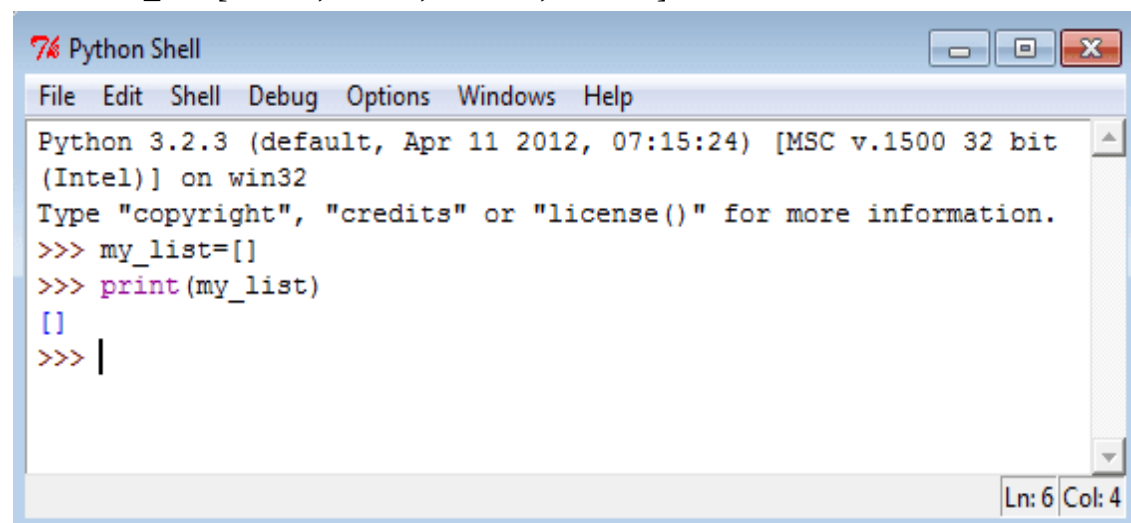
The status bar at the bottom right shows "Ln: 12 Col: 4".

- A list without any element is called an empty list. See the following statements.

List indices

List indices work the same way as string indices, list indices start at 0. If an index has a positive value it counts from the beginning and similarly it counts backward if the index has a negative value. As positive integers are used to index from the left end and negative integers are used to index from the right end, so every item of a list gives two alternatives indices. Let create a list called color_list with four items.

```
color_list=["RED", "Blue", "Green", "Black"]
```

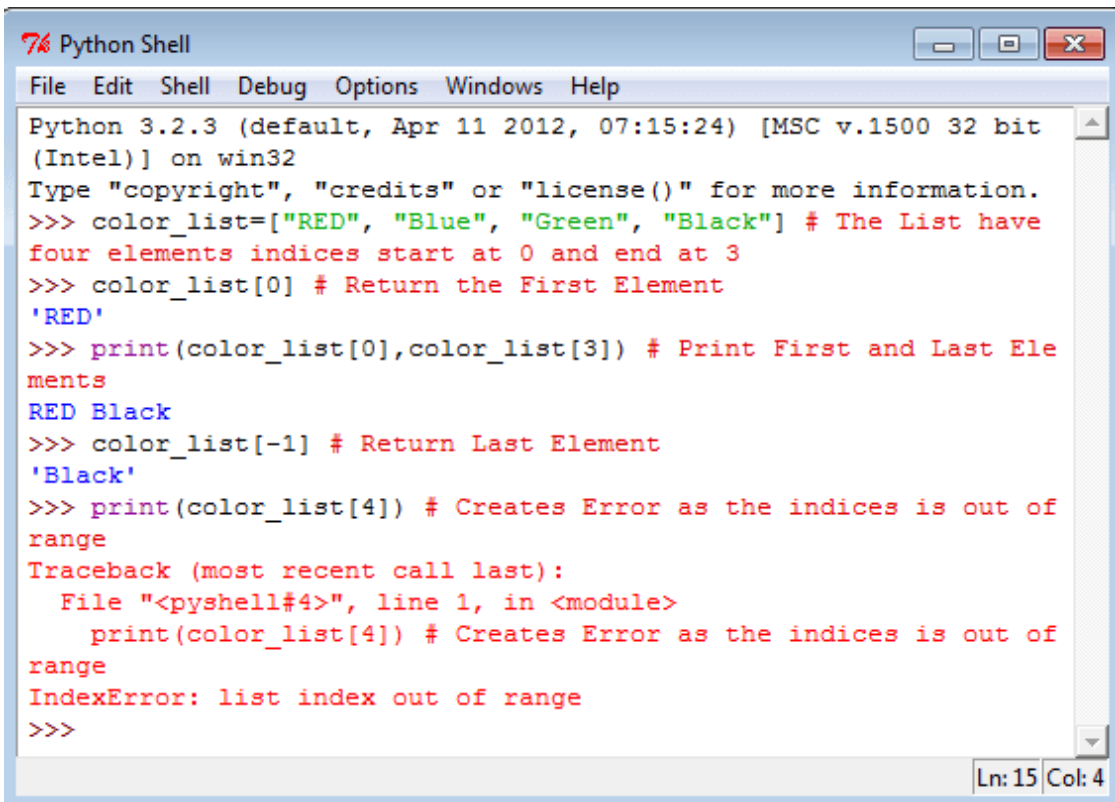
A screenshot of a Python Shell window titled "Python Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Windows", and "Help". The main text area shows the following code:

```
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> my_list=[]
>>> print(my_list)
[]
>>> |
```

The status bar at the bottom right shows "Ln: 6 Col: 4".

Item	RED	Blue	Green	Black
Index (from left)	0	1	2	3
Index (from right)	-4	-3	-2	-1

If you give any index value which is out of range then interpreter creates an error message. See the following statements.



```

Python Shell
File Edit Shell Debug Options Windows Help
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit
(Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> color_list=["RED", "Blue", "Green", "Black"] # The List have
four elements indices start at 0 and end at 3
>>> color_list[0] # Return the First Element
'RED'
>>> print(color_list[0],color_list[3]) # Print First and Last Ele
ments
RED Black
>>> color_list[-1] # Return Last Element
'Black'
>>> print(color_list[4]) # Creates Error as the indices is out of
range
Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    print(color_list[4]) # Creates Error as the indices is out of
range
IndexError: list index out of range
>>>
Ln: 15 Col: 4

```

List Slice

Lists can be sliced like strings and other sequences. The syntax of list slices is easy; sliced_list = List_Name[startIndex:endIndex]

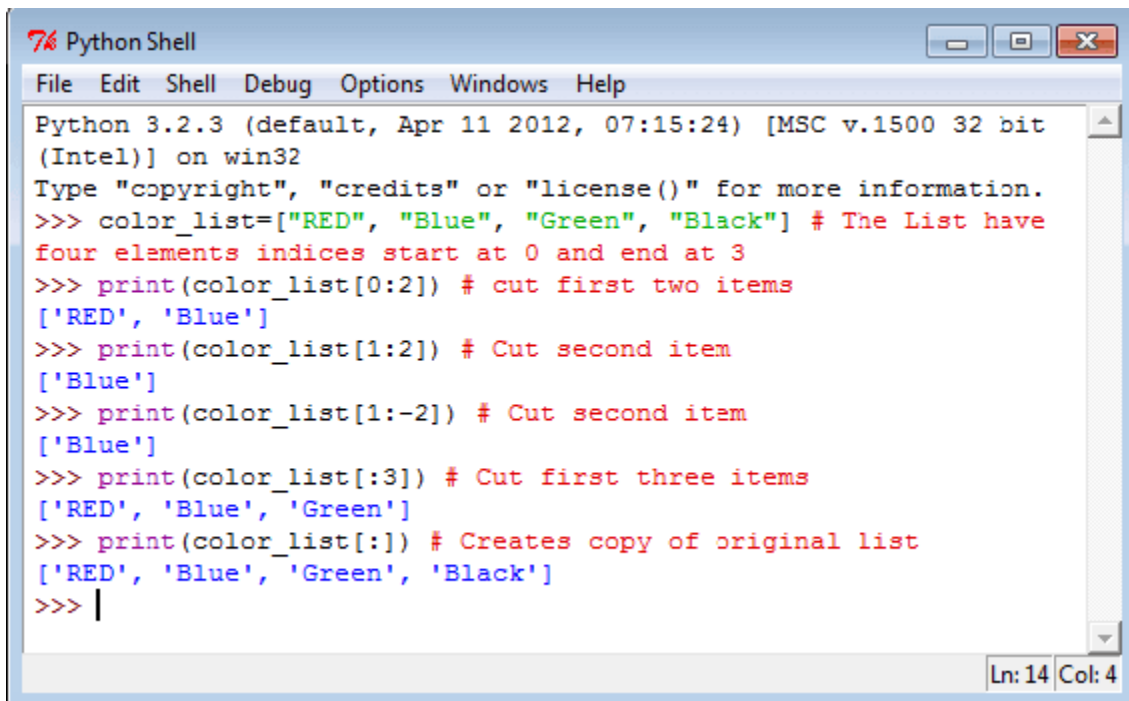
Conditional Statements

Python supports the usual logical conditions from mathematics:

- Equals: a == b
- Not Equals: a != b
- Less than: a < b
- Less than or equal to: a <= b
- Greater than: a > b
- Greater than or equal to: a >= b

e.g if b > a:

```
print("b is greater than a")
```

A screenshot of a Python Shell window titled "Python Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Windows", and "Help". The main text area shows the following code:

```
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> color_list=["RED", "Blue", "Green", "Black"] # The List have four elements indices start at 0 and end at 3
>>> print(color_list[0:2]) # cut first two items
['RED', 'Blue']
>>> print(color_list[1:2]) # Cut second item
['Blue']
>>> print(color_list[1:-2]) # Cut second item
['Blue']
>>> print(color_list[:3]) # Cut first three items
['RED', 'Blue', 'Green']
>>> print(color_list[:]) # Creates copy of original list
['RED', 'Blue', 'Green', 'Black']
>>> |
```

The status bar at the bottom right shows "Ln: 14 Col: 4".

Artificial Intelligence

Lab: 02

Lab 2

Iterative Structures in Python

Python programming language provides following types of loops to handle looping requirements. Python provides three ways for executing the loops. While all the ways provide similar basic functionality, they differ in their syntax and condition checking time.

While Loop:

In python, while loop is used to execute a block of statements repeatedly until a given a condition is satisfied. And when the condition becomes false, the line immediately after the loop in program is executed.

Syntax :

```
while expression:
```

```
    statement(s)
```

All the statements indented by the same number of character spaces after a programming construct are considered to be part of a single block of code. Python uses indentation as its method of grouping statements.

Example:

```
# Python program to illustrate  
  
# while loop  
  
count = 0  
  
while (count < 3):  
  
    count = count + 1  
  
    print("Hello Geek")
```

Single statement while block:

Just like the if block, if the while block consists of a single statement the we can declare the entire loop in a single line as shown below:

Example:

```
# Python program to illustrate
```

```
# Single statement while block

count = 0

while (count == 0): print("Hello Geek")
```

for in Loop:

For loops are used for sequential traversal. For example: traversing a list or string or array etc. In Python, there is no C style for loop, i.e., for (i=0; i<n; i++). There is “for in” loop which is similar to [for each](#) loop in other languages. Let us learn how to use for in loop for sequential traversals.

Syntax:

```
for iterator_var in sequence:

    statements(s)
```

It can be used to iterate over iterators and a range

Example 1

```
# Python program to illustrate

# Iterating over a list

print("List Iteration")

l = ["geeks", "for", "geeks"]

for i in l:

    print(i)
```

Example 2

```
# Iterating over a tuple (immutable)

print("\nTuple Iteration")

t = ("geeks", "for", "geeks")

for i in t:

    print(i)
```

Example 3

```
# Iterating over a String

print("\nString Iteration")

s = "Geeks"

for i in s :

    print(i)
```

Iterating by index of sequences:

We can also use the index of elements in the sequence to iterate. The key idea is to first calculate the length of the list and in iterate over the sequence within the range of this length. See the below example:

```
# Python program to illustrate

# Iterating by index

list = ["geeks", "for", "geeks"]

for index in range(len(list)):

    print list[index]
```

Loop Control Statements:

Loop control statements change execution from its normal sequence. When execution leaves a scope, all automatic objects that were created in that scope are destroyed. Python supports the following control statements.

Continue Statement:

It returns the control to the beginning of the loop.

```
# Prints all letters except 'e' and 's'

for letter in 'geeksforgeeks':

    if letter == 'e' or letter == 's':

        continue
```

```
print 'Current Letter :', letter
```

```
var = 10
```

Break Statement:

It brings control out of the loop

```
for letter in 'geeksforgeeks':
```

```
    # break the loop as soon it sees 'e'
```

```
    # or 's'
```

```
    if letter == 'e' or letter == 's':
```

```
        break
```

```
print 'Current Letter :', letter
```

Python Functions

A function is a block of code which only runs when it is called.

You can pass data, known as parameters, into a function.

A function can return data as a result.

Creating a Function

In Python a function is defined using the `def` keyword:

Example

```
def my_function():  
    print("Hello from a function")
```

Calling a Function

To call a function, use the function name followed by parenthesis:

Example

```
def my_function():  
    print("Hello from a function")  
  
my_function()
```

Parameters

Information can be passed to functions as parameter.

Parameters are specified after the function name, inside the parentheses. You can add as many parameters as you want, just separate them with a comma.

The following example has a function with one parameter (fname). When the function is called, we pass along a first name, which is used inside the function to print the full name:

Example

```
def my_function(fname):  
    print(fname + " Refsnes")
```

```
my_function("Emil")
```

```
my_function("Tobias")
```

```
my_function("Linus")
```

Default Parameter Value

The following example shows how to use a default parameter value.

If we call the function without parameter, it uses the default value:

Example

```
def  
    my_functi  
on(count  
y =
```

```
"Norway"
```

```
): print("I  
am from "  
+ country)
```

```
my_function("Sweden")
```

```
my_function("India")
```

```
my_function()
```

```
my_function("Brazil")
```

Passing a List as a Parameter

You can send any data types of parameter to a function (string, number, list, dictionary etc.), and it will be treated as the same data type inside the function.

E.g. if you send a List as a parameter, it will still be a List when it reaches the function:

Example

```
def my_function(food):  
    for x in food:  
        print(x)
```

```
fruits = ["apple", "banana", "cherry"] my_function(fruits)
```

Return Values

To let a function return a value, use the `return` statement:

Example

```
def my_function(x):  
    return 5 * x
```

```
print(my_function(3))  
print(my_function(5))  
print(my_function(9))
```

Keyword Arguments

You can also send arguments with the *key = value* syntax. This way the order of the arguments does not matter.

Example

```
def my_function(child3, child2, child1):  
    print("The youngest child is " + child3)  
  
my_function(child1 = "Emil", child2 = "Tobias", child3 = "Linus")
```

Python Classes/Objects

Python is an object oriented programming language.

Almost everything in Python is an object, with its properties and methods. A Class is like an object constructor, or a "blueprint" for creating objects.

Create a Class

To create a class,

use the

keyword

class:

Example

Create a class named MyClass, with a property named x:

```
class MyClass: x = 5
```

Create Object

Now we can use the class named myClass to create objects:

Example

Create an object named p1, and print the value of x:

```
p1 = MyClass() print(p1.x)
```

The `__init__()` Function

The examples above are classes and objects in their simplest form, and are not really useful in real life applications.

To understand the meaning of classes we have to understand the built-in `__init__()` function.

All classes have a function called `__init__()`, which is always executed when the class is being initiated.

Use the `__init__()` function to assign values to object properties, or other operations that are necessary to do when the object is being created:

Example

Create a class named Person, use the `__init__()` function to assign values for name and age:

```
class Person:  
def __init__(self, name, age): self.name = name
```

```
self.age = age
```

```
p1 = Person("John", 36)
```

```
print(p1.name)
```

```
print(p1.age)
```

Note: The `__init__()` function is called automatically every time the class is being used to create a new object.

Object Methods

Objects can also contain methods. Methods in objects are functions that belong to the object.

Let us create

a method in

the Person

class:

Example

Insert a function that prints a greeting, and execute it on the p1 object:

```
class Person:  
    def __init__(self, name, age): self.name = name
```

```
    self.age = age
```

```
    def myfunc(self):  
        print("Hello my name is " + self.name)
```

```
p1 = Person("John", 36) p1.myfunc()
```

Artificial Intelligence

Lab: 03

Lab 3

1. Write a Python program to find those numbers which are divisible by 7 and multiple of 5, between 1500 and 2700 (both included).
2. Write a Python program to convert temperatures to and from Celsius, Fahrenheit. [Formula: $c/5 = f-32/9$ [where c = temperature in Celsius and f = temperature in Fahrenheit]

Expected Output:

60°C is 140 in Fahrenheit 45°F is 7 in Celsius

3. Write a Python program to guess a number between 1 to 9.

Note: User is prompted to enter a guess. If the user guesses wrong then the prompt appears again until the guess is correct, on successful guess, user will get a "Well guessed!" message, and the program will exit.

4. Write a Python program to construct the following pattern, using a nested for loop.

```
*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*
*
```

5. Write a Python program that accepts a word from the user and reverse it.
6. Write a Python program to count the number of even and odd numbers from a series of numbers.

Sample numbers : numbers = (1, 2, 3, 4, 5, 6, 7, 8, 9)

Expected Output :

Number of even numbers : 5 Number of odd numbers : 4

7. Write a Python program that prints each item and its corresponding type from the following list.

Sample List : datalist = [1452, 11.23, 1+2j, True, 'w3resource', (0, -1), [5, 12], {"class": 'V', "section": 'A'}]

8. Write a Python program that prints all the numbers from 0 to 6 except 3 and 6.
Note : Use 'continue' statement.

Expected Output : 0 1 2 4 5

9. Write a Python program to get the Fibonacci series between 0 to 50. Note : The Fibonacci Sequence is the series of numbers :

0, 1, 1, 2, 3, 5, 8, 13, 21,

Every next number is found by adding up the two numbers before it.

Expected Output : 1 1 2 3 5 8 13 21 34

Write a Python program which iterates the integers from 1 to 50. For multiples of three print "Fizz" instead of the number and for the multiples of five print "Buzz". For numbers which are multiples of both three and five print "FizzBuzz".

Sample Output : fizzbuzz

1

2

fizz 4

buzz

10. Write a Python program which takes two digits m (row) and n (column) as input and generates a two-dimensional array. The element value in the i-th row and j-th column of the array should be $i*j$.

Note :

$i = 0, 1, \dots, m-1$

$j = 0, 1, \dots, n-1$.

Test Data : Rows = 3, Columns = 4

Expected Result : $[[0, 0, 0, 0], [0, 1, 2, 3], [0, 2, 4, 6]]$

11. Write a Python program that accepts a sequence of lines (blank line to terminate) as input and prints the lines as output (all characters in lower case).

12. Write a Python program which accepts a sequence of comma separated 4 digit binary numbers as its input and print the numbers that are divisible by 5 in a comma separated sequence.

Sample Data : 0100,0011,1010,1001,1100,1001

Expected Output : 1010

13. Write a Python program that accepts a string and calculate the number of digits and letters.

Sample Data : Python 3.2 Expected Output :

Letters 6

Digits 2

14. Write a Python program to check the validity of password input by users. Validation

:

- At least 1 letter between [a-z] and 1 letter between [A-Z].
- At least 1 number between [0-9].
- At least 1 character from [\$#@].
- Minimum length 6 characters.
- Maximum length 16 characters.

Artificial Intelligence

Lab: 04

Lab 4

Question:1

Implement Stack using Python.

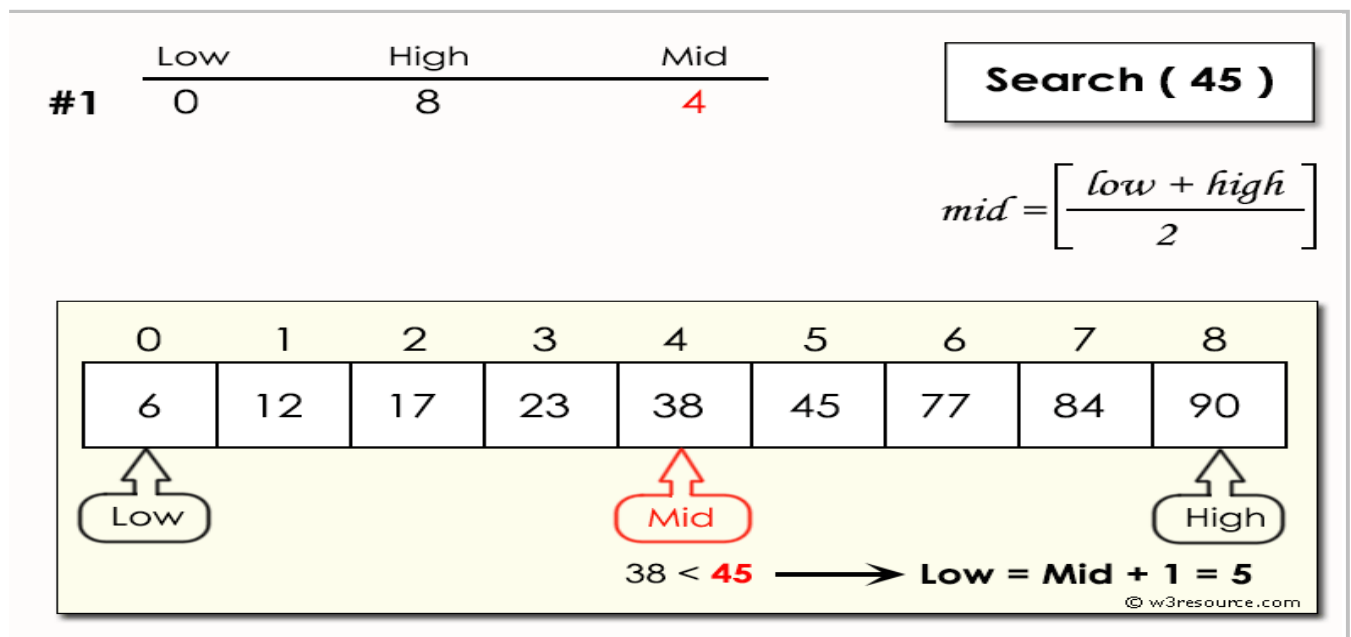
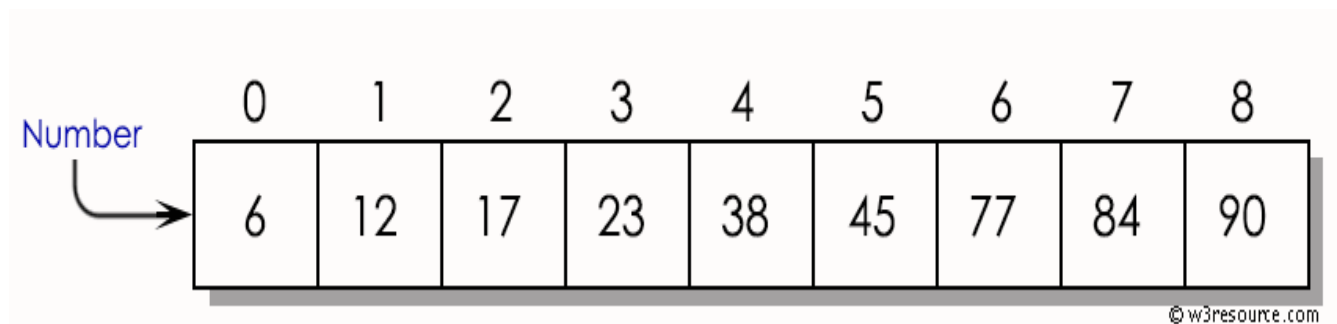
Question:2

Implement Queue using Python.

Question:3

In computer science, a binary search or half-interval search algorithm finds the position of a target value within a sorted array. The binary search algorithm can be classified as a dichotomies divide-and-conquer search algorithm and executes in logarithmic time.

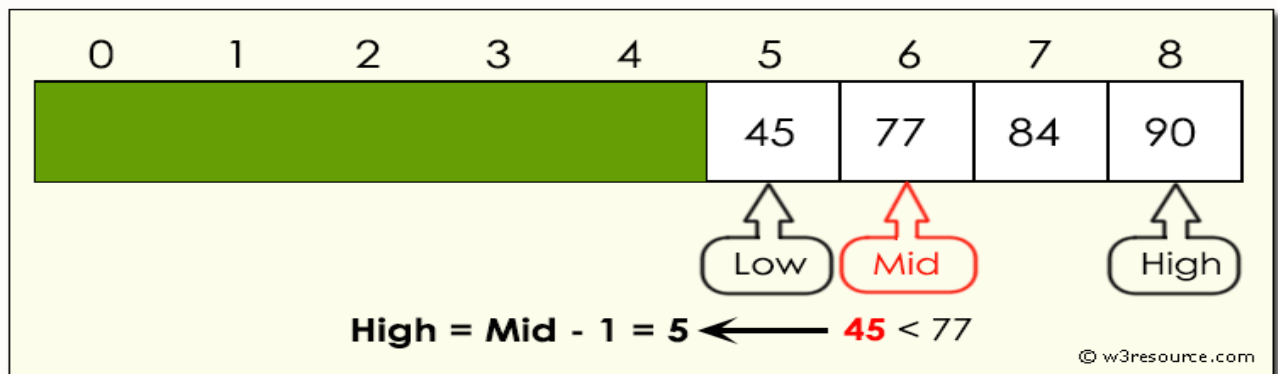
Step by step Example:



	Low	High	Mid
#1	0	8	4
#2	5	8	6

Search (45)

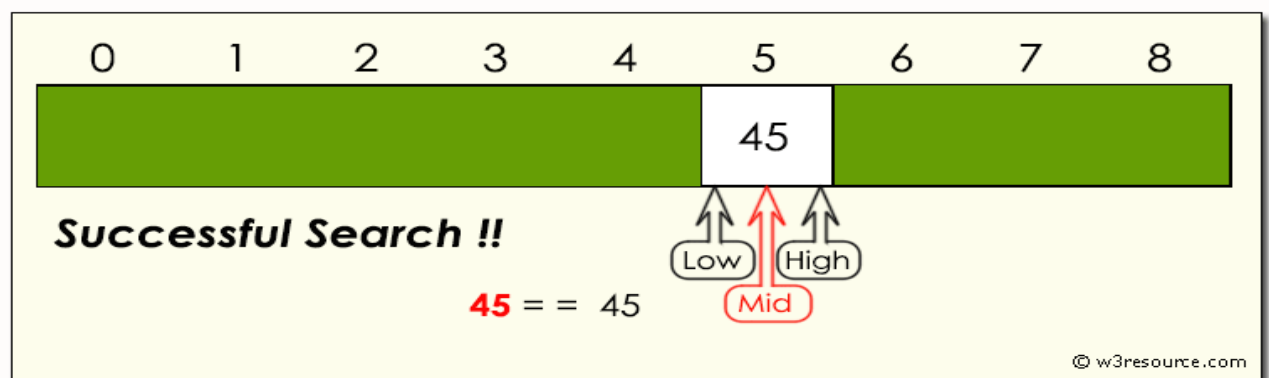
$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$



	Low	High	Mid
#1	0	8	4
#2	5	8	6
#3	5	5	5

Search (45)

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$



Artificial Intelligence

Lab: 05

Lab 5

Introduction:

Breadth-first search (BFS) is an algorithm for traversing or searching tree or graph data structures. It starts at the tree root and explores all of the neighbor nodes at the present depth prior to moving on to the nodes at the next depth level. It uses a queue (First In First Out) instead of a stack and it checks whether a vertex has been discovered before enqueueing the vertex.

Relevant Lecture Readings:

1. Revise Lecture given on BFS.

2. Objective of the Experiment

After completing this lab, the student should be able to:

- Clearly understand Implementation of Breadth First Search.

3. Concept Map

3.1 Breadth First Search (BFS)

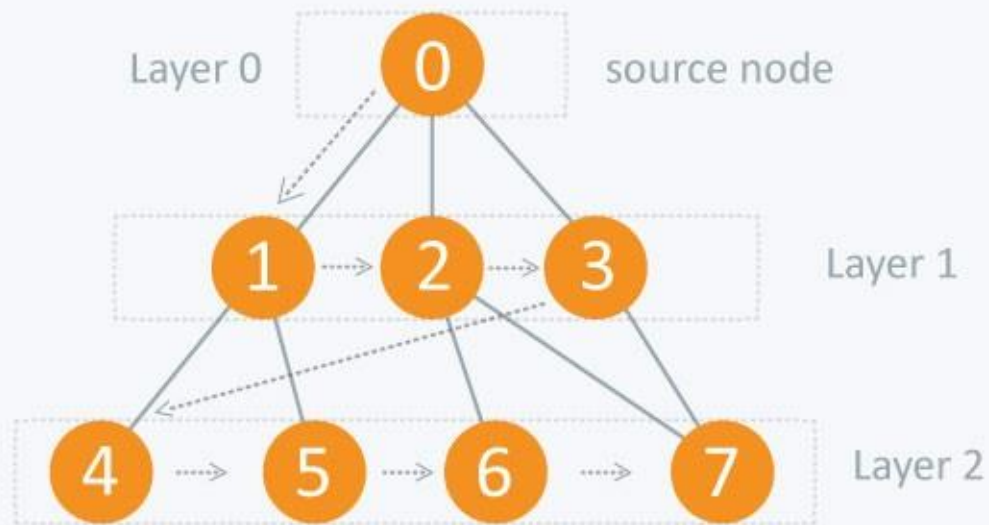
BFS is a traversing algorithm where you should start traversing from a selected node (source or starting node) and traverse the graph layer wise thus exploring the neighbour nodes (nodes which are directly

connected to source node). You must then move towards the next-level neighbour

nodes. As the name BFS suggests, you are required to traverse the graph breadthwise as

follows:

1. First move horizontally and visit all the nodes of the current layer.
2. Move to the next layer.



Pseudocode:

BFS (G, s) //Where G is the graph and s is the source node let Q be queue.
 Q.enqueue(s) //Inserting s in queue until all its neighbor vertices are marked.

mark s as visited. while (Q is not empty)

 //Removing that vertex from queue, whose neighbor will be visited now v = Q.dequeue()

 //processing all the neighbours of v for all
 neighbours w of v in Graph G

 if w is not visited

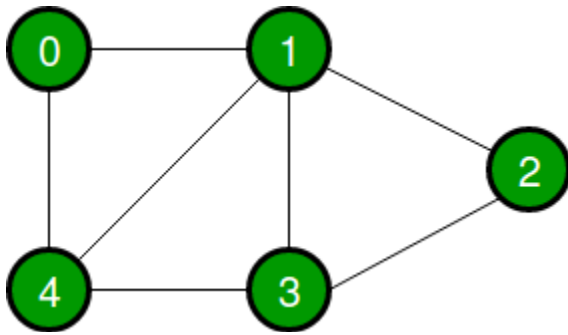
 Q.enqueue(w) //Stores w in Q to further visit its neighbor mark w as visited.

4. Task:1

Attempt the following questions.

4.1 Problem description

Write a program that will implement the following graph in python.



5. Procedure & Tools

5.1 Tools

Anaconda, python language

5.2 Walkthrough Task

[Expected time = 40 mins]

5.2.1 Implementation of BFS in c++.

```
#include<iostream>
#include <list>

using namespace std;

// This class represents a directed graph using
// adjacency list representation
class Graph
{
    int V; // No. of vertices

    // Pointer to an array containing adjacency
    // lists
    list<int> *adj;
public:
    Graph(int V); // Constructor

    // function to add an edge to graph
    void addEdge(int v, int w);

    // prints BFS traversal from a given source s
    void BFS(int s);
};
```

```

Graph::Graph(int V)
{
    this->V = V;
    adj = new list<int>[V];
}

void Graph::addEdge(int v, int w)
{
    adj[v].push_back(w); // Add w to v's list.
}

void Graph::BFS(int s)
{
    // Mark all the vertices as not visited
    bool *visited = new bool[V];
    for(int i = 0; i < V; i++)
        visited[i] = false;
    // Create a queue for BFS
    list<int> queue;
    // Mark the current node as visited and enqueue it
    visited[s] = true;
    queue.push_back(s);
    // 'i' will be used to get all adjacent
    // vertices of a vertex
    list<int>::iterator i;
    // vertices of a vertex
    list<int>::iterator i;

    while(!queue.empty())
    {
        // Dequeue a vertex from queue and print it
        s = queue.front();
        cout << s << " ";
        queue.pop_front();

        // Get all adjacent vertices of the dequeued
        // vertex s. If a adjacent has not been visited,
        // then mark it visited and enqueue it
        for (i = adj[s].begin(); i != adj[s].end(); ++i)
        {
            if (!visited[*i])
            {
                visited[*i] = true;
                queue.push_back(*i);
            }
        }
    }
}

```

```
// Driver program to test methods of graph class
int main()
{
    // Create a graph given in the above diagram
    Graph g(4);
    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 2);
    g.addEdge(2, 0);
    g.addEdge(2, 3);
    g.addEdge(3, 3);

    cout << "Following is Breadth First Traversal "
         << "(starting from vertex 2) \n";
    g.BFS(2);

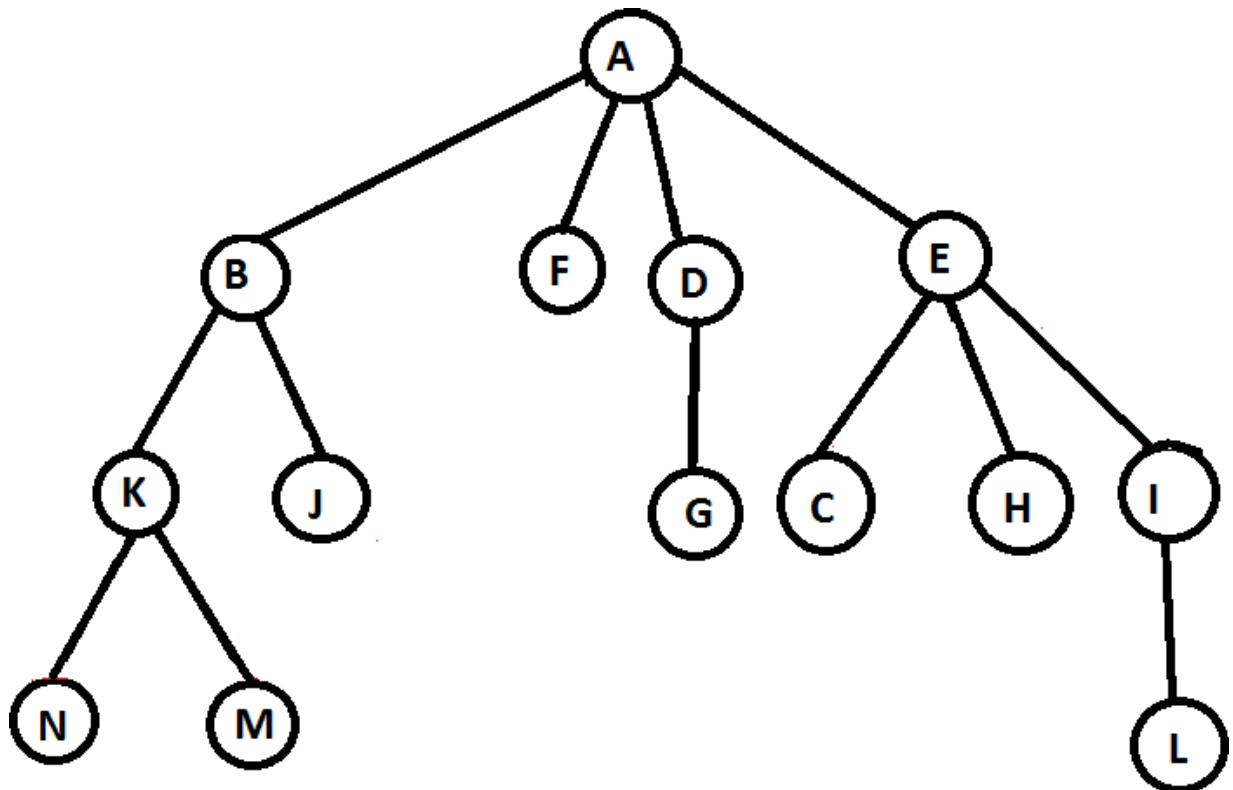
    return 0;
}
```

6. Tasks

6.1 Task 2

[Expected time = 40 mins]

You are given the following tree whose starting node is A and Goal node is G



You are required to implement the following graph in python and then apply the following search(Stop the search when goal is achieved)

a) Breadth First Search using Queue.

1.1 Task3: Implement priority Queue.

1.2 Outcomes.

After completing this lab, students will know the implementation of BFS in Python. Students will be able to solve problems using BFS.

Artificial Intelligence

Lab: 06

Lab 6

Depth-first search (DFS) is an algorithm for traversing or searching tree or graph data structures. The algorithm starts at the root node (selecting some arbitrary node as the root node in the case of a graph) and explores as far as possible along each branch before backtracking.

The basic idea is as follows:

Pick a starting node and push all its adjacent nodes into a stack.

Pop a node from stack to select the next node to visit and push all its adjacent nodes into a stack. Repeat this process until the stack is empty. However, ensure that the nodes that are visited are marked. This will prevent you from visiting the same node more than once. If you do not mark the nodes that are visited and you visit the same node more than once, you may end up in an infinite loop.

1. Objective of the Experiment

After completing this lab, the student should be able to:

- Clearly understand the difference between Stack and Queue.
- Implementation of DFS in python

2. Concept Map

3.1 Depth First Search

The DFS algorithm is a recursive algorithm that uses the idea of backtracking. It involves exhaustive searches of all the nodes by going ahead, if possible, else by backtracking.

This recursive nature of DFS can be implemented using stacks. The basic idea is as follows:

Pick a starting node and push all its adjacent nodes into a stack.

Pop a node from stack to select the next node to visit and push all its adjacent nodes into a stack.

Repeat this process until the stack is empty. However, ensure that the nodes that are visited are marked.

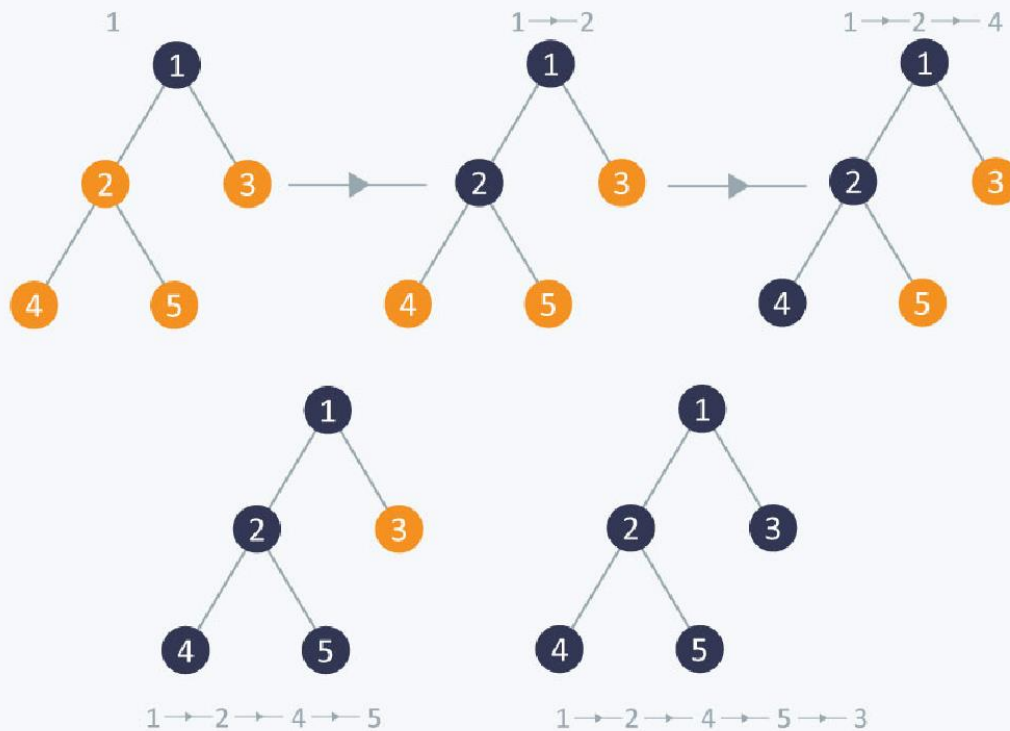
This will prevent you from visiting the same node more than once. If you do not mark the nodes that are visited and you visit the same node more than once, you may end up in an infinite loop.

Pseudocode

```
DFS-iterative (G, s):                                     //Where G is graph and s is source vertex
    let S be stack
    S.push( s )      //Inserting s in stack
    mark s as visited.
    while ( S is not empty):
        //Pop a vertex from stack to visit next
        v = S.top( )
        S.pop( )
        //Push all the neighbours of v in stack that are not visited
        for all neighbours w of v in Graph G:
            if w is not visited :
                S.push( w )
                mark w as visited

DFS-recursive(G, s):
    mark s as visited
    for all neighbours w of s in Graph G:
        if w is not visited:
            DFS-recursive(G, w)
```

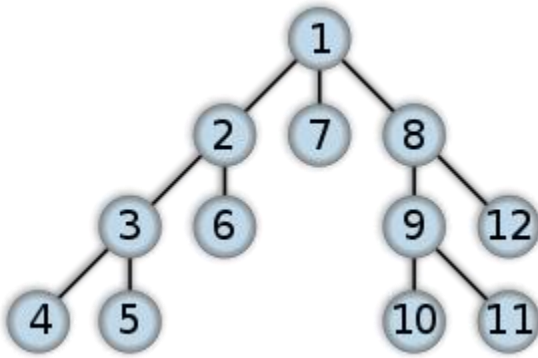
DFS



4.

4.1 Problem description

Write a program to implement the graph in python.



5. Procedure & Tools

5.1 Tools

Anaconda, python language

5.2 Walkthrough Task

[Expected time = 40 mins]

5.2.1 Implementation of DFS in C++

```

#include<iostream>
#include<list>
using namespace std;

// Graph class represents a directed graph
// using adjacency list representation
class Graph
{
    int V;    // No. of vertices

    // Pointer to an array containing
    // adjacency lists
    list<int> *adj;

    // A recursive function used by DFS
    void DFSUtil(int v, bool visited[]);
public:
    Graph(int V);    // Constructor

    // function to add an edge to graph
    void addEdge(int v, int w);

    // DFS traversal of the vertices
    // reachable from v
    void DFS(int v);
};
  
```

```

Graph::Graph(int V)
{
    this->V = V;
    adj = new list<int>[V];
}

void Graph::addEdge(int v, int w)
{
    adj[v].push_back(w); // Add w to v's list.
}

void Graph::DFSUtil(int v, bool visited[])
{
    // Mark the current node as visited and
    // print it
    visited[v] = true;
    cout << v << " ";

    // Recur for all the vertices adjacent
    // to this vertex
    list<int>::iterator i;
    for (i = adj[v].begin(); i != adj[v].end(); ++i)
        if (!visited[*i])
            DFSUtil(*i, visited);
}

void Graph::DFS(int v)
{
    // Mark all the vertices as not visited
    bool *visited = new bool[V];
    for (int i = 0; i < V; i++)
        visited[i] = false;

    // Call the recursive helper function
    // to print DFS traversal
    DFSUtil(v, visited);
}

```

```

int main()
{
    // Create a graph given in the above diagram
    Graph g(4);
    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 2);
    g.addEdge(2, 0);
    g.addEdge(2, 3);
    g.addEdge(3, 3);

    cout << "Following is Depth First Traversal"
          " (starting from vertex 2) \n";
    g.DFS(2);

    return 0;
}

```

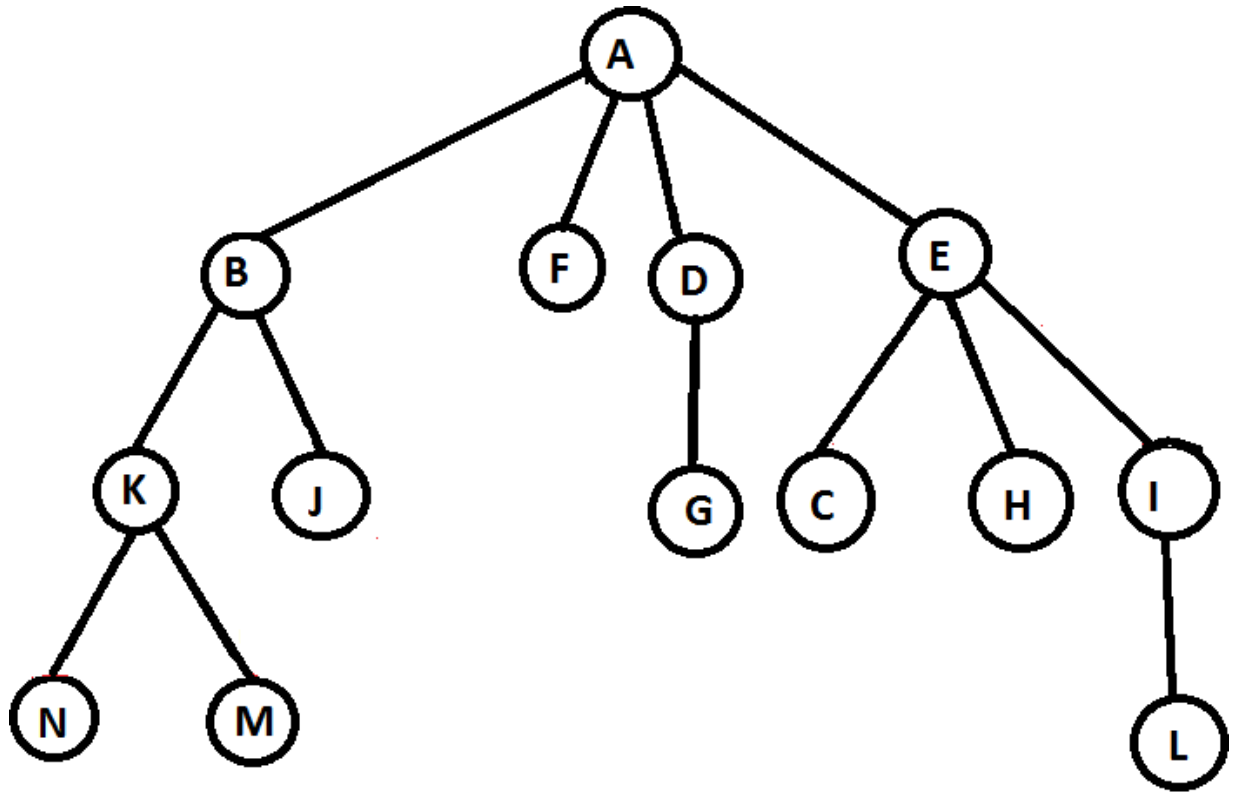
6. Practice Tasks

This section will provide more practice exercise which you need to finish during the lab. You need to finish the task in the required time.

6.1 Practice Task 1

[Expected time = 40 mins]

You are given the following tree whose starting node is **A** and Goal node is **G**.



You are required implement the following graph in python and then apply the following search

a) Depth First Search using Stack.

6.2 Outcomes

After completing this lab, students will know the implementation of DFS in python. Students will be able to solve problems using DFS in python.

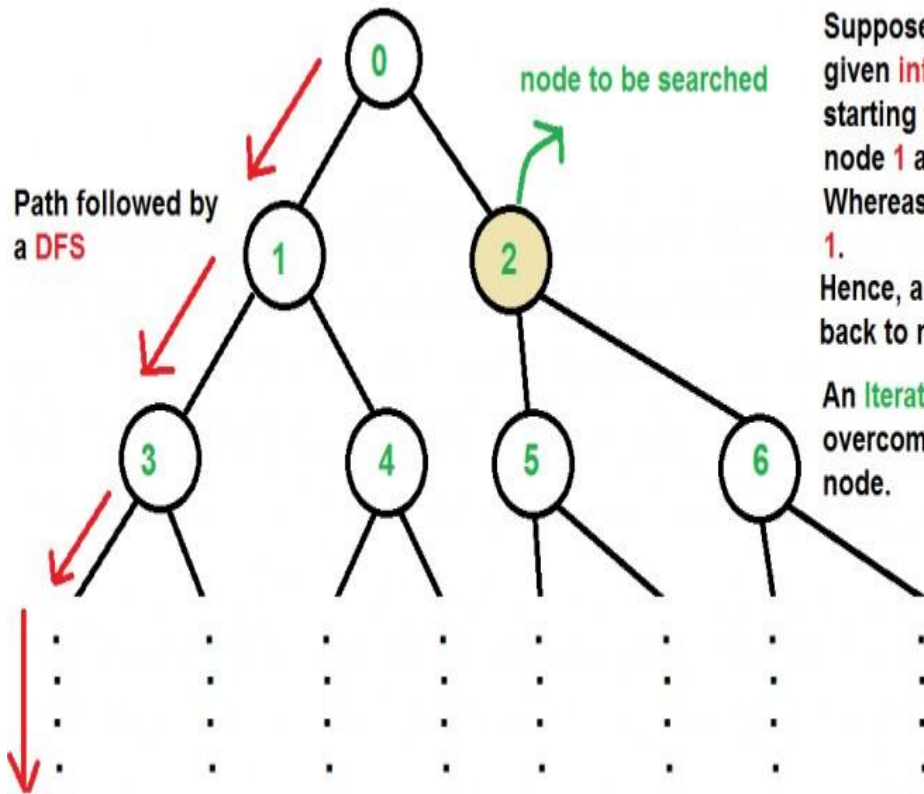
Artificial Intelligence

Lab: 07

Lab 7

There are two common ways to traverse a graph, BFS and DFS. Considering a Tree (or Graph) of huge height and width, both BFS and DFS are not very efficient due to following reasons.

1. **DFS:** first traverses nodes going through one adjacent of root, then next adjacent. The problem with this approach is, if there is a node close to root, but not in first few subtrees explored by DFS, then DFS reaches that node very late. Also, DFS may not find shortest path to a node (in terms of number of edges).



Suppose, we want to find node- '2' of the given infinite undirected graph/tree. A DFS starting from node- 0 will dive left, towards node 1 and so on. Whereas, the node 2 is just adjacent to node 1.

Hence, a DFS wastes a lot of time in coming back to node 2.

An Iterative Deepening Depth First Search overcomes this and quickly find the required node.

2. **BFS:** goes level by level, but requires more space. The space required by DFS is $O(d)$ where d is depth of tree, but space required by BFS is $O(n)$ where n is number of nodes in tree (Why? Note that the last level of tree can have around $n/2$ nodes and second last level $n/4$ nodes and in BFS we need to have every level one by one in queue).

IDDFS combines depth-first search's space-efficiency and breadth-first search's fast search (for nodes closer to root).

2. Objective of the Experiment

After completing this lab, the student should be able to:

- Clearly understand difference between IDS, DLS.
- Better understanding of flaws in BFS & DFS.

3. Concept Map

IDDFS is a combination of both DFS and BFS.

3.1 Depth Limited Search

Depth Limited Search is a basic DFS with a cut-off limit that it stops expanding after.

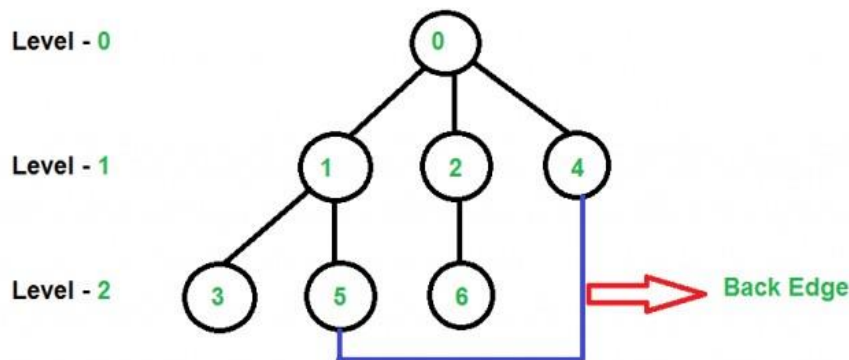
The problem is how to pick a good limit. Sometimes in DFS due to infinite loop the path to the goal node might never be found, in order to combat this we can add a limit to the depth that our search recurses down the tree, this essentially transforms our depth first algorithm into a depth-limited algorithm

3.2 Iterative Deepening Search/ Iterative Deepening Depth First Search

There can be two cases-

a) **When the graph has no cycle:** This case is simple. We can DFS multiple times with different height limits.

b) **When the graph has cycles:** This is interesting as there is no visited flag in IDDFS.



Although, at first sight, it may seem that since there are only 3 levels, so we might think that **Iterative Deepening Depth First Search** of level 3, 4, 5,...and so on will remain same. But, this is not the case. You can see that there is a **cycle** in the above graph, hence **IDDFS** will change for level-3,4,5..and so on.

Figure 1: IDDFS

Depth	Iterative Deepening Depth First Search
0	0
1	0 1 2 4
2	0 1 3 5 2 6 4 5
3	0 1 3 5 4 2 6 4 5 1

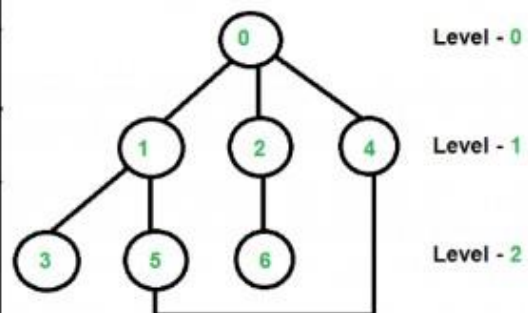


Figure 2: IDDFS search path

A comparison table between DFS, BFS and IDDFS

	Time Complexity	Space Complexity	When to Use ?
DFS	$O(b^d)$	$O(d)$	=> Don't care if the answer is closest to the starting vertex/root. => When graph/tree is not very big/infinite.
BFS	$O(b^d)$	$O(b^d)$	=> When space is not an issue => When we do care/want the closest answer to the root.
IDDFS	$O(b^d)$	$O(bd)$	=> You want a BFS, you don't have enough memory, and somewhat slower performance is accepted. In short, you want a BFS + DFS.

Figure 3: DFS, BFS and IDDFS Comparison

4. Procedure & Tools

4.1 Tools

Anaconda

4.2 Iterative Deepening Search Algorithm

IDDFS calls DFS for different depths starting from an initial value. In every call, DFS is restricted from going beyond given depth. So basically we do DFS in a BFS fashion.

```
// Returns true if target is reachable from
// src within max_depth
bool IDDFS(src, target, max_depth)
    for limit from 0 to max_depth
        if DLS(src, target, limit) == true
            return true
    return false

bool DLS(src, target, limit)
    if (src == target)
        return true;

    // If reached the maximum depth,
    // stop recursing.
```

```

if (limit <= 0)
    return false;

foreach adjacent i of src
    if DLS(i, target, limit-1)
        return true

return false

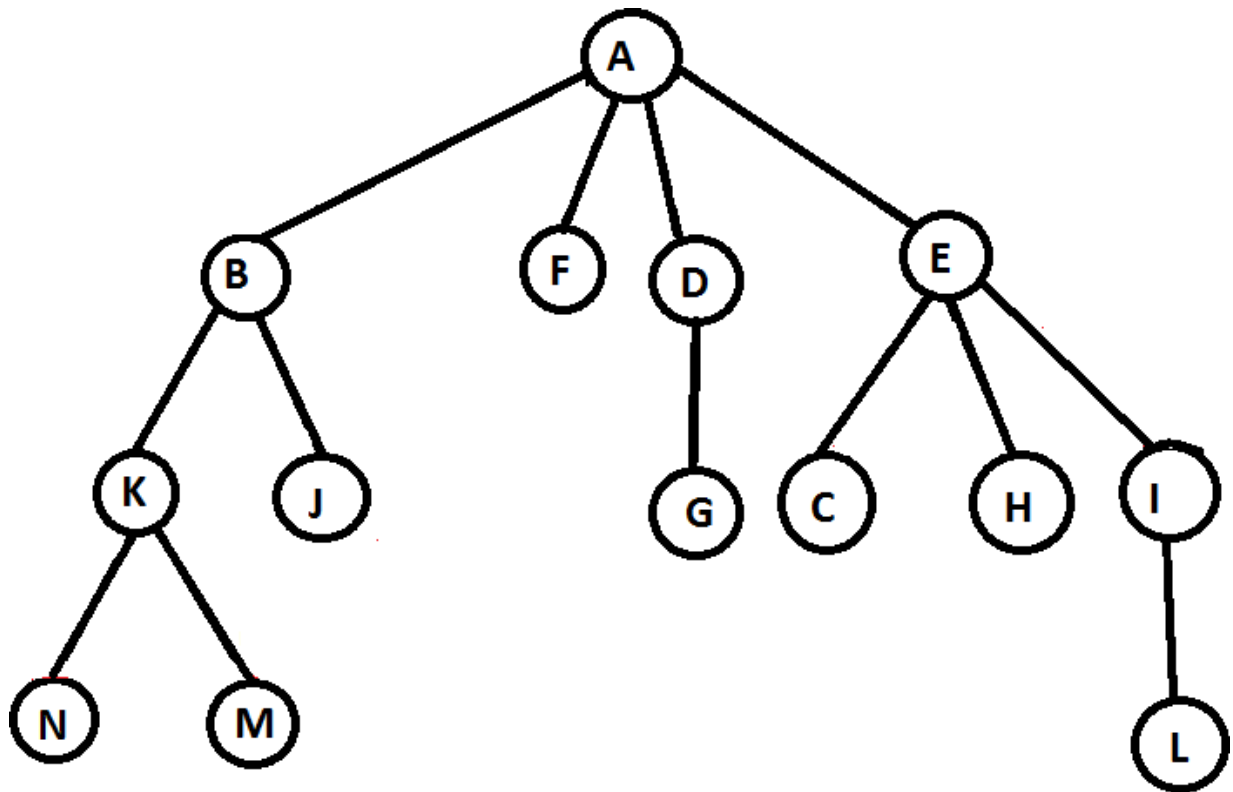
```

An important thing to note is, we visit top level nodes multiple times. The last (or max depth) level is visited once, second last level is visited twice, and so on. It may seem expensive, but it turns out to be not so costly, since in a tree most of the nodes are in the bottom level. So it does not matter much if the upper levels are visited multiple times

Tasks

4.3 Task 1

You are given the following tree whose starting node is **A** and Goal node is **G**.



You are required to apply the following searches and submit in MS Word file.

- Depth Limited Search
- Iterative Deepening Search.
- Compare searches and conclude which search gave the best path in the above situation.

Artificial Intelligence

Lab: 08

Lab 8

Introduction

A* Search algorithm is one of the best and popular technique used in path-finding and graph traversals. The idea is avoid expanding nodes which are expensive.

1. Objective of the Experiment

After completing this lab, the student should be able to:

- Clearly understand difference between uninformed searches and informed searches.
- Better understanding of heuristic function.
- Understand search mechanism where cost is involved.

2. Concept Map

A search using domain-specific knowledge. Suppose that we have a way to estimate how close a state is to the goal, with an evaluation function.

3.1 A* Search

Idea behind A* search is to avoid expanding paths that are already expensive. Cost of node “n” is calculated using following function:

Evaluation function: $f(n) = g(n) + h(n)$.

Where $g(n)$ is the actual cost to reach **n** node from initial state, $h(n)$ is estimated cost from **n** to goal. $h(n)$ is also called heuristic function. $f(n)$ is the estimated cost of path through initial to goal through **n**. So A* has $f(n) = g(n) + h(n)$, whereas greedy best first search has $f(n) = h(n)$ and uniform cost search has $f(n) = g(n)$.

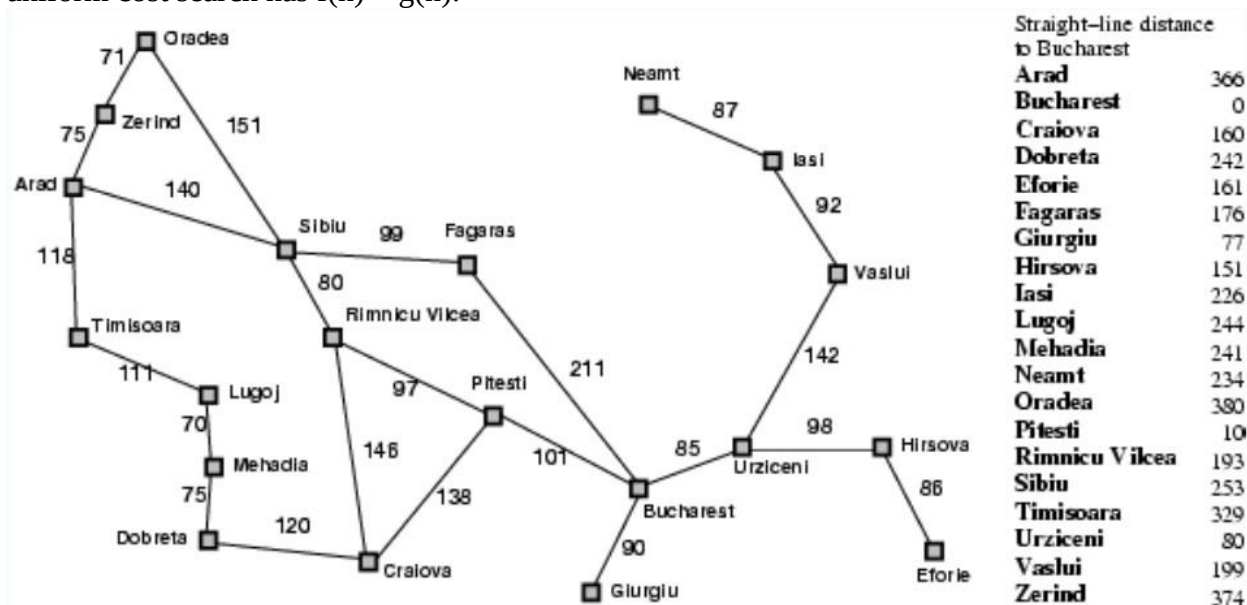
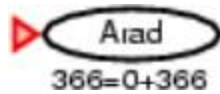
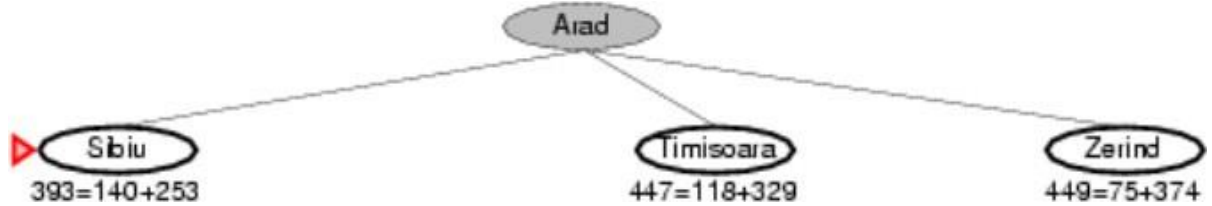


Figure 3 Bucharest Map

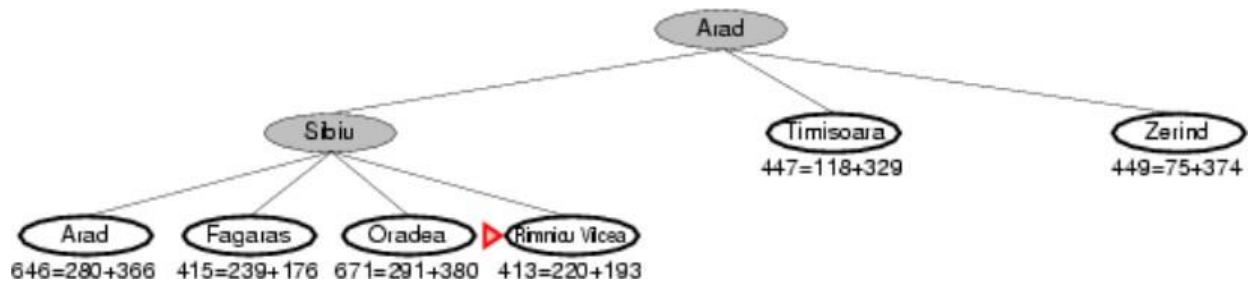
Let's run A* algorithm on above problem.



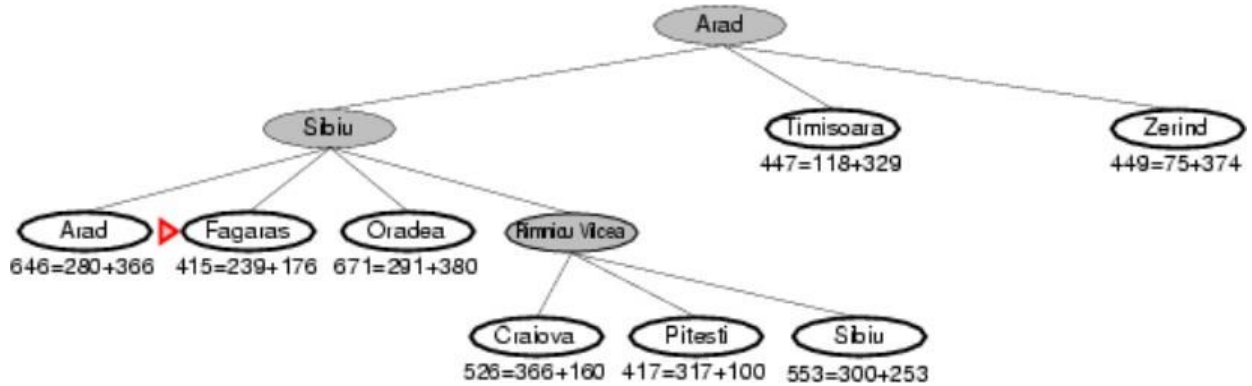
Add Arad in close list. As Arad is not the goal state, expand its child.



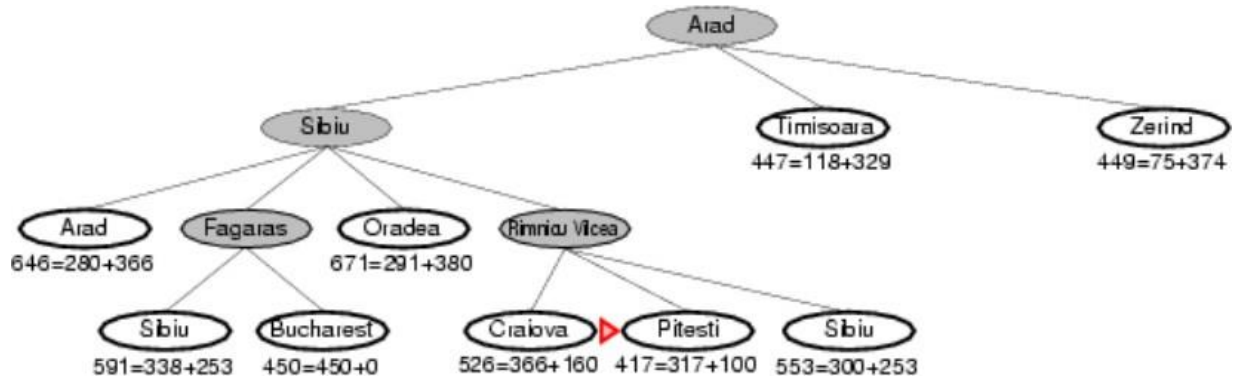
Continue the process until goal node is reached.



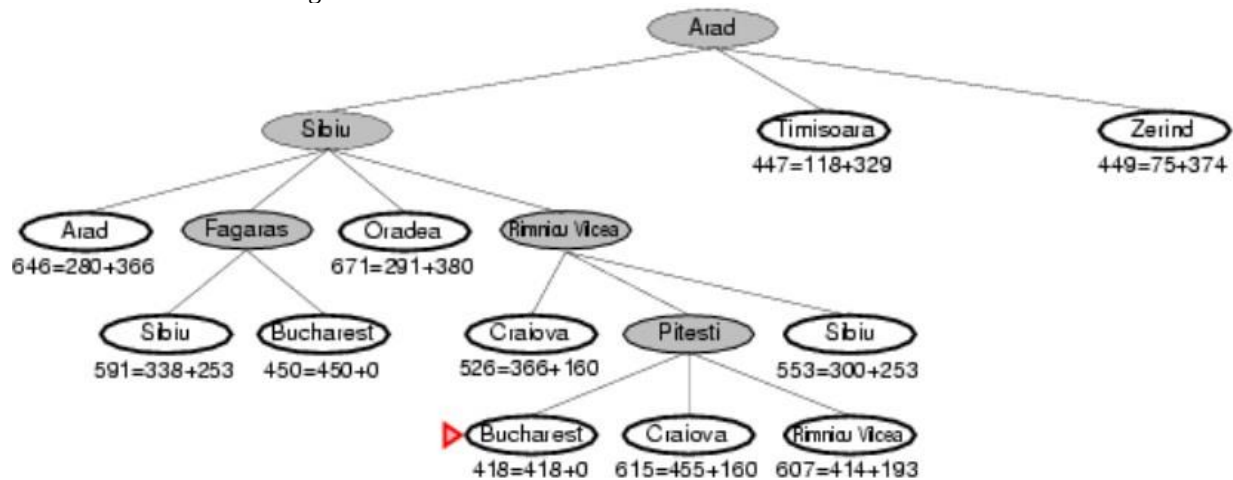
Goal state still not found, continue...



Now $f(n)$ of Fagaras seems shorter than others, maybe we can find better solution through Fagaras.



Now Pitesti seems closer to goal.



Finally we reached the goal.

3. Procedure & Tools

4.1 Walkthrough Task

We maintain two lists: OPEN and CLOSE:

OPEN consists on nodes that have been visited but not expanded (meaning that successors have not been explored yet). This is the list of pending tasks.

CLOSE consists on nodes that have been visited and expanded (successors have been explored already and included in the open list, if this was the case).

Pseudocode of A* Search.

The goal node is denoted by node_goal and the source node is denoted by node_start.

- 1 Put node_start in the OPEN list with $f(\text{node_start}) = h(\text{node_start})$ (initialization)
- 2 while the OPEN list is not empty {
 - 3 Take from the open list the node node_current with the lowest
 - 4 $f(\text{node_current}) = g(\text{node_current}) + h(\text{node_current})$
 - 5 if node_current is node_goal we have found the solution; break
 - 6 Generate each state node_successor that come after node_current
 - 7 for each node_successor of node_current {
 - 8 Set successor_current_cost = $g(\text{node_current}) + w(\text{node_current}, \text{node_successor})$
 - 9 if node_successor is in the OPEN list {
 - 10 if $g(\text{node_successor}) \leq \text{successor_current_cost}$ continue (to line 20) }
 - 11 else if node_successor is in the CLOSED list {
 - 12 if $g(\text{node_successor}) \leq \text{successor_current_cost}$ continue (to line 20)
 - 13 Move node_successor from the CLOSED list to the OPEN list }
 - 14 else {
 - 15 Add node_successor to the OPEN list
 - 16 Set $h(\text{node_successor})$ to be the heuristic distance to node_goal
 - 17 }
 - 18 Set $g(\text{node_successor}) = \text{successor_current_cost}$
 - 19 Set the parent of node_successor to node_current
 - 20 }
 - 21 Add node_current to the CLOSED list
 - 22 }

23 if(node_current != node_goal) exit with error (the OPEN list is empty)

8 Puzzle Problem

This problem can be solved by searching for a solution, which is a sequence of actions (tile moves) that leads from the initial state to the goal state. Two possible states of the 8-puzzle are shown in figure. The state on the right is a typical goal state.

Initial configuration			Final configuration		
1	2	3	1	2	3
5	6		5	8	6
7	8	4		7	4

Tasks

4.1 Task 1

Implement greedy best first search in Python.

4.2 Task 2

Implement the A* search in python.

4.3 Task 3

Implement the 8 Puzzle Problem using A* search in python.

The puzzle consists of an area divided into a grid, 3 by 3 for the 8-puzzle. On each grid square is a tile, expect for one square which remains empty. Thus, there are eight tiles in the 8-puzzle. A tile that is next to the empty grid square can be moved into the empty space, leaving its previous position empty in turn. Tiles are numbered, 1 thru 8 for the 8-puzzle, so that each tile can be uniquely identified. Heuristic for 8-puzzle problem is Number of Misplaced Tiles. You can design your own heuristic.

4. Further Readings

5.1 Books

- Artificial Intelligence: A modern approach 3rd Edition by Stuart Russell & Peter Norvig

5.2 Slides

The slides and reading material can be accessed from the folder of the class instructor.

-

5.3 Links

https://www.tutorialspoint.com/prolog_in_artificial_intelligence/index.asp

<http://lpn.swi-prolog.org/lpnpage.php?pageid=online>
<https://www.geeksforgeeks.org/a-search-algorithm/>
<https://www.geeksforgeeks.org/best-first-search-informed-search/>

Artificial Intelligence

Lab: 09

Lab 9

Topics:

In this lab we try to cover the implementation of the following:

- Alpha-Beta Pruning
- Minimax Algorithm

Questions:

1. Implement Minimax Algo in python.(pseudocode is given in chapter slides as well as in this manual below)
2. Implement alpha beta pruning in python. .(pseudocode is given in chapter slides as well as in this manual below)

Alpha Beta pruning of Minimax Algorithm:

Alpha-Beta pruning is not actually a new algorithm, rather an optimization technique for minimax algorithm. It reduces the computation time by a huge factor. This allows us to search much faster and even go into deeper levels in the game tree. It cuts off branches in the game tree which need not be searched because there already exists a better move available. It is called Alpha-Beta pruning because it passes 2 extra parameters in the minimax function, namely alpha and beta.

Let's define the parameters alpha and beta.

Alpha is the best value that the **maximizer** currently can guarantee at that level or above.

Beta is the best value that the **minimizer** currently can guarantee at that level or above.

Pseudocode :

```
function minimax(node, depth, isMaximizingPlayer, alpha, beta):  
  
    if node is a leaf node:  
        return value of the node  
  
    if isMaximizingPlayer:  
        bestVal = -INFINITY
```

```

    for each child node :
        value=minimax(node, depth+1, false, alpha, beta)
        bestVal = max( bestVal, value)
        alpha = max( alpha, bestVal)
        if beta <= alpha:
            break
    return bestVal

else :
    bestVal = +INFINITY
    for each child node :
        value=minimax(node, depth+1, true, alpha, beta)
        bestVal = min( bestVal, value)
        beta = min( beta, bestVal)
        if beta <= alpha:
            break
    return bestVal
// Calling the function for the first time.
minimax(0, 0, true, -INFINITY, +INFINITY)
Let's make above algorithm clear with an example.

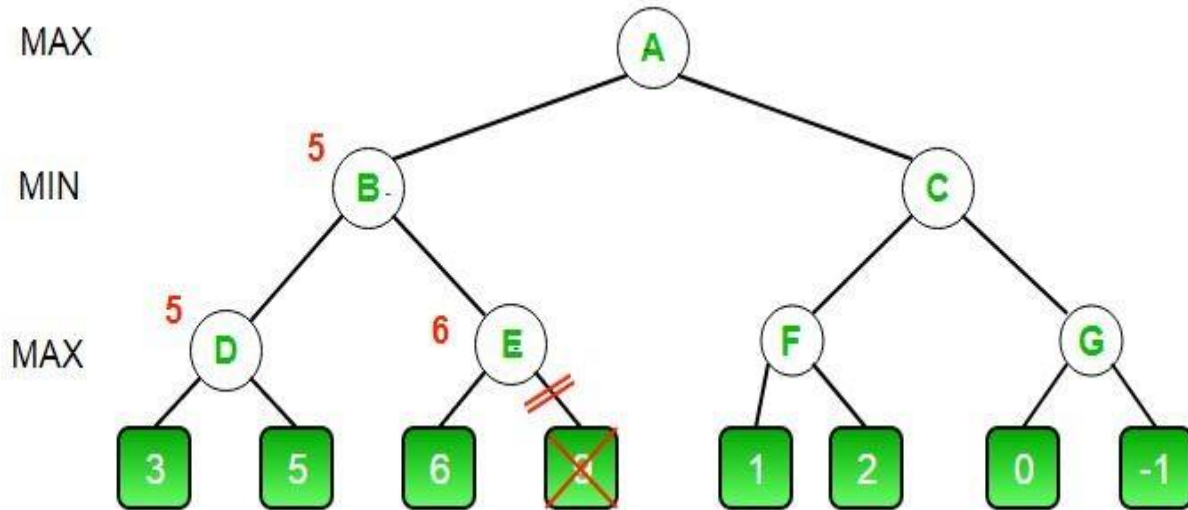
```

- The initial call starts from **A**. The value of alpha here is **-INFINITY** and the value of beta is **+INFINITY**. These values are passed down to subsequent nodes in the tree. At **A** the maximizer must choose max of **B** and **C**, so **A** calls **B** first
- At **B** it the minimizer must choose min of **D** and **E** and hence calls **D** first.
- At **D**, it looks at its left child which is a leaf node. This node returns a value of 3. Now the value of alpha at **D** is $\max(-\text{INF}, 3)$ which is 3.
- To decide whether its worth looking at its right node or not, it checks the condition $\text{beta} \leq \text{alpha}$. This is false since $\text{beta} = +\text{INF}$ and $\text{alpha} = 3$. So it continues the search.
- **D** now looks at its right child which returns a value of 5. At **D**, $\text{alpha} = \max(3, 5)$ which is 5. Now the value of node **D** is 5
- **D** returns a value of 5 to **B**. At **B**, $\text{beta} = \min(+\text{INF}, 5)$ which is 5. The minimizer is now guaranteed a value of 5 or lesser. **B** now calls **E** to see if he can get a lower value than 5.
- At **E** the values of alpha and beta is not $-\text{INF}$ and $+\text{INF}$ but instead $-\text{INF}$ and 5 respectively, because the value of beta was changed at **B** and that is what **B** passed down to **E**
- Now **E** looks at its left child which is 6. At **E**, $\text{alpha} = \max(-\text{INF}, 6)$ which is 6. Here the condition becomes true. beta is 5 and alpha is 6. So $\text{beta} \leq \text{alpha}$ is true. Hence it breaks and **E** returns 6 to **B**
- Note how it did not matter what the value of **E**'s right child is. It could have been $+\text{INF}$ or $-\text{INF}$, it still wouldn't matter, We never even had to look at it because the minimizer was guaranteed a value of 5 or lesser. So as soon as the maximizer saw the 6 he knew the minimizer would never come this way because he can get a 5

on the left side of **B**. This way we don't have to look at that 9 and hence saved computation time.

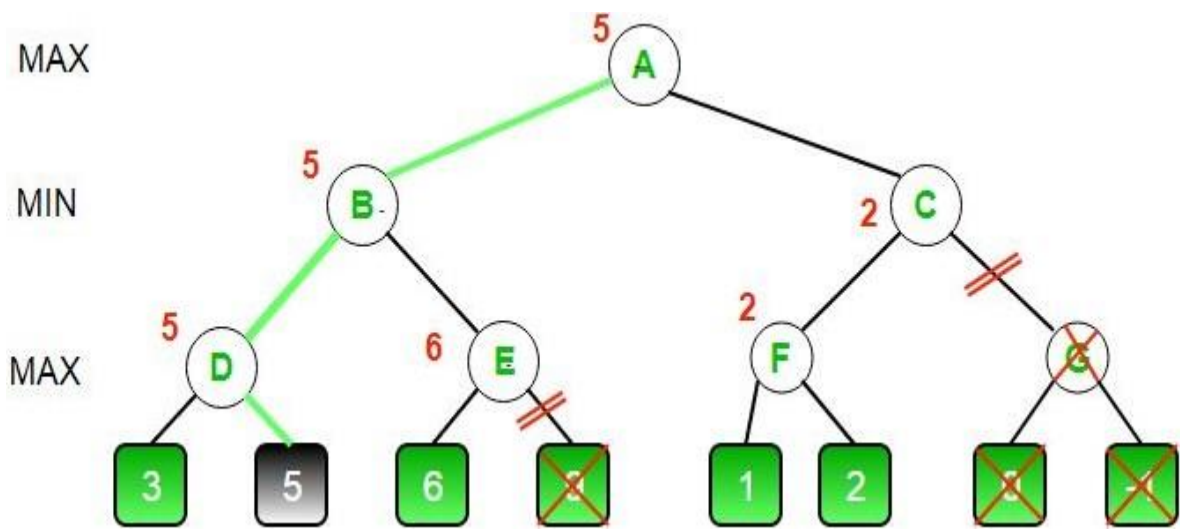
- **E** returns a value of 6 to **B**. At **B**, $\beta = \min(5, 6)$ which is 5. The value of node **B** is also 5

So far this is how our game tree looks. The 9 is crossed out because it was never computed.



- **B** returns 5 to **A**. At **A**, $\alpha = \max(-\text{INF}, 5)$ which is 5. Now the maximizer is guaranteed a value of 5 or greater. **A** now calls **C** to see if it can get a higher value than 5.
- At **C**, $\alpha = 5$ and $\beta = +\text{INF}$. **C** calls **F**
- At **F**, $\alpha = 5$ and $\beta = +\text{INF}$. **F** looks at its left child which is a 1. $\alpha = \max(5, 1)$ which is still 5.
- **F** looks at its right child which is a 2. Hence the best value of this node is 2. Alpha still remains 5
- **F** returns a value of 2 to **C**. At **C**, $\beta = \min(+\text{INF}, 2)$. The condition $\beta \leq \alpha$ becomes true as $\beta = 2$ and $\alpha = 5$. So it breaks and it does not even have to compute the entire sub-tree of **G**.
- The intuition behind this break off is that, at **C** the minimizer was guaranteed a value of 2 or lesser. But the maximizer was already guaranteed a value of 5 if he choose **B**. So why would the maximizer ever choose **C** and get a value less than 2
? Again you can see that it did not matter what those last 2 values were. We also saved a lot of computation by skipping a whole sub tree.
- **C** now returns a value of 2 to **A**. Therefore the best value at **A** is $\max(5, 2)$ which is a 5.
- Hence the optimal value that the maximizer can get is 5

This is how our final game tree looks like. As you can see **G** has been crossed out as it was never computed.



Mini-Max Algorithm in Artificial Intelligence

- Mini-max algorithm is a recursive or backtracking algorithm which is used in decision-making and game theory. It provides an optimal move for the player assuming that opponent is also playing optimally.
- Mini-Max algorithm uses recursion to search through the game-tree.
- Min-Max algorithm is mostly used for game playing in AI. Such as Chess, Checkers, tic-tac-toe, go, and various tow-players game. This Algorithm computes the minimax decision for the current state.
- In this algorithm two players play the game, one is called MAX and other is called MIN.
- Both the players fight it as the opponent player gets the minimum benefit while they get the maximum benefit.
- Both Players of the game are opponent of each other, where MAX will select the maximized value and MIN will select the minimized value.
- The minimax algorithm performs a depth-first search algorithm for the exploration of the complete game tree.
- The minimax algorithm proceeds all the way down to the terminal node of the tree, then backtrack the tree as the recursion.

Pseudo-code for MinMax Algorithm:

1. function minimax(node, depth, maximizingPlayer) is
2. **if** depth ==0 or node is a terminal node then
3. **return static** evaluation of node 4.
5. **if** MaximizingPlayer then // for Maximizer Player
6. maxEva= -infinity
7. **for** each child of node **do**
8. eva= minimax(child, depth-1, **false**)
9. maxEva= max(maxEva,eva) //gives Maximum of the values
10. **return** maxEva 11.
12. **else** // for Minimizer player
13. minEva= +infinity
14. **for** each child of node **do**
15. eva= minimax(child, depth-1, **true**)
16. minEva= min(minEva, eva) //gives minimum of the values

17. **return** minEva

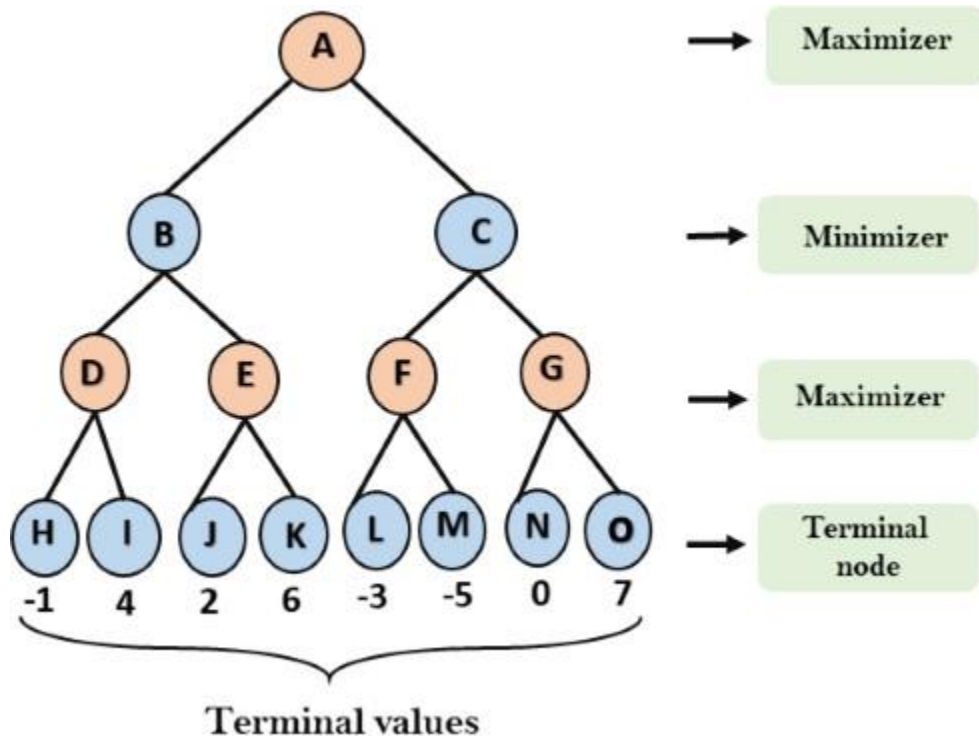
Initial call:

Minimax(node, 3, true)

Working of Min-Max Algorithm:

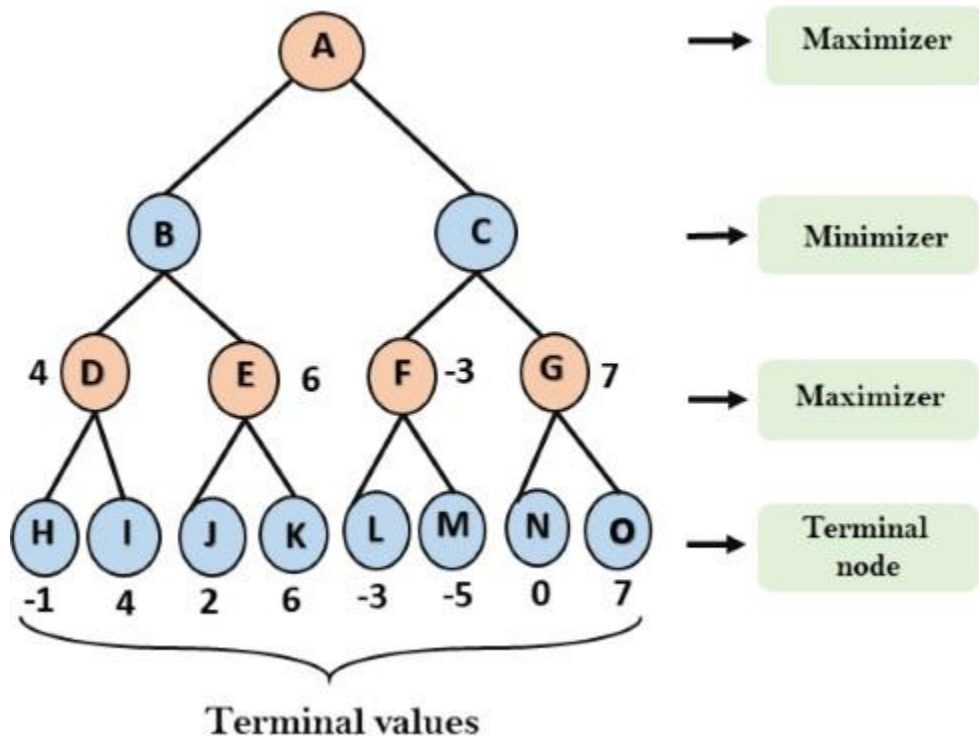
- The working of the minimax algorithm can be easily described using an example. Below we have taken an example of game-tree which is representing the two- player game.
- In this example, there are two players one is called Maximizer and other is called Minimizer.
- Maximizer will try to get the Maximum possible score, and Minimizer will try to get the minimum possible score.
- This algorithm applies DFS, so in this game-tree, we have to go all the way through the leaves to reach the terminal nodes.
- At the terminal node, the terminal values are given so we will compare those value and backtrack the tree until the initial state occurs. Following are the main steps involved in solving the two-player game tree:

Step-1: In the first step, the algorithm generates the entire game-tree and apply the utility function to get the utility values for the terminal states. In the below tree diagram, let's take A is the initial state of the tree. Suppose maximizer takes first turn which has worst-case initial value =- infinity, and minimizer will take next turn which has worst- case initial value = +infinity.



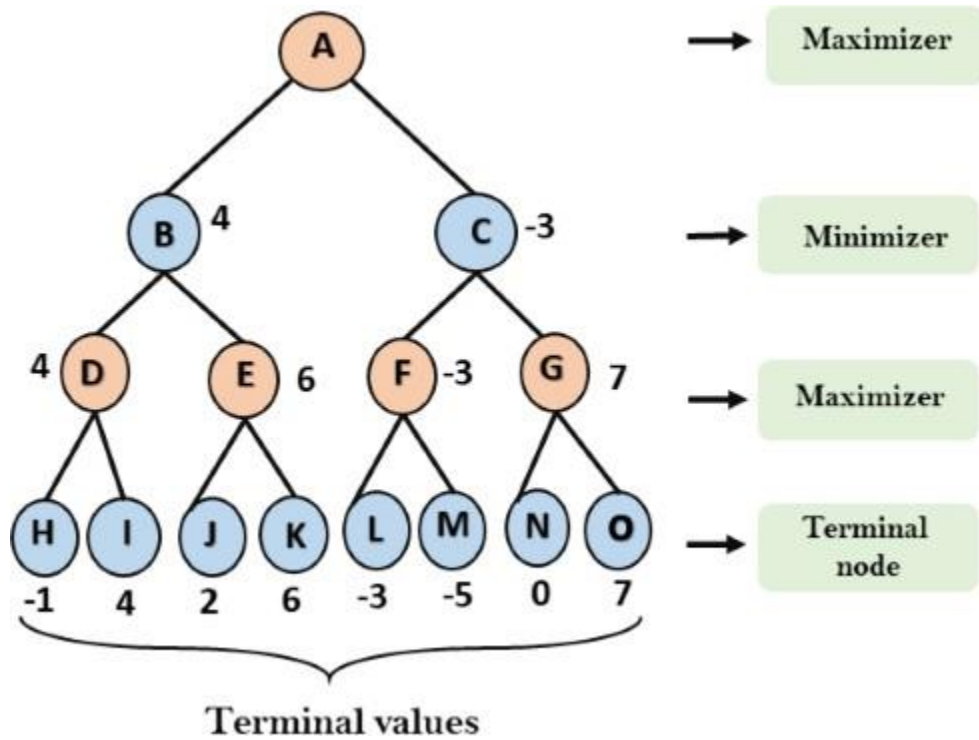
Step 2: Now, first we find the utilities value for the Maximizer, its initial value is $-\infty$, so we will compare each value in terminal state with initial value of Maximizer and determines the higher nodes values. It will find the maximum among the all.

- For node D $\max(-1, -\infty) \Rightarrow \max(-1, 4) = 4$
- For Node E $\max(2, -\infty) \Rightarrow \max(2, 6) = 6$
- For Node F $\max(-3, -\infty) \Rightarrow \max(-3, -5) = -3$
- For node G $\max(0, -\infty) = \max(0, 7) = 7$



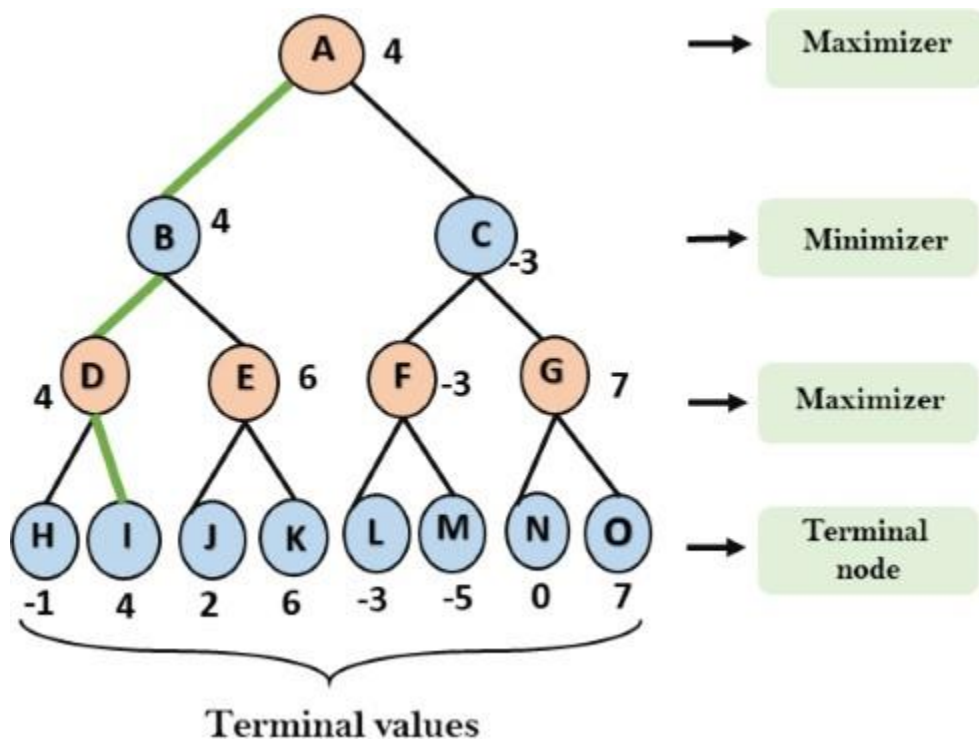
Step 3: In the next step, it's a turn for minimizer, so it will compare all nodes value with $+\infty$, and will find the 3rd layer node values.

- For node B = $\min(4, 6) = 4$
- For node C = $\min(-3, 7) = -3$



Step 3: Now it's a turn for Maximizer, and it will again choose the maximum of all nodes value and find the maximum value for the root node. In this game tree, there are only 4 layers, hence we reach immediately to the root node, but in real games, there will be more than 4 layers.

- For node A $\max(4, -3) = 4$



That was the complete workflow of the minimax two player game.

Properties of Mini-Max algorithm:

- **Complete**- Min-Max algorithm is Complete. It will definitely find a solution (if exist), in the finite search tree.
- **Optimal**- Min-Max algorithm is optimal if both opponents are playing optimally.
- **Time complexity**- As it performs DFS for the game-tree, so the time complexity of Min-Max algorithm is $O(b^m)$, where b is branching factor of the game-tree, and m is the maximum depth of the tree.
- **Space Complexity**- Space complexity of Mini-max algorithm is also similar to DFS which is $O(bm)$.

Limitation of the minimax Algorithm:

The main drawback of the minimax algorithm is that it gets really slow for complex games such as Chess, go, etc. This type of games has a huge branching factor, and the player has lots of choices to decide. This limitation of the minimax algorithm can be improved from **alpha-beta pruning** which we have discussed in the next topic.

2nd Version

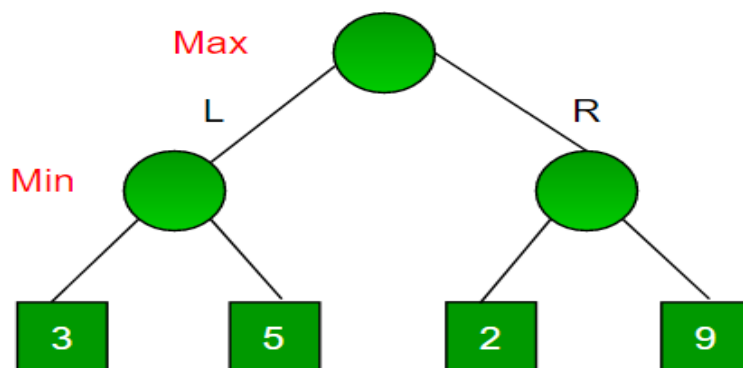
Minimax Algorithm

Minimax is a kind of backtracking algorithm that is used in decision making and game theory to find the optimal move for a player, assuming that your opponent also plays optimally. It is widely used in two player turn-based games such as Tic-Tac-Toe, Backgammon, Mancala, Chess, etc. In Minimax the two players are called maximizer and minimizer. The **maximizer** tries to get the highest score possible while the **minimizer** tries to do the opposite and get the lowest score possible.

Every board state has a value associated with it. In a given state if the maximizer has upper hand then, the score of the board will tend to be some positive value. If the minimizer has the upper hand in that board state then it will tend to be some negative value. The values of the board are calculated by some heuristics which are unique for every type of game.

Example:

Consider a game which has 4 final states and paths to reach final state are from root to 4 leaves of a perfect binary tree as shown below. Assume you are the maximizing player and you get the first chance to move, i.e., you are at the root and your opponent at next level. **Which move you would make as a maximizing player considering that your opponent also plays optimally?**

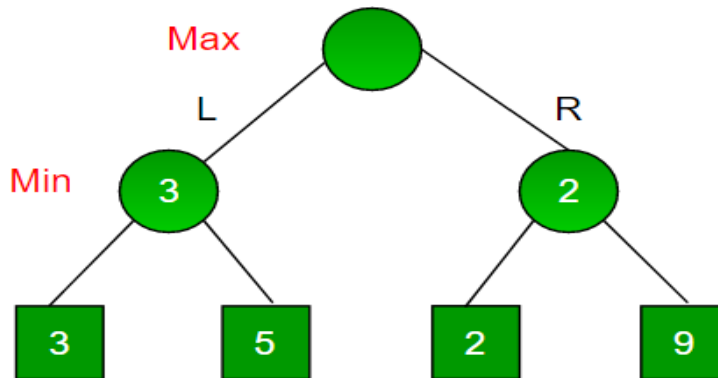


Since this is a backtracking based algorithm, it tries all possible moves, then backtracks and makes a decision.

- Maximizer goes LEFT: It is now the minimizers turn. The minimizer now has a choice between 3 and 5. Being the minimizer it will definitely choose the least among both, that is 3
- Maximizer goes RIGHT: It is now the minimizers turn. The minimizer now has a choice between 2 and 9. He will choose 2 as it is the least among the two values.

Being the maximizer you would choose the larger value that is 3. Hence the optimal move for the maximizer is to go LEFT and the optimal value is 3.

Now the game tree looks like below :



The above tree shows

two possible scores when maximizer makes left and right moves.

Note: Even though there is a value of 9 on the right subtree, the minimizer will never pick that. We must always assume that our opponent plays optimally.

Artificial Intelligence

Lab: 10

Lab 10

Linear Regression

Welcome to your first practice lab! In this lab, you will implement linear regression with one variable to predict profits for a restaurant franchise.

Outline

- [1 - Packages](#)
- [2 - Linear regression with one variable](#)
 - [2.1 Problem Statement](#)
 - [2.2 Dataset](#)
 - [2.3 Refresher on linear regression](#)
 - [2.4 Compute Cost](#)
 - [Exercise 1](#)
 - [2.5 Gradient descent](#)
 - [Exercise 2](#)
 - [2.6 Learning parameters using batch gradient descent](#)

NOTE: To prevent errors from the autograder, you are not allowed to edit or delete non-graded cells in this notebook. Please also refrain from adding any new cells. **Once you have passed this assignment** and want to experiment with any of the non-graded code, you may follow the instructions at the bottom of this notebook.

1 - Packages

First, let's run the cell below to import all the packages that you will need during this assignment.

- [numpy](#) is the fundamental package for working with matrices in Python.
- [matplotlib](#) is a famous library to plot graphs in Python.
- `utils.py` contains helper functions for this assignment. You do not need to modify code in this file.

In [6]:

```
import numpy as np
import matplotlib.pyplot as plt
from utils import *
import copy
import math
%matplotlib inline
```

2 - Problem Statement

Suppose you are the CEO of a restaurant franchise and are considering different cities for opening a new outlet.

- You would like to expand your business to cities that may give your restaurant higher profits.
- The chain already has restaurants in various cities and you have data for profits and populations from the cities.
- You also have data on cities that are candidates for a new restaurant.
 - For these cities, you have the city population.

Can you use the data to help you identify which cities may potentially give your business higher profits?

3 - Dataset

You will start by loading the dataset for this task.

- The `load_data()` function shown below loads the data into variables `x_train` and `y_train`
 - `x_train` is the population of a city
 - `y_train` is the profit of a restaurant in that city. A negative value for profit indicates a loss.
 - Both `x_train` and `y_train` are numpy arrays.

In [7]:

```
# load the dataset
x_train, y_train = load_data()
```

View the variables

Before starting on any task, it is useful to get more familiar with your dataset.

- A good place to start is to just print out each variable and see what it contains.

The code below prints the variable `x_train` and the type of the variable.

In [8]:

```
# print x_train
print("Type of x_train:", type(x_train))
print("First five elements of x_train are:\n", x_train[:5])
```

```
Type of x_train: <class 'numpy.ndarray'>
```

```
First five elements of x_train are:
```

```
[6.1101 5.5277 8.5186 7.0032 5.8598]
```

`x_train` is a numpy array that contains decimal values that are all greater than zero.

- These values represent the city population times 10,000
- For example, 6.1101 means that the population for that city is 61,101

Now, let's print `y_train`

In [9]:

```
# print y_train
print("Type of y_train:", type(y_train))
print("First five elements of y_train are:\n", y_train[:5])
```

```
Type of y_train: <class 'numpy.ndarray'>
```

```
First five elements of y_train are:
```

```
[17.592  9.1302 13.662 11.854  6.8233]
```

Similarly, `y_train` is a numpy array that has decimal values, some negative, some positive.

- These represent your restaurant's average monthly profits in each city, in units of \$10,000.
 - For example, 17.592 represents \$175,920 in average monthly profits for that city.
 - -2.6807 represents -\$26,807 in average monthly loss for that city.

Check the dimensions of your variables

Another useful way to get familiar with your data is to view its dimensions.

Please print the shape of `x_train` and `y_train` and see how many training examples you have in your dataset.

In [10]:

```
print ('The shape of x_train is:', x_train.shape)
print ('The shape of y_train is: ', y_train.shape)
print ('Number of training examples (m):', len(x_train))
```

```
The shape of x_train is: (97,)
```

```
The shape of y_train is: (97,)
```

```
Number of training examples (m): 97
```

The city population array has 97 data points, and the monthly average profits also has 97 data points. These are NumPy 1D arrays.

Visualize your data

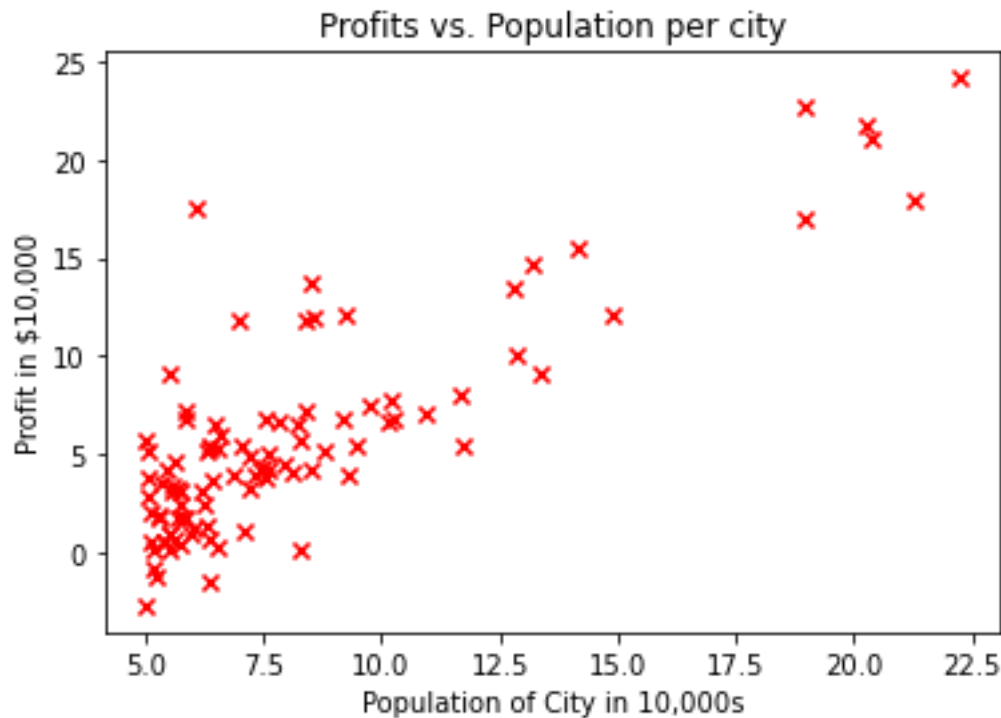
It is often useful to understand the data by visualizing it.

- For this dataset, you can use a scatter plot to visualize the data, since it has only two properties to plot (profit and population).
- Many other problems that you will encounter in real life have more than two properties (for example, population, average household income, monthly profits, monthly sales). When you have more than two properties, you can still use a scatter plot to see the relationship between each pair of properties.

In [11]:

```
# Create a scatter plot of the data. To change the markers to red "x",
# we used the 'marker' and 'c' parameters
plt.scatter(x_train, y_train, marker='x', c='r')
```

```
# Set the title
plt.title("Profits vs. Population per city")
# Set the y-axis label
plt.ylabel('Profit in $10,000')
# Set the x-axis label
plt.xlabel('Population of City in 10,000s')
plt.show()
```



Your goal is to build a linear regression model to fit this data.

- With this model, you can then input a new city's population, and have the model estimate your restaurant's potential monthly profits for that city.

4 - Refresher on linear regression

In this practice lab, you will fit the linear regression parameters (w, b) to your dataset.

- The model function for linear regression, which is a function that maps from $\mathbf{x}$ (city population) to $\mathbf{y}$ (your restaurant's monthly profit for that city) is represented as $f_{w,b}(x) = wx + b$
- To train a linear regression model, you want to find the best (w, b) parameters that fit your dataset.
 - To compare how one choice of (w, b) is better or worse than another choice, you can evaluate it with a cost function $J(w, b)$
 - J is a function of (w, b) . That is, the value of the cost $J(w, b)$ depends on the value of (w, b) .
 - The choice of (w, b) that fits your data the best is the one that has the smallest cost $J(w, b)$.

- To find the values (w,b) that gets the smallest possible cost $J(w,b)$, you can use a method called **gradient descent**.
 - With each step of gradient descent, your parameters (w,b) come closer to the optimal values that will achieve the lowest cost $J(w,b)$.
- The trained linear regression model can then take the input feature x (city population) and output a prediction $f_{w,b}(x)$ (predicted monthly profit for a restaurant in that city).

5 - Compute Cost

Gradient descent involves repeated steps to adjust the value of your parameter (w,b) to gradually get a smaller and smaller cost $J(w,b)$.

- At each step of gradient descent, it will be helpful for you to monitor your progress by computing the cost $J(w,b)$ as (w,b) gets updated.
- In this section, you will implement a function to calculate $J(w,b)$ so that you can check the progress of your gradient descent implementation.

Cost function

As you may recall from the lecture, for one variable, the cost function for linear regression $J(w,b)$ is defined as

$$J(w,b) = \frac{1}{2m} \sum_{i=0}^{m-1} (f_{w,b}(x(i)) - y(i))^2$$

- You can think of $f_{w,b}(x(i))$ as the model's prediction of your restaurant's profit, as opposed to $y(i)$, which is the actual profit that is recorded in the data.
- m is the number of training examples in the dataset

Model prediction

- For linear regression with one variable, the prediction of the model $f_{w,b}$ for an example $x(i)$ is represented as:

$$f_{w,b}(x(i)) = wx(i) + b$$

This is the equation for a line, with an intercept b and a slope w

Implementation

Please complete the `compute_cost()` function below to compute the cost $J(w,b)$.

Exercise 1

Complete the `compute_cost` below to:

- Iterate over the training examples, and for each example, compute:
 - The prediction of the model for that example $f_{w,b}(x(i)) = wx(i) + b$
 - The cost for that example $cost(i) = (f_{w,b}(x(i)) - y(i))^2$

- Return the total cost over all examples $J(\mathbf{w}, b) = \frac{1}{2m} \sum_{i=0}^{m-1} \text{cost}(i)$
 - Here, m is the number of training examples and $\sum$ is the summation operator

If you get stuck, you can check out the hints presented after the cell below to help you with the implementation.

In [17]:

```
# UNQ_C1
# GRADED FUNCTION: compute_cost

def compute_cost(x, y, w, b):
    """
    Computes the cost function for linear regression.

    Args:
        x (ndarray): Shape (m,) Input to the model (Population of cities)
        y (ndarray): Shape (m,) Label (Actual profits for the cities)
        w, b (scalar): Parameters of the model

    Returns
        total_cost (float): The cost of using w,b as the parameters for linear regression
                           to fit the data points in x and y
    """
    # number of training examples
    m = x.shape[0]

    # You need to return this variable correctly
    total_cost = 0

    ### START CODE HERE ###

    yp = np.zeros(m)
    yp = np.dot(x, w) + b
    total_cost = (1 / (2 * m)) * np.sum((yp - y)**2)

    ### END CODE HERE ###

    return total_cost
```

Click for hints

You can check if your implementation was correct by running the following test code:

In [18]:

```
# Compute cost with some initial values for parameters w, b
initial_w = 2
initial_b = 1

cost = compute_cost(x_train, y_train, initial_w, initial_b)
print(type(cost))
```

```
print(f'Cost at initial w: {cost:.3f}')
```

```
# Public tests
```

```
from public_tests import *
```

```
compute_cost_test(compute_cost)
```

```
<class 'numpy.float64'>
```

```
Cost at initial w: 75.203
```

```
All tests passed!
```

Expected Output:

```
Cost at initial w: 75.203
```

6 - Gradient descent

In this section, you will implement the gradient for parameters w, b for linear regression.

As described in the lecture videos, the gradient descent algorithm is:

repeat until convergence: $\{b := b - \alpha \frac{\partial J(w, b)}{\partial b}, w := w - \alpha \frac{\partial J(w, b)}{\partial w}\}$

where, parameters w, b are both updated simultaneously and where

$$\frac{\partial J(w, b)}{\partial b} = \frac{1}{m} \sum_{i=0}^{m-1} (f_{w, b}(x(i)) - y(i))$$

$$\frac{\partial J(w, b)}{\partial w} = \frac{1}{m} \sum_{i=0}^{m-1} (f_{w, b}(x(i)) - y(i)) x(i)$$

- m is the number of training examples in the dataset
- $f_{w, b}(x(i))$ is the model's prediction, while $y(i)$, is the target value

You will implement a function called `compute_gradient` which calculates $\frac{\partial J(w)}{\partial w}$, $\frac{\partial J(w)}{\partial b}$

Exercise 2

Please complete the `compute_gradient` function to:

- Iterate over the training examples, and for each example, compute:
 - The prediction of the model for that example $f_{w, b}(x(i)) = wx(i) + b$
 - The gradient for the parameters w, b from that example $\frac{\partial J(w, b)}{\partial b(i)} = (f_{w, b}(x(i)) - y(i))$

$$\frac{\partial J(w, b)}{\partial w(i)} = (f_{w, b}(x(i)) - y(i)) x(i)$$

- Return the total gradient update from all the examples $\frac{\partial J(w, b)}{\partial b} = \frac{1}{m} \sum_{i=0}^{m-1} \frac{\partial J(w, b)}{\partial b(i)}$
 $\frac{\partial J(w, b)}{\partial w} = \frac{1}{m} \sum_{i=0}^{m-1} \frac{\partial J(w, b)}{\partial w(i)}$

- Here, m is the number of training examples and $\sum$ is the summation operator

If you get stuck, you can check out the hints presented after the cell below to help you with the implementation.

In [19]:

```
# UNQ_C2
```

```
# GRADED FUNCTION: compute_gradient
```

```
def compute_gradient(x, y, w, b):
    """
    Computes the gradient for linear regression
    Args:
        x (ndarray): Shape (m,) Input to the model (Population of cities)
        y (ndarray): Shape (m,) Label (Actual profits for the cities)
        w, b (scalar): Parameters of the model
    Returns
        dj_dw (scalar): The gradient of the cost w.r.t. the parameters w
        dj_db (scalar): The gradient of the cost w.r.t. the parameter b
    """

    # Number of training examples
    m = x.shape[0]

    # You need to return the following variables correctly
    dj_dw = 0
    dj_db = 0

    ### START CODE HERE ###

    yp = np.zeros(m)
    yp = np.dot(x, w) + b
    dj_db = (1 / m) * np.sum(yp-y)
    dj_dw = (1 / m) * np.sum((yp-y)*x)

    ### END CODE HERE ###

    return dj_dw, dj_db
```

Click for hints

Run the cells below to check your implementation of the `compute_gradient` function with two different initializations of the parameters w, b .

In [20]:

```
# Compute and display gradient with w initialized to zeroes
initial_w = 0
initial_b = 0

tmp_dj_dw, tmp_dj_db = compute_gradient(x_train, y_train, initial_w, initial_b)
print('Gradient at initial w, b (zeros):', tmp_dj_dw, tmp_dj_db)

compute_gradient_test(compute_gradient)

Gradient at initial w, b (zeros): -65.32884974555671 -5.839135051546392
Using X with shape (4, 1)
All tests passed!
```

Now let's run the gradient descent algorithm implemented above on our dataset.

Expected Output:

```
Gradient at initial , b (zeros)  -65.32884975 -5.83913505154639
```

In [21]:

```
# Compute and display cost and gradient with non-zero w
test_w = 0.2
test_b = 0.2
tmp_dj_dw, tmp_dj_db = compute_gradient(x_train, y_train, test_w, test_b)

print('Gradient at test w, b:', tmp_dj_dw, tmp_dj_db)
Gradient at test w, b: -47.41610118114433 -4.007175051546392
```

Expected Output:

```
Gradient at test w  -47.41610118 -4.007175051546391
```

2.6 Learning parameters using batch gradient descent

You will now find the optimal parameters of a linear regression model by using batch gradient descent. Recall batch refers to running all the examples in one iteration.

- You don't need to implement anything for this part. Simply run the cells below.
- A good way to verify that gradient descent is working correctly is to look at the value of $J(w,b)$ and check that it is decreasing with each step.
- Assuming you have implemented the gradient and computed the cost correctly and you have an appropriate value for the learning rate α , $J(w,b)$ should never increase and should converge to a steady value by the end of the algorithm.

In [22]:

```
def gradient_descent(x, y, w_in, b_in, cost_function, gradient_function, alpha, num_iters):
    """
    Performs batch gradient descent to learn theta. Updates theta by taking
    num_iters gradient steps with learning rate alpha

    Args:
        x : (ndarray): Shape (m,)
        y : (ndarray): Shape (m,)
        w_in, b_in : (scalar) Initial values of parameters of the model
        cost_function: function to compute cost
        gradient_function: function to compute the gradient
        alpha : (float) Learning rate
        num_iters : (int) number of iterations to run gradient descent

    Returns
        w : (ndarray): Shape (1,) Updated values of parameters of the model after
            running gradient descent
        b : (scalar) Updated value of parameter of the model after
```

```

        running gradient descent
        """

        # number of training examples
        m = len(x)

        # An array to store cost J and w's at each iteration - primarily for graphing later
        J_history = []
        w_history = []
        w = copy.deepcopy(w_in)  #avoid modifying global w within function
        b = b_in

        for i in range(num_iters):

            # Calculate the gradient and update the parameters
            dj_dw, dj_db = gradient_function(x, y, w, b )

            # Update Parameters using w, b, alpha and gradient
            w = w - alpha * dj_dw
            b = b - alpha * dj_db

            # Save cost J at each iteration
            if i<100000:      # prevent resource exhaustion
                cost = cost_function(x, y, w, b)
                J_history.append(cost)

            # Print cost every at intervals 10 times or as many iterations if <
10
            if i% math.ceil(num_iters/10) == 0:
                w_history.append(w)
                print(f"Iteration {i:4}: Cost {float(J_history[-1]):8.2f}    ")

        return w, b, J_history, w_history #return w and J,w history for graphing
g

```

Now let's run the gradient descent algorithm above to learn the parameters for our dataset.

In [23]:

```

# initialize fitting parameters. Recall that the shape of w is (n,)
initial_w = 0.
initial_b = 0.

# some gradient descent settings
iterations = 1500
alpha = 0.01

w,b,_,_ = gradient_descent(x_train ,y_train, initial_w, initial_b,
                           compute_cost, compute_gradient, alpha, iterations)

```

```
print("w,b found by gradient descent:", w, b)
```

```
Iteration    0: Cost      6.74
Iteration   150: Cost      5.31
Iteration   300: Cost      4.96
Iteration   450: Cost      4.76
Iteration   600: Cost      4.64
Iteration   750: Cost      4.57
Iteration   900: Cost      4.53
Iteration  1050: Cost      4.51
Iteration  1200: Cost      4.50
Iteration  1350: Cost      4.49
```

```
w,b found by gradient descent: 1.166362350335582 -3.6302914394043597
```

Expected Output:

```
w, b found by gradient descent  1.16636235 -3.63029143940436
```

We will now use the final parameters from gradient descent to plot the linear fit.

Recall that we can get the prediction for a single example $f(x(i))=wx(i)+b$.

To calculate the predictions on the entire dataset, we can loop through all the training examples and calculate the prediction for each example. This is shown in the code block below.

In [24]:

```
m = x_train.shape[0]
predicted = np.zeros(m)

for i in range(m):
    predicted[i] = w * x_train[i] + b
```

We will now plot the predicted values to see the linear fit.

In [25]:

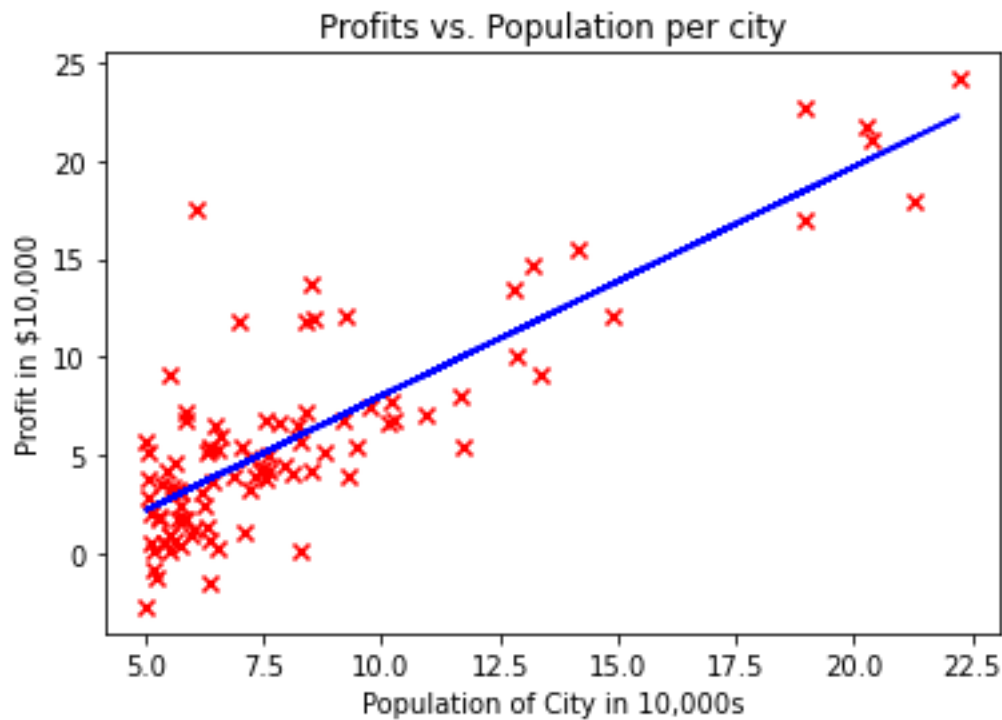
```
# Plot the linear fit
plt.plot(x_train, predicted, c = "b")

# Create a scatter plot of the data.
plt.scatter(x_train, y_train, marker='x', c='r')

# Set the title
plt.title("Profits vs. Population per city")
# Set the y-axis label
plt.ylabel('Profit in $10,000')
# Set the x-axis label
plt.xlabel('Population of City in 10,000s')

Text(0.5, 0, 'Population of City in 10,000s')
```

Out[25]:



Your final values of w, b can also be used to make predictions on profits. Let's predict what the profit would be in areas of 35,000 and 70,000 people.

- The model takes in population of a city in 10,000s as input.
- Therefore, 35,000 people can be translated into an input to the model as `np.array([3.5])`
- Similarly, 70,000 people can be translated into an input to the model as `np.array([7.])`

In [26]:

```
predict1 = 3.5 * w + b
print('For population = 35,000, we predict a profit of $%.2f' % (predict1*10000))
```

```
predict2 = 7.0 * w + b
print('For population = 70,000, we predict a profit of $%.2f' % (predict2*10000))
```

```
For population = 35,000, we predict a profit of $4519.77
For population = 70,000, we predict a profit of $45342.45
```

Expected Output:

```
For population = 35,000, we predict a profit of    $4519.77
```

```
For population = 70,000, we predict a profit of    $45342.45
```

Artificial Intelligence

Lab: 11

Lab 11

Logistic Regression

In this exercise, you will implement logistic regression and apply it to two different datasets.

Outline

- [1 - Packages](#)
- [2 - Logistic Regression](#)
 - [2.1 Problem Statement](#)
 - [2.2 Loading and visualizing the data](#)
 - [2.3 Sigmoid function](#)
 - [2.4 Cost function for logistic regression](#)
 - [2.5 Gradient for logistic regression](#)
 - [2.6 Learning parameters using gradient descent](#)
 - [2.7 Plotting the decision boundary](#)
 - [2.8 Evaluating logistic regression](#)
- [3 - Regularized Logistic Regression](#)
 - [3.1 Problem Statement](#)
 - [3.2 Loading and visualizing the data](#)
 - [3.3 Feature mapping](#)
 - [3.4 Cost function for regularized logistic regression](#)
 - [3.5 Gradient for regularized logistic regression](#)
 - [3.6 Learning parameters using gradient descent](#)
 - [3.7 Plotting the decision boundary](#)
 - [3.8 Evaluating regularized logistic regression model](#)

NOTE: To prevent errors from the autograder, you are not allowed to edit or delete non-graded cells in this lab. Please also refrain from adding any new cells. **Once you have passed this assignment** and want to experiment with any of the non-graded code, you may follow the instructions at the bottom of this notebook.

1 - Packages

First, let's run the cell below to import all the packages that you will need during this assignment.

- [numpy](#) is the fundamental package for scientific computing with Python.
- [matplotlib](#) is a famous library to plot graphs in Python.
- `utils.py` contains helper functions for this assignment. You do not need to modify code in this file.

In [3]:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
from utils import *
import copy
import math

%matplotlib inline
```

2 - Logistic Regression

In this part of the exercise, you will build a logistic regression model to predict whether a student gets admitted into a university.

2.1 Problem Statement

Suppose that you are the administrator of a university department and you want to determine each applicant's chance of admission based on their results on two exams.

- You have historical data from previous applicants that you can use as a training set for logistic regression.
- For each training example, you have the applicant's scores on two exams and the admissions decision.
- Your task is to build a classification model that estimates an applicant's probability of admission based on the scores from those two exams.

2.2 Loading and visualizing the data

You will start by loading the dataset for this task.

- The `load_dataset()` function shown below loads the data into variables `X_train` and `y_train`
 - `X_train` contains exam scores on two exams for a student
 - `y_train` is the admission decision
 - `y_train = 1` if the student was admitted
 - `y_train = 0` if the student was not admitted
 - Both `X_train` and `y_train` are numpy arrays.

In [4]:

```
# load dataset
X_train, y_train = load_data("data/ex2data1.txt")
```

View the variables

Let's get more familiar with your dataset.

- A good place to start is to just print out each variable and see what it contains.

The code below prints the first five values of `X_train` and the type of the variable.

In [5]:

```
print("First five elements in X_train are:\n", X_train[:5])
print("Type of X_train:", type(X_train))
```

First five elements in X_train are:

```
[[34.62365962 78.02469282]
 [30.28671077 43.89499752]
 [35.84740877 72.90219803]
 [60.18259939 86.3085521 ]
 [79.03273605 75.34437644]]
```

Type of X_train: <class 'numpy.ndarray'>

Now print the first five values of y_train

In [6]:

```
print("First five elements in y_train are:\n", y_train[:5])
print("Type of y_train:", type(y_train))
```

First five elements in y_train are:

```
[0. 0. 0. 1. 1.]
```

Type of y_train: <class 'numpy.ndarray'>

Check the dimensions of your variables

Another useful way to get familiar with your data is to view its dimensions. Let's print the shape of X_train and y_train and see how many training examples we have in our dataset.

In [7]:

```
print ('The shape of X_train is: ' + str(X_train.shape))
print ('The shape of y_train is: ' + str(y_train.shape))
print ('We have m = %d training examples' % (len(y_train)))
```

The shape of X_train is: (100, 2)

The shape of y_train is: (100,)

We have m = 100 training examples

Visualize your data

Before starting to implement any learning algorithm, it is always good to visualize the data if possible.

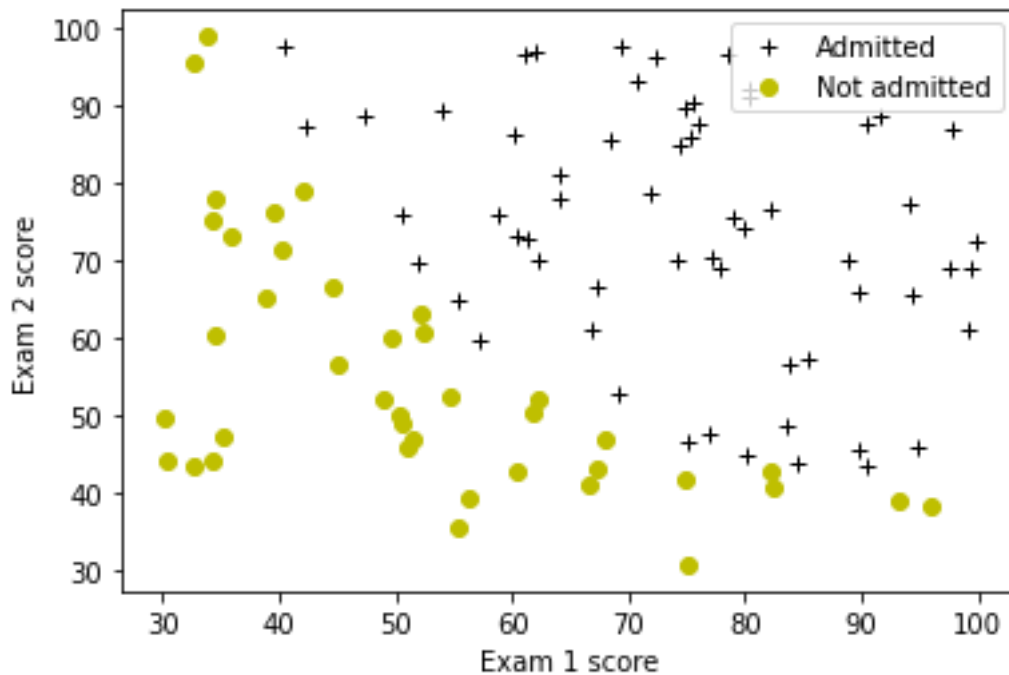
- The code below displays the data on a 2D plot (as shown below), where the axes are the two exam scores, and the positive and negative examples are shown with different markers.
- We use a helper function in the `utils.py` file to generate this plot.

In [8]:

```
# Plot examples
plot_data(X_train, y_train[:,], pos_label="Admitted", neg_label="Not admitted")
```

```
# Set the y-axis label
plt.ylabel('Exam 2 score')
```

```
# Set the x-axis label
plt.xlabel('Exam 1 score')
plt.legend(loc="upper right")
plt.show()
```



Your goal is to build a logistic regression model to fit this data.

- With this model, you can then predict if a new student will be admitted based on their scores on the two exams.

2.3 Sigmoid function

Recall that for logistic regression

Let's implement the sigmoid function first, so it can be used by the rest of this assignment.

Exercise 1

Please complete the `sigmoid`

Note that

- `z` is not always a single number, but can also be an array of numbers.
- If the input is an array of numbers, we'd like to apply the sigmoid function to each value in the input array.

If you get stuck, you can check out the hints presented after the cell below to help you with the implementation.

In [9]:

```
# UNQ_C1
```

```
# GRADED FUNCTION: sigmoid

def sigmoid(z):
    """
    Compute the sigmoid of z

    Args:
        z (ndarray): A scalar, numpy array of any size.

    Returns:
        g (ndarray): sigmoid(z), with the same shape as z

    """

    ### START CODE HERE ###

    g = 1 / (1+np.exp(-z))

    ### END SOLUTION ###

    return g
```

Click for hints

When you are finished, try testing a few values by calling `sigmoid(x)` in the cell below.

- For large positive values of x , the sigmoid should be close to 1, while for large negative values, the sigmoid should be close to 0.
- Evaluating `sigmoid(0)` should give you exactly 0.5.

In [10]:

```
# Note: You can edit this value
value = 0

print (f"sigmoid({value}) = {sigmoid(value)}")

sigmoid(0) = 0.5
```

Expected Output:

```
sigmoid(0) 0.5
```

- As mentioned before, your code should also work with vectors and matrices. For a matrix, your function should perform the sigmoid function on every element.

In [11]:

```
print ("sigmoid([ -1, 0, 1, 2]) = " + str(sigmoid(np.array([-1, 0, 1, 2])))
)

# UNIT TESTS
```

```

from public_tests import *
sigmoid_test(sigmoid)

sigmoid([-1, 0, 1, 2]) = [0.26894142 0.5 0.73105858 0.88079708]
All tests passed!

```

Expected Output:

```
sigmoid([-1, 0, 1, 2]) [0.26894142 0.5 0.73105858 0.88079708]
```

2.4 Cost function for logistic regression

In this section, you will implement the cost function for logistic regression.

Exercise 2

Please complete the `compute_cost` function using the equations below.

where

- m is the number of training examples in the dataset

Note:

- As you are doing this, remember that the variables `X_train` and `y_train` are not scalar values but matrices of shape (m, n) and $(m, 1)$ respectively, where n is the number of features and m is the number of training examples.
- You can use the sigmoid function that you implemented above for this part.

If you get stuck, you can check out the hints presented after the cell below to help you with the implementation.

In [12]:

```

# UNQ_C2
# GRADED FUNCTION: compute_cost
def compute_cost(X, y, w, b, *argv):
    """
    Computes the cost over all examples
    Args:
        X : (ndarray Shape (m,n)) data, m examples by n features
        y : (ndarray Shape (m,)) target value
        w : (ndarray Shape (n,)) values of parameters of the model
        b : (scalar) value of bias parameter of the model
        *argv : unused, for compatibility with regularized version below
    Returns:
        total_cost : (scalar) cost
    """

    m, n = X.shape

```

```

z = np.zeros(m)
loss = np.zeros(n)
### START CODE HERE ###
for i in range(0,m):
    z[i] = np.dot(w,X[i]) + b
g = sigmoid(z)
loss = (-y*np.log(g))-((1-y)*(np.log(1-g)))
total_cost = np.sum(loss)/m
### END CODE HERE ###

return total_cost

```

Click for hints

Run the cells below to check your implementation of the `compute_cost` function with two different initializations of the parameters w and b

In [13]:

```

m, n = X_train.shape

# Compute and display cost with w and b initialized to zeros
initial_w = np.zeros(n)
initial_b = 0.
cost = compute_cost(X_train, y_train, initial_w, initial_b)
print('Cost at initial w and b (zeros): {:.3f}'.format(cost))

Cost at initial w and b (zeros): 0.693

```

Expected Output:

```
Cost at initial w and b (zeros)  0.693
```

In [14]:

```

# Compute and display cost with non-zero w and b
test_w = np.array([0.2, 0.2])
test_b = -24.
cost = compute_cost(X_train, y_train, test_w, test_b)

print('Cost at test w and b (non-zeros): {:.3f}'.format(cost))

```

```

# UNIT TESTS
compute_cost_test(compute_cost)

Cost at test w and b (non-zeros): 0.218
All tests passed!

```

Expected Output:

```
Cost at test w and b (non-zeros): 0.218
```

2.5 Gradient for logistic regression

In this section, you will implement the gradient for logistic regression.

Exercise 3

Please complete the `compute_gradient`

As before, you can use the sigmoid function that you implemented above and if you get stuck, you can check out the hints presented after the cell below to help you with the implementation.

In [31]:

```
# UNQ_C3
# GRADED FUNCTION: compute_gradient
def compute_gradient(X, y, w, b, *argv):
    """
    Computes the gradient for logistic regression

    Args:
        X : (ndarray Shape (m,n)) data, m examples by n features
        y : (ndarray Shape (m,)) target value
        w : (ndarray Shape (n,)) values of parameters of the model
        b : (scalar) value of bias parameter of the model
        *argv : unused, for compatibility with regularized version below
    Returns
        dj_dw : (ndarray Shape (n,)) The gradient of the cost w.r.t. the parameters w.
        dj_db : (scalar) The gradient of the cost w.r.t. the parameter b.
    """
    m, n = X.shape
    dj_dw = np.zeros(w.shape)
    dj_db = 0.

    ### START CODE HERE ###

    z = np.zeros(m)
    for i in range(m):
        z[i] = np.dot(w, X[i]) + b
    f = sigmoid(z)

    dj_db = np.sum(f - y) / m

    for i in range(w.shape[0]):
        dj_dw[i] = np.sum((f - y) * X[:, i]) / m

    ### END CODE HERE ###

    return dj_db, dj_dw
```

Click for hints

Run the cells below to check your implementation of the `compute_gradient` function with two different initializations of the parameters w and b

In [32]:

```
# Compute and display gradient with w and b initialized to zeros
initial_w = np.zeros(n)
initial_b = 0.

dj_db, dj_dw = compute_gradient(X_train, y_train, initial_w, initial_b)
print(f'dj_db at initial w and b (zeros):{dj_db}' )
print(f'dj_dw at initial w and b (zeros):{dj_dw.tolist()}' )

dj_db at initial w and b (zeros):-0.1
dj_dw at initial w and b (zeros):[-12.009216589291151, -11.262842205513596]
```

Expected Output:

dj_db at initial w and b (zeros)	-0.1
---	-------------

dj_dw at initial w and b (zeros): [-12.00921658929115, -11.262842205513591]

In [33]:

```
# Compute and display cost and gradient with non-zero w and b
test_w = np.array([ 0.2, -0.5])
test_b = -24
dj_db, dj_dw = compute_gradient(X_train, y_train, test_w, test_b)

print('dj_db at test w and b:', dj_db)
print('dj_dw at test w and b:', dj_dw.tolist())

# UNIT TESTS
compute_gradient_test(compute_gradient)

dj_db at test w and b: -0.59999999999991071
dj_dw at test w and b: [-44.831353617873795, -44.37384124953977]
All tests passed!
```

Expected Output:

dj_db at test w and b (non-zeros)	-0.59999999999991071
--	-----------------------------

dj_dw at test w and b (non-zeros): [-44.831353617873795, -44.37384124953978]

2.6 Learning parameters using gradient descent

Similar to the previous assignment, you will now find the optimal parameters of a logistic regression model by using gradient descent.

In [34]:

```
def gradient_descent(X, y, w_in, b_in, cost_function, gradient_function, al
pha, num_iters, lambda_):
    """
```

Performs batch gradient descent to learn theta. Updates theta by taking num_iters gradient steps with learning rate alpha

Args:

X : (ndarray Shape (m, n)) data, m examples by n features
y : (ndarray Shape (m,)) target value
w_in : (ndarray Shape (n,)) Initial values of parameters of the model
1 b_in : (scalar) Initial value of parameter of the model
cost_function : function to compute cost
gradient_function : function to compute gradient
alpha : (float) Learning rate
num_iters : (int) number of iterations to run gradient descent
cent lambda_ : (scalar, float) regularization constant

Returns:

w : (ndarray Shape (n,)) Updated values of parameters of the model after
running gradient descent
b : (scalar) Updated value of parameter of the model after
running gradient descent
"""

```
# number of training examples
m = len(X)

# An array to store cost J and w's at each iteration primarily for graphing later
J_history = []
w_history = []

for i in range(num_iters):

    # Calculate the gradient and update the parameters
    dj_db, dj_dw = gradient_function(X, y, w_in, b_in, lambda_)

    # Update Parameters using w, b, alpha and gradient
    w_in = w_in - alpha * dj_dw
    b_in = b_in - alpha * dj_db

    # Save cost J at each iteration
    if i<100000: # prevent resource exhaustion
        cost = cost_function(X, y, w_in, b_in, lambda_)
        J_history.append(cost)
```

```

        # Print cost every at intervals 10 times or as many iterations if <
10
        if i% math.ceil(num_iters/10) == 0 or i == (num_iters-1):
            w_history.append(w_in)
            print(f"Iteration {i:4}: Cost {float(J_history[-1]):8.2f} ")

    return w_in, b_in, J_history, w_history #return w and J,w history for g
raphing

```

Now let's run the gradient descent algorithm above to learn the parameters for our dataset.

Note The code block below takes a couple of minutes to run, especially with a non-vectorized version. You can reduce the `iterations` to test your implementation and iterate faster. If you have time later, try running 100,000 iterations for better results.

In [35]:

```

np.random.seed(1)
initial_w = 0.01 * (np.random.rand(2) - 0.5)
initial_b = -8

# Some gradient descent settings
iterations = 10000
alpha = 0.001

w,b, J_history,_ = gradient_descent(X_train ,y_train, initial_w, initial_b,
                                   compute_cost, compute_gradient, alpha, i
terations, 0)

```

Expected Output: Cost 0.30, (Click to see details):

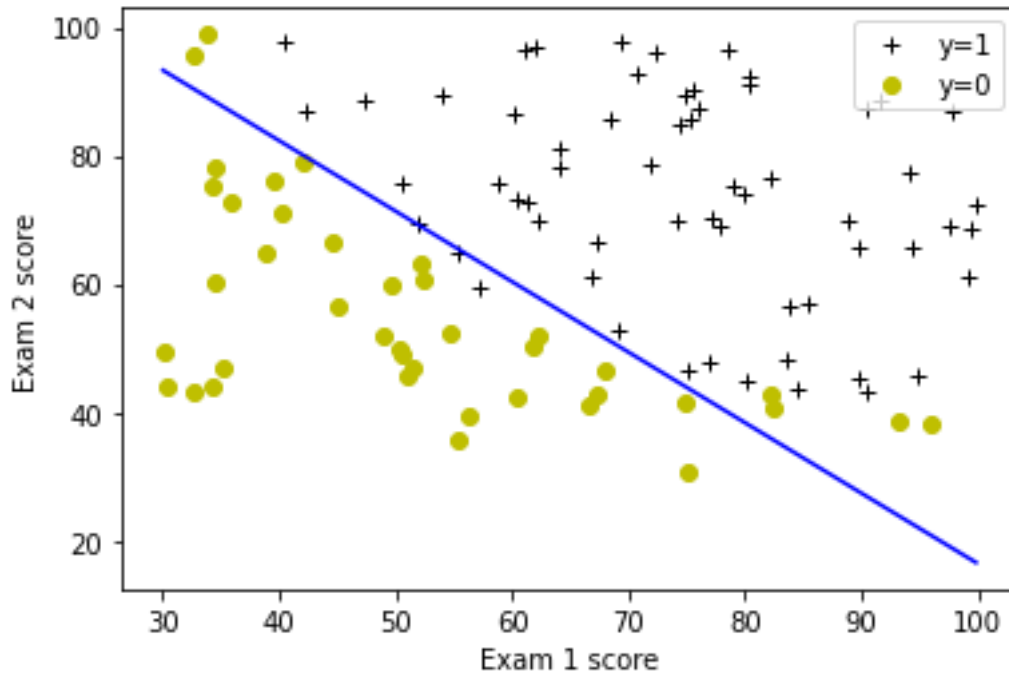
2.7 Plotting the decision boundary

We will now use the final parameters from gradient descent to plot the linear fit. If you implemented the previous parts correctly, you should see a plot similar to the following plot:

We will use a helper function in the `utils.py` file to create this plot.

In [36]:

```
plot_decision_boundary(w, b, X_train, y_train)
# Set the y-axis label
plt.ylabel('Exam 2 score')
# Set the x-axis label
plt.xlabel('Exam 1 score')
plt.legend(loc="upper right")
plt.show()
```



2.8 Evaluating logistic regression

We can evaluate the quality of the parameters we have found by seeing how well the learned model predicts on our training set.

You will implement the `predict` function below to do this.

Exercise 4

Please complete the `predict` function to produce 1 or 0 predictions given a dataset and a learned parameter vector.

If you get stuck, you can check out the hints presented after the cell below to help you with the implementation.

In [39]:

```
# UNQ_C4
# GRADED FUNCTION: predict

def predict(X, w, b):
    """
    Predict whether the label is 0 or 1 using learned logistic
    regression parameters w

    Args:
        X : (ndarray Shape (m,n)) data, m examples by n features
        w : (ndarray Shape (n,)) values of parameters of the model
        b : (scalar) value of bias parameter of the model

    Returns:
```

```

    p : (ndarray (m,)) The predictions for X using a threshold at 0.5
    """
    # number of training examples
    m, n = X.shape
    p = np.zeros(m)

    ### START CODE HERE ###

    z = np.zeros(m)
    for i in range(m):
        z[i] = np.dot(w,X[i]) + b
    f = sigmoid(z)

    p = np.where(f >= 0.5, 1, 0)

    ### END CODE HERE ###
    return p

```

In [40]:

```

# Test your predict code
np.random.seed(1)
tmp_w = np.random.randn(2)
tmp_b = 0.3
tmp_X = np.random.randn(4, 2) - 0.5

tmp_p = predict(tmp_X, tmp_w, tmp_b)
print(f'Output of predict: shape {tmp_p.shape}, value {tmp_p}')

# UNIT TESTS
predict_test(predict)

Output of predict: shape (4,), value [0 1 1 1]
All tests passed!

```

Expected output

```
Output of predict: shape (4,),value [0. 1. 1. 1.]
```

Now let's use this to compute the accuracy on the training set

In [41]:

```

#Compute accuracy on our training set
p = predict(X_train, w,b)
print('Train Accuracy: %f'%(np.mean(p == y_train) * 100))

Train Accuracy: 92.000000

```

```
Train Accuracy (approx): 92.00
```

3 - Regularized Logistic Regression

In this part of the exercise, you will implement regularized logistic regression to predict whether microchips from a fabrication plant passes quality assurance (QA). During QA, each microchip goes through various tests to ensure it is functioning correctly.

3.1 Problem Statement

Suppose you are the product manager of the factory and you have the test results for some microchips on two different tests.

- From these two tests, you would like to determine whether the microchips should be accepted or rejected.
- To help you make the decision, you have a dataset of test results on past microchips, from which you can build a logistic regression model.

3.2 Loading and visualizing the data

Similar to previous parts of this exercise, let's start by loading the dataset for this task and visualizing it.

- The `load_dataset()` function shown below loads the data into variables `X_train` and `y_train`
 - `X_train` contains the test results for the microchips from two tests
 - `y_train` contains the results of the QA
 - `y_train = 1` if the microchip was accepted
 - `y_train = 0` if the microchip was rejected
 - Both `X_train` and `y_train` are numpy arrays.

In [42]:

```
# load dataset
X_train, y_train = load_data("data/ex2data2.txt")
```

View the variables

The code below prints the first five values of `X_train` and `y_train` and the type of the variables.

In [43]:

```
# print X_train
print("X_train:", X_train[:5])
print("Type of X_train:", type(X_train))

# print y_train
print("y_train:", y_train[:5])
print("Type of y_train:", type(y_train))

X_train: [[ 0.051267  0.69956 ]
 [-0.092742  0.68494 ]
 [-0.21371   0.69225 ]
 [-0.375     0.50219 ]
```

```
[-0.51325  0.46564 ]]  
Type of X_train: <class 'numpy.ndarray'>  
y_train: [1. 1. 1. 1. 1.]  
Type of y_train: <class 'numpy.ndarray'>
```

Check the dimensions of your variables

Another useful way to get familiar with your data is to view its dimensions. Let's print the shape of `X_train` and `y_train` and see how many training examples we have in our dataset.

In [44]:

```
print ('The shape of X_train is: ' + str(X_train.shape))  
print ('The shape of y_train is: ' + str(y_train.shape))  
print ('We have m = %d training examples' % (len(y_train)))
```

```
The shape of X_train is: (118, 2)  
The shape of y_train is: (118,)  
We have m = 118 training examples
```

Visualize your data

The helper function `plot_data` (from `utils.py`) is used to generate a figure like Figure 3, where the axes are the two test scores, and the positive ($y = 1$, accepted) and negative ($y = 0$, rejected) examples are shown with different markers.

In [45]:

```
# Plot examples
```

```

plot_data(X_train, y_train[:,], pos_label="Accepted", neg_label="Rejected")

# Set the y-axis label
plt.ylabel('Microchip Test 2')
# Set the x-axis label
plt.xlabel('Microchip Test 1')
plt.legend(loc="upper right")
plt.show()

```

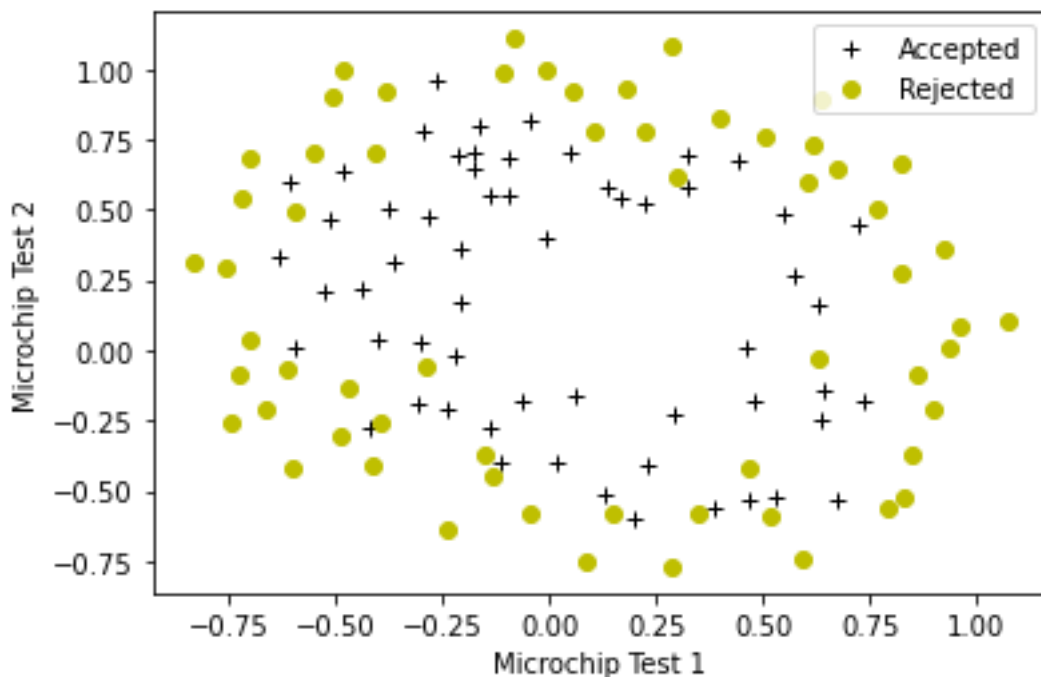


Figure 3 shows that our dataset cannot be separated into positive and negative examples by a straight-line through the plot. Therefore, a straight forward application of logistic regression will not perform well on this dataset since logistic regression will only be able to find a linear decision boundary.

3.3 Feature mapping

One way to fit the data better is to create more features from each data point.

As a result of this mapping, our vector of two features (the scores on two QA tests) has been transformed into a 27-dimensional vector.

- A logistic regression classifier trained on this higher-dimension feature vector will have a more complex decision boundary and will be nonlinear when drawn in our 2-dimensional plot.
- We have provided the `map_feature` function for you in `utils.py`.

In [46]:

```

print("Original shape of data:", X_train.shape)

mapped_X = map_feature(X_train[:, 0], X_train[:, 1])
print("Shape after feature mapping:", mapped_X.shape)

```

Original shape of data: (118, 2)

Shape after feature mapping: (118, 27)

Let's also print the first elements of `X_train` and `mapped_X` to see the transformation.

In [47]:

```
print("X_train[0]:", X_train[0])
print("mapped X_train[0]:", mapped_X[0])

X_train[0]: [0.051267 0.69956 ]
mapped X_train[0]: [5.12670000e-02 6.99560000e-01 2.62830529e-03 3.58643425
e-02
4.89384194e-01 1.34745327e-04 1.83865725e-03 2.50892595e-02
3.42353606e-01 6.90798869e-06 9.42624411e-05 1.28625106e-03
1.75514423e-02 2.39496889e-01 3.54151856e-07 4.83255257e-06
6.59422333e-05 8.99809795e-04 1.22782870e-02 1.67542444e-01
1.81563032e-08 2.47750473e-07 3.38066048e-06 4.61305487e-05
6.29470940e-04 8.58939846e-03 1.17205992e-01]
```

While the feature mapping allows us to build a more expressive classifier, it is also more susceptible to overfitting. In the next parts of the exercise, you will implement regularized logistic regression to fit the data and also see for yourself how regularization can help combat the overfitting problem.

3.4 Cost function for regularized logistic regression

In this part, you will implement the cost function for regularized logistic regression.

Exercise 5

Please complete the `compute_cost_reg`

The starter code then adds this to the cost without regularization (which you computed above in `compute_cost`) to calculate the cost with regularization.

If you get stuck, you can check out the hints presented after the cell below to help you with the implementation.

In [48]:

```
# UNQ_C5
def compute_cost_reg(X, y, w, b, lambda_ = 1):
    """
    Computes the cost over all examples

    Args:
        X : (ndarray Shape (m,n)) data, m examples by n features
        y : (ndarray Shape (m,)) target value
        w : (ndarray Shape (n,)) values of parameters of the model
        b : (scalar) value of bias parameter of the model
        lambda_ : (scalar, float) Controls amount of regularization

    Returns:
        total_cost : (scalar) cost
    """
```

```

m, n = X.shape

# Calls the compute_cost function that you implemented above
cost_without_reg = compute_cost(X, y, w, b)

# You need to calculate this value
reg_cost = 0.

### START CODE HERE ###

reg_cost = lambda_*(1/(2*m))*np.sum(np.square(w))

### END CODE HERE ###

# Add the regularization cost to get the total cost
total_cost = cost_without_reg + reg_cost

return total_cost

```

Click for hints

Run the cell below to check your implementation of the `compute_cost_reg` function.

In [49]:

```

X_mapped = map_feature(X_train[:, 0], X_train[:, 1])
np.random.seed(1)
initial_w = np.random.rand(X_mapped.shape[1]) - 0.5
initial_b = 0.5
lambda_ = 0.5
cost = compute_cost_reg(X_mapped, y_train, initial_w, initial_b, lambda_)

print("Regularized cost :", cost)

# UNIT TEST
compute_cost_reg_test(compute_cost_reg)

Regularized cost : 0.6618252552483951
All tests passed!

```

Expected Output:

```
Regularized cost : 0.6618252552483948
```

3.5 Gradient for regularized logistic regression

In this section, you will implement the gradient for regularized logistic regression.

Exercise 6

Please complete the `compute_gradient_reg` function below.

In [53]:

```

# UNQ_C6
def compute_gradient_reg(X, y, w, b, lambda_ = 1):
    """
    Computes the gradient for logistic regression with regularization

    Args:
        X : (ndarray Shape (m,n)) data, m examples by n features
        y : (ndarray Shape (m,)) target value
        w : (ndarray Shape (n,)) values of parameters of the model
        b : (scalar) value of bias parameter of the model
        lambda_ : (scalar,float) regularization constant
    Returns
        dj_db : (scalar) The gradient of the cost w.r.t. the parameter b.
        dj_dw : (ndarray Shape (n,)) The gradient of the cost w.r.t. the parameters w.

    """
    m, n = X.shape

    dj_db, dj_dw = compute_gradient(X, y, w, b)

    ### START CODE HERE ###

    for i in range(w.shape[0]):
        dj_dw[i] += lambda_/m*w[i]

    ### END CODE HERE ###

    return dj_db, dj_dw

```

Run the cell below to check your implementation of the `compute_gradient_reg` function.

In [54]:

```

X_mapped = map_feature(X_train[:, 0], X_train[:, 1])
np.random.seed(1)
initial_w = np.random.rand(X_mapped.shape[1]) - 0.5
initial_b = 0.5

lambda_ = 0.5
dj_db, dj_dw = compute_gradient_reg(X_mapped, y_train, initial_w, initial_b,
, lambda_)

print(f"dj_db: {dj_db}", )
print(f"First few elements of regularized dj_dw:\n {dj_dw[:4].tolist()}", )

# UNIT TESTS
compute_gradient_reg_test(compute_gradient_reg)

dj_db: 0.07138288792343654

```

First few elements of regularized dj_dw :

```
[-0.010386028450548689, 0.011409852883280116, 0.05362734632745741, 0.0031402782673134602]
```

All tests passed!

Expected Output:

```
dj_db:0.07138288792343
```

First few elements of regularized dj_dw :

```
[[-0.010386028450548], [0.011409852883280], [0.0536273463274], [0.003140278267313]]
```

3.6 Learning parameters using gradient descent

Similar to the previous parts, you will use your gradient descent function implemented above to learn the optimal parameters

- If you have completed the cost and gradient for regularized logistic regression correctly, you should be able to step through the next cell to learn the parameters $\mathbf{w}$.
- After training our parameters, we will use it to plot the decision boundary.

Note

The code block below takes quite a while to run, especially with a non-vectorized version. You can reduce the `iterations` to test your implementation and iterate faster. If you have time later, run for 100,000 iterations to see better results.

In [55]:

```
# Initialize fitting parameters
np.random.seed(1)
initial_w = np.random.rand(X_mapped.shape[1]) - 0.5
initial_b = 1.

# Set regularization parameter lambda_ (you can try varying this)
lambda_ = 0.01

# Some gradient descent settings
iterations = 10000
alpha = 0.01

w, b, J_history, _ = gradient_descent(X_mapped, y_train, initial_w, initial_b,
                                     compute_cost_reg, compute_gradient_reg,
                                     alpha, iterations, lambda_)
```

Expected Output: Cost < 0.5 (Click for details)

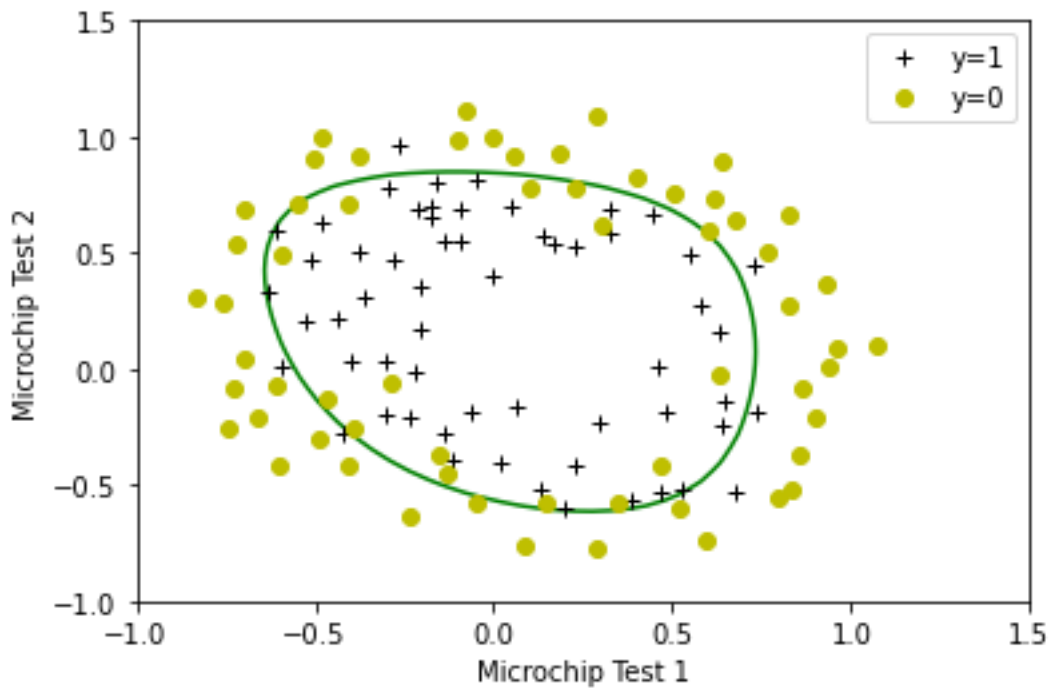
3.7 Plotting the decision boundary

To help you visualize the model learned by this classifier, we will use our `plot_decision_boundary` function which plots the (non-linear) decision boundary that separates the positive and negative examples.

- In the function, we plotted the non-linear decision boundary by computing the classifier's predictions on an evenly spaced grid and then drew a contour plot of where the predictions change from $y = 0$ to $y = 1$.
- After learning the parameters w and b , the next step is to plot a decision boundary similar to Figure 4.

In [56]:

```
plot_decision_boundary(w, b, X_mapped, y_train)
# Set the y-axis label
plt.ylabel('Microchip Test 2')
# Set the x-axis label
plt.xlabel('Microchip Test 1')
plt.legend(loc="upper right")
plt.show()
```



3.8 Evaluating regularized logistic regression model

You will use the `predict` function that you implemented above to calculate the accuracy of the regularized logistic regression model on the training set

In [57]:

```
#Compute accuracy on the training set
p = predict(X_mapped, w, b)

print('Train Accuracy: %f'%(np.mean(p == y_train) * 100))
```

Train Accuracy: 82.203390

Expected Output:

Train Accuracy:~ 80%

Artificial Intelligence

Lab: 12

Lab 12

Practice Lab: Neural Networks for Handwritten Digit Recognition, Binary

In this exercise, you will use a neural network to recognize the hand-written digits zero and one.

Outline

- [1 - Packages](#)
- [2 - Neural Networks](#)
 - [2.1 Problem Statement](#)
 - [2.2 Dataset](#)
 - [2.3 Model representation](#)
 - [2.4 Tensorflow Model Implementation](#)
 - [Exercise 1](#)
 - [2.5 NumPy Model Implementation \(Forward Prop in NumPy\)](#)
 - [Exercise 2](#)
 - [2.6 Vectorized NumPy Model Implementation \(Optional\)](#)
 - [Exercise 3](#)
 - [2.7 Congratulations!](#)
 - [2.8 NumPy Broadcasting Tutorial \(Optional\)](#)

NOTE: To prevent errors from the autograder, you are not allowed to edit or delete non-graded cells in this notebook . Please also refrain from adding any new cells. **Once you have passed this assignment** and want to experiment with any of the non-graded code, you may follow the instructions at the bottom of this notebook.

1 - Packages

First, let's run the cell below to import all the packages that you will need during this assignment.

- [numpy](#) is the fundamental package for scientific computing with Python.
- [matplotlib](#) is a popular library to plot graphs in Python.
- [tensorflow](#) a popular platform for machine learning.

In [1]:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt
from autils import *
%matplotlib inline
```

```
import logging
logging.getLogger("tensorflow").setLevel(logging.ERROR)
tf.autograph.set_verbosity(0)
```

Tensorflow and Keras

Tensorflow is a machine learning package developed by Google. In 2019, Google integrated Keras into Tensorflow and released Tensorflow 2.0. Keras is a framework developed independently by François Chollet that creates a simple, layer-centric interface to Tensorflow. This course will be using the Keras interface.

2 - Neural Networks

In Course 1, you implemented logistic regression. This was extended to handle non-linear boundaries using polynomial regression. For even more complex scenarios such as image recognition, neural networks are preferred.

2.1 Problem Statement

In this exercise, you will use a neural network to recognize two handwritten digits, zero and one. This is a binary classification task. Automated handwritten digit recognition is widely used today - from recognizing zip codes (postal codes) on mail envelopes to recognizing amounts written on bank checks. You will extend this network to recognize all 10 digits (0-9) in a future assignment.

This exercise will show you how the methods you have learned can be used for this classification task.

2.2 Dataset

You will start by loading the dataset for this task.

- The `load_data()` function shown below loads the data into variables $\mathbf{x}$ and $\mathbf{y}$
- The data set contains 1000 training examples of handwritten digits 11, here limited to zero and one.
 - Each training example is a 20-pixel x 20-pixel grayscale image of the digit.
 - Each pixel is represented by a floating-point number indicating the grayscale intensity at that location.
 - The 20 by 20 grid of pixels is “unrolled” into a 400-dimensional vector.
 - Each training example becomes a single row in our data matrix $\mathbf{x}$.
 - This gives us a 1000 x 400 matrix $\mathbf{x}$ where every row is a training example of a handwritten digit image.
- The second part of the training set is a 1000 x 1 dimensional vector $\mathbf{y}$ that contains labels for the training set
 - $y = 0$ if the image is of the digit 0, $y = 1$ if the image is of the digit 1.

This is a subset of the MNIST handwritten digit dataset
(<http://yann.lecun.com/exdb/mnist/>)

In [2]:

```
# load dataset
X, y = load_data()
```

2.2.1 View the variables

Let's get more familiar with your dataset.

- A good place to start is to print out each variable and see what it contains.

The code below prints elements of the variables `X` and `y`.

In [3]:

```
print ('The first element of X is: ', X[0])
In [4]:
print ('The first element of y is: ', y[0,0])
print ('The last element of y is: ', y[-1,0])
The first element of y is:  0
The last element of y is:  1
```

2.2.2 Check the dimensions of your variables

Another way to get familiar with your data is to view its dimensions. Please print the shape of `X` and `y` and see how many training examples you have in your dataset.

In [5]:

```
print ('The shape of X is: ' + str(X.shape))
print ('The shape of y is: ' + str(y.shape))
The shape of X is: (1000, 400)
The shape of y is: (1000, 1)
```

2.2.3 Visualizing the Data

You will begin by visualizing a subset of the training set.

- In the cell below, the code randomly selects 64 rows from `X`, maps each row back to a 28 pixel by 28 pixel grayscale image and displays the images together.
- The label for each image is displayed above the image

In [6]:

```
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
# You do not need to modify anything in this cell
```

```
m, n = X.shape

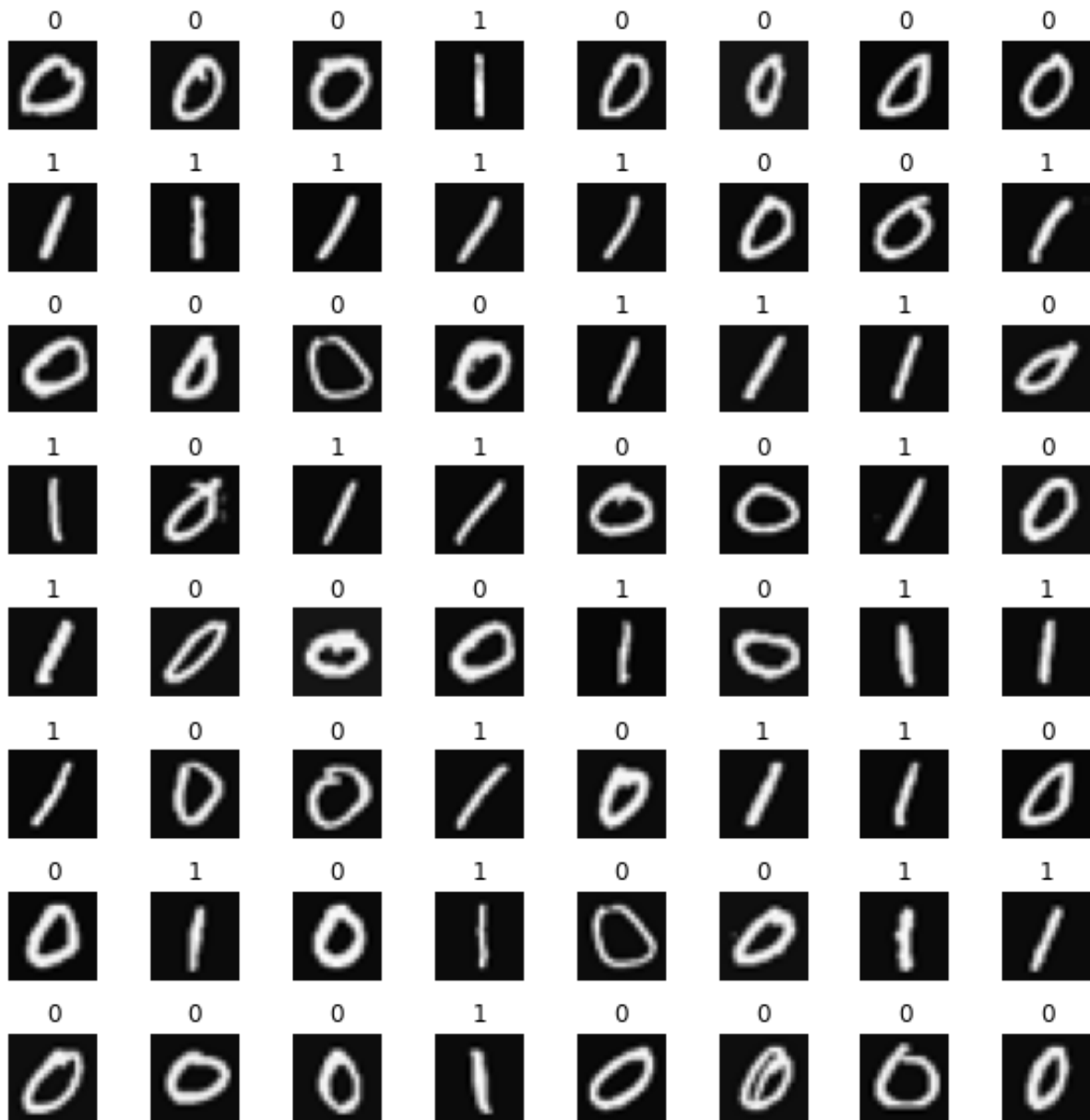
fig, axes = plt.subplots(8,8, figsize=(8,8))
fig.tight_layout(pad=0.1)

for i,ax in enumerate(axes.flat):
    # Select random indices
    random_index = np.random.randint(m)

    # Select rows corresponding to the random indices and
    # reshape the image
    X_random_resaped = X[random_index].reshape((20,20)).T

    # Display the image
    ax.imshow(X_random_resaped, cmap='gray')

    # Display the label above the image
    ax.set_title(y[random_index,0])
    ax.set_axis_off()
```



2.3 Model representation

The neural network you will use in this assignment is shown in the figure below.

- This has three dense layers with sigmoid activations.
 - Recall that our inputs are pixel values of digit images.
 - Since the images are of size $28 \times 28 \times 1$, this gives us 784 inputs

- The parameters have dimensions that are sized for a neural network with 2525 units in layer 1, 1515 units in layer 2 and 11 output unit in layer 3.
 - Therefore, the shapes of $\bar{w}$, and $\bar{b}$, are
 - layer1: The shape of $\bar{w}_1$ is (400, 25) and the shape of $\bar{b}_1$ is (25,)
 - layer2: The shape of $\bar{w}_2$ is (25, 15) and the shape of $\bar{b}_2$ is: (15,)
 - layer3: The shape of $\bar{w}_3$ is (15, 1) and the shape of $\bar{b}_3$ is: (1,)

Note: The bias vector $\bar{b}$ could be represented as a 1-D (n,) or 2-D (1,n) array. Tensorflow utilizes a 1-D representation and this lab will maintain that convention.

2.4 Tensorflow Model Implementation

Tensorflow models are built layer by layer. A layer's input dimensions (See diagram above) are calculated for you. You specify a layer's *output dimensions* and this determines the next layer's input dimension. The input dimension of the first layer is derived from the size of the input data specified in the `model.fit` statement below.

Note: It is also possible to add an input layer that specifies the input dimension of the first layer. For example:

```
tf.keras.Input(shape=(400,)), #specify input shape
```

We will include that here to illuminate some model sizing.

Exercise 1

Below, using Keras [Sequential model](#) and [Dense Layer](#) with a sigmoid activation to construct the network described above.

In [13]:

```
# UNQ_C1
# GRADED CELL: Sequential model

model = Sequential(
    [
        tf.keras.Input(shape=(400,)),      #specify input size
        ### START CODE HERE ###
        Dense(25, activation='sigmoid', name = 'layer1'),
        Dense(15, activation='sigmoid', name = 'layer2'),
        Dense(1, activation='sigmoid', name = 'layer3')
        ### END CODE HERE ###
    ], name = "my_model"
)
```

In [14]:

```
model.summary()
Model: "my_model"
```

Layer (type)	Output Shape	Param #
=====		
layer1 (Dense)	(None, 25)	10025
layer2 (Dense)	(None, 15)	390
layer3 (Dense)	(None, 1)	16
=====		
Total params: 10,431		
Trainable params: 10,431		
Non-trainable params: 0		

Expected Output (Click to Expand)

Click for hints

In [15]:

```
# UNIT TESTS
from public_tests import *

test_c1(model)

All tests passed!
```

The parameter counts shown in the summary correspond to the number of elements in the weight and bias arrays as shown below.

In [16]:

```
L1_num_params = 400 * 25 + 25 # W1 parameters + b1 parameters
L2_num_params = 25 * 15 + 15  # W2 parameters + b2 parameters
L3_num_params = 15 * 1 + 1    # W3 parameters + b3 parameters
print("L1 params = ", L1_num_params, ", L2 params = ", L2_num_params, ", L3 params = ", L3_num_params )
```

L1 params = 10025 , L2 params = 390 , L3 params = 16

We can examine details of the model by first extracting the layers with `model.layers` and then extracting the weights with `layerx.get_weights()` as shown below.

In [17]:

```
[layer1, layer2, layer3] = model.layers
```

In [18]:

```
#### Examine Weights shapes
W1,b1 = layer1.get_weights()
W2,b2 = layer2.get_weights()
W3,b3 = layer3.get_weights()
print(f"W1 shape = {W1.shape}, b1 shape = {b1.shape}")
print(f"W2 shape = {W2.shape}, b2 shape = {b2.shape}")
print(f"W3 shape = {W3.shape}, b3 shape = {b3.shape}")

W1 shape = (400, 25), b1 shape = (25,)
W2 shape = (25, 15), b2 shape = (15,)
W3 shape = (15, 1), b3 shape = (1,)
```

Expected Output

```
W1 shape = (400, 25), b1 shape = (25,)
```

```
W2 shape = (25, 15), b2 shape = (15,)
```

```
W3 shape = (15, 1), b3 shape = (1,)
```

`xx.get_weights` returns a NumPy array. One can also access the weights directly in their tensor form. Note the shape of the tensors in the final layer.

In [19]:

```
print(model.layers[2].weights)

[<tf.Variable 'layer3/kernel:0' shape=(15, 1) dtype=float32, numpy=
array([[ -0.32046527],
       [ 0.29993433],
       [ 0.23611599],
       [ 0.07300431],
       [ 0.2574625 ],
       [ 0.40102762],
       [-0.42942435],
       [ 0.18527728],
       [ 0.41227418],
       [-0.37742114],
       [ 0.04011494],
       [-0.46928918],
       [ 0.33736765],
       [ 0.07295626],
       [-0.03607988]], dtype=float32)>, <tf.Variable 'layer3/bias:0' shape=
(1,) dtype=float32, numpy=array([0.], dtype=float32)>]
```

The following code will define a loss function and run gradient descent to fit the weights of the model to the training data. This will be explained in more detail in the following week.

In [20]:

```
model.compile(
    loss=tf.keras.losses.BinaryCrossentropy(),
    optimizer=tf.keras.optimizers.Adam(0.001),
)
```

```
model.fit(
    X,y,
    epochs=20
)
```

```
<keras.callbacks.History at 0x7fa92f8f75d0>
```

To run the model on an example to make a prediction, use `model.predict`. The input to `predict` is an array so the single example is reshaped to be two dimensional.

In [21]:

```
prediction = model.predict(X[0].reshape(1,400)) # a zero
print(f" predicting a zero: {prediction}")
prediction = model.predict(X[500].reshape(1,400)) # a one
print(f" predicting a one: {prediction}")

predicting a zero: [[0.0206227]]
predicting a one: [[0.9879255]]
```

The output of the model is interpreted as a probability. In the first example above, the input is a zero. The model predicts the probability that the input is a one is nearly zero. In the second example, the input is a one. The model predicts the probability that the input is a one is nearly one. As in the case of logistic regression, the probability is compared to a threshold to make a final prediction.

In [22]:

```
if prediction >= 0.5:
    yhat = 1
else:
    yhat = 0
print(f"prediction after threshold: {yhat}")

prediction after threshold: 1
```

Let's compare the predictions vs the labels for a random sample of 64 digits. This takes a moment to run.

In [23]:

```
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
# You do not need to modify anything in this cell

m, n = X.shape

fig, axes = plt.subplots(8,8, figsize=(8,8))
fig.tight_layout(pad=0.1,rect=[0, 0.03, 1, 0.92]) #[left, bottom, right, to
p]

for i,ax in enumerate(axes.flat):
```

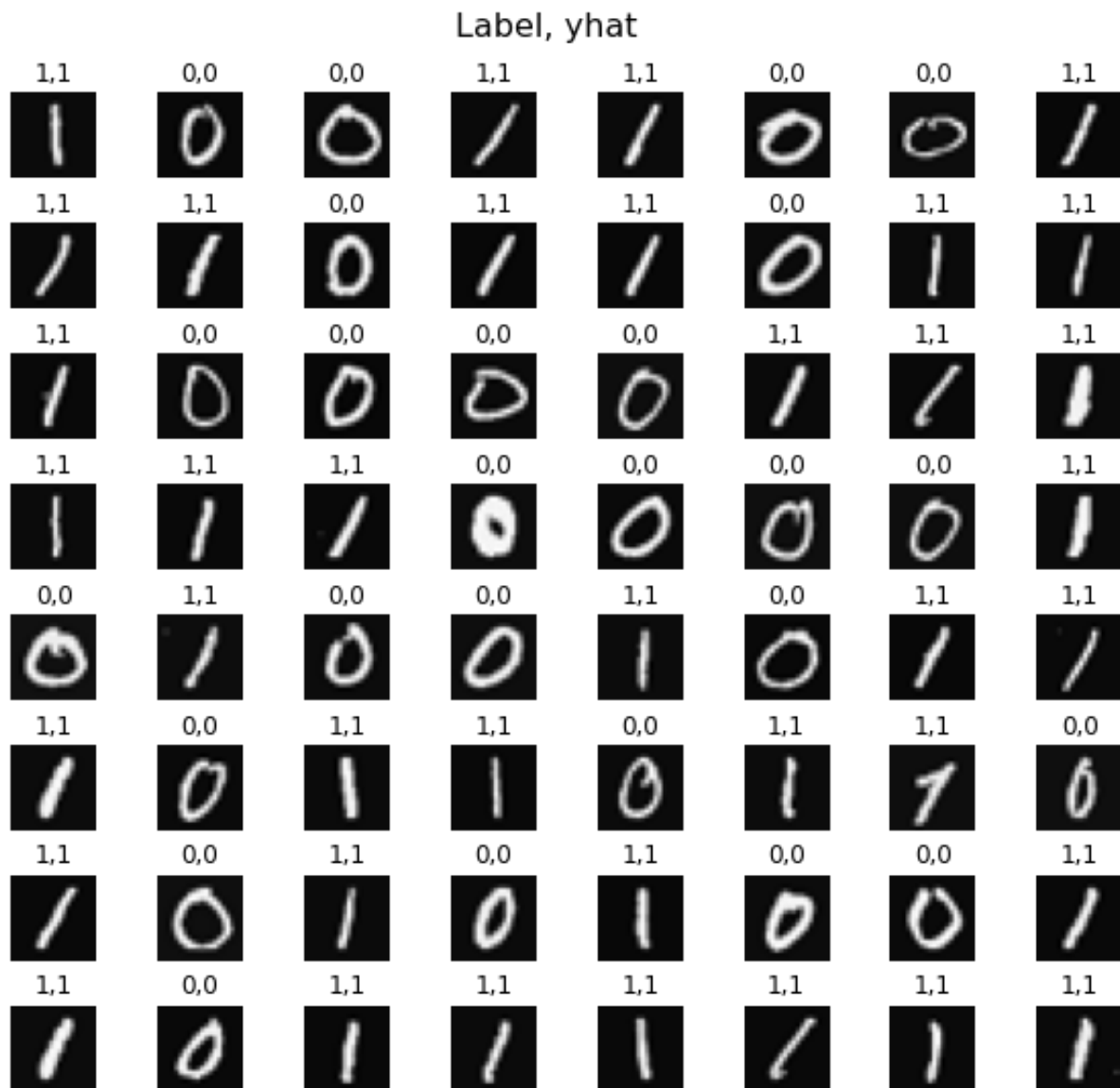
```
# Select random indices
random_index = np.random.randint(m)

# Select rows corresponding to the random indices and
# reshape the image
X_random_reshaped = X[random_index].reshape((20,20)).T

# Display the image
ax.imshow(X_random_reshaped, cmap='gray')

# Predict using the Neural Network
prediction = model.predict(X[random_index].reshape(1,400))
if prediction >= 0.5:
    yhat = 1
else:
    yhat = 0

# Display the label above the image
ax.set_title(f"{y[random_index,0]}, {yhat}")
ax.set_axis_off()
fig.suptitle("Label, yhat", fontsize=16)
plt.show()
```



2.5 NumPy Model Implementation (Forward Prop in NumPy)

As described in lecture, it is possible to build your own dense layer using NumPy. This can then be utilized to build a multi-layer neural network.

Exercise 2

Below, build a dense layer subroutine. The example in lecture utilized a for loop to visit each unit (j) in the layer and perform the dot product of the weights for that unit ($w[:, j]$) and sum the bias for the unit ($b[j]$) to form z . An activation function $g(z)$ is then applied to that result. This section will not utilize some of the matrix operations described in the optional lectures. These will be explored in a later section.

In [24]:

```
# UNQ_C2
# GRADED FUNCTION: my_dense
```

```
def my_dense(a_in, W, b, g):
    """
    Computes dense layer
    Args:
        a_in (ndarray (n, )) : Data, 1 example
        W     (ndarray (n,j)) : Weight matrix, n features per unit, j units
        b     (ndarray (j, )) : bias vector, j units
        g     activation function (e.g. sigmoid, relu..)
    Returns
        a_out (ndarray (j,)) : j units
    """
    units = W.shape[1]
    a_out = np.zeros(units)
    ### START CODE HERE ###

    for j in range(units):
        # Calculate z = w.T * a_in + b
        z = np.dot(W[:, j], a_in) + b[j]
        # Apply the activation function
        a_out[j] = g(z)

    ### END CODE HERE ###
    return(a_out)
```

In [25]:

```
# Quick Check
x_tst = 0.1*np.arange(1,3,1).reshape(2,) # (1 examples, 2 features)
W_tst = 0.1*np.arange(1,7,1).reshape(2,3) # (2 input features, 3 output features)
b_tst = 0.1*np.arange(1,4,1).reshape(3,) # (3 features)
A_tst = my_dense(x_tst, W_tst, b_tst, sigmoid)
print(A_tst)

[0.54735762 0.57932425 0.61063923]
```

Expected Output

```
[0.54735762 0.57932425 0.61063923]
```

Click for hints

In [26]:

```
# UNIT TESTS
```

```
test_c2(my_dense)
```

```
All tests passed!
```

The following cell builds a three-layer neural network utilizing the `my_dense` subroutine above.

In [27]:

```
def my_sequential(x, W1, b1, W2, b2, W3, b3):
```

```

a1 = my_dense(x, W1, b1, sigmoid)
a2 = my_dense(a1, W2, b2, sigmoid)
a3 = my_dense(a2, W3, b3, sigmoid)
return (a3)

```

We can copy trained weights and biases from Tensorflow.

In [28]:

```

W1_tmp,b1_tmp = layer1.get_weights()
W2_tmp,b2_tmp = layer2.get_weights()
W3_tmp,b3_tmp = layer3.get_weights()

```

In [29]:

```

# make predictions
prediction = my_sequential(X[0], W1_tmp, b1_tmp, W2_tmp, b2_tmp, W3_tmp, b3_tmp)
if prediction >= 0.5:
    yhat = 1
else:
    yhat = 0
print( "yhat = ", yhat, " label= ", y[0,0])
prediction = my_sequential(X[500], W1_tmp, b1_tmp, W2_tmp, b2_tmp, W3_tmp, b3_tmp)
if prediction >= 0.5:
    yhat = 1
else:
    yhat = 0
print( "yhat = ", yhat, " label= ", y[500,0])

yhat = 0 label= 0
yhat = 1 label= 1

```

Run the following cell to see predictions from both the Numpy model and the Tensorflow model. This takes a moment to run.

In [30]:

```

import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
# You do not need to modify anything in this cell

m, n = X.shape

fig, axes = plt.subplots(8,8, figsize=(8,8))
fig.tight_layout(pad=0.1,rect=[0, 0.03, 1, 0.92]) #[left, bottom, right, top]

for i,ax in enumerate(axes.flat):
    # Select random indices
    random_index = np.random.randint(m)

    # Select rows corresponding to the random indices and
    # reshape the image

```

```

X_random_resaped = X[random_index].reshape((20,20)).T

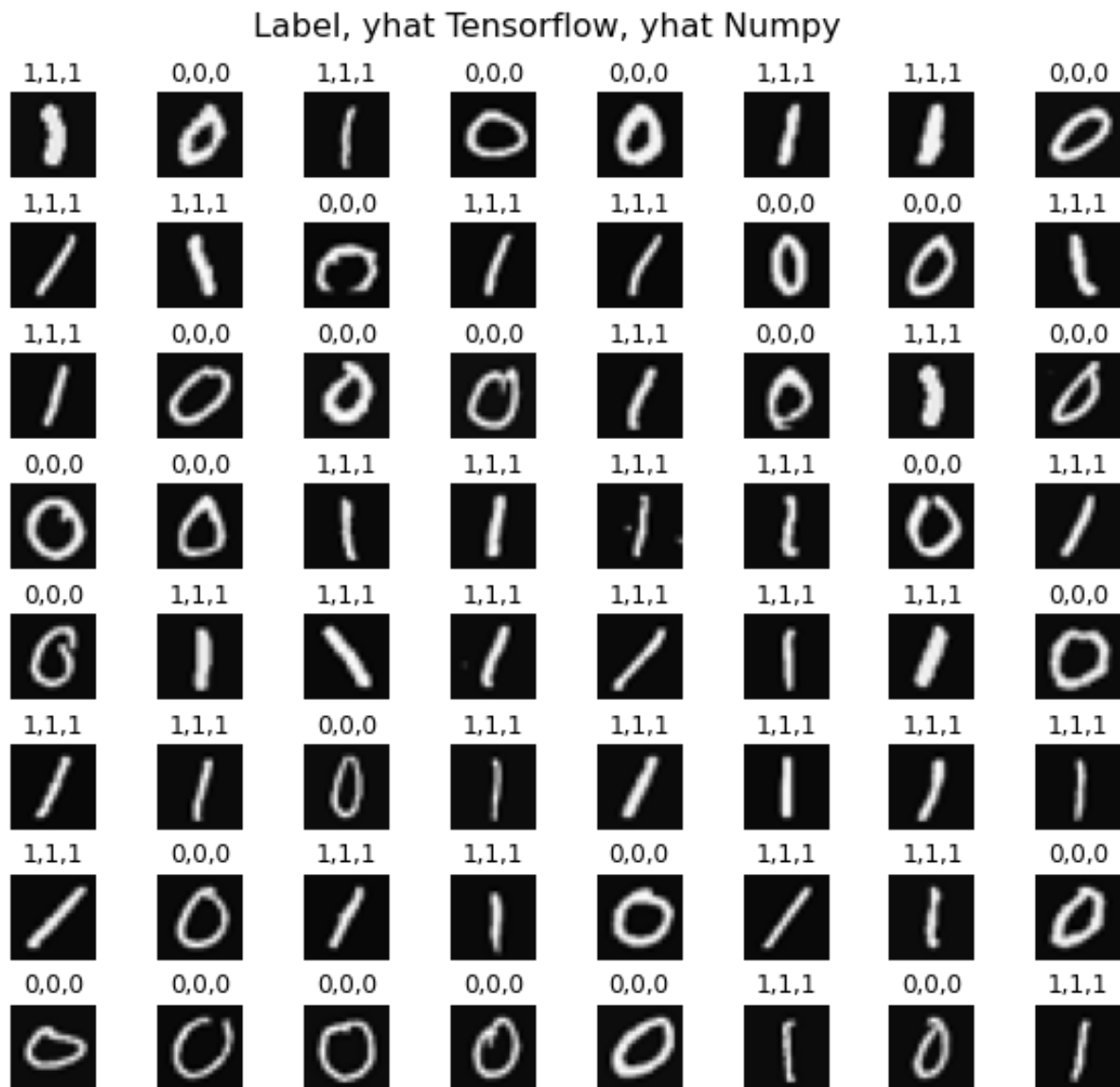
# Display the image
ax.imshow(X_random_resaped, cmap='gray')

# Predict using the Neural Network implemented in Numpy
my_prediction = my_sequential(X[random_index], W1_tmp, b1_tmp, W2_tmp,
b2_tmp, W3_tmp, b3_tmp )
my_yhat = int(my_prediction >= 0.5)

# Predict using the Neural Network implemented in Tensorflow
tf_prediction = model.predict(X[random_index].reshape(1,400))
tf_yhat = int(tf_prediction >= 0.5)

# Display the label above the image
ax.set_title(f"{y[random_index,0]},{tf_yhat},{my_yhat}")
ax.set_axis_off()
fig.suptitle("Label, yhat Tensorflow, yhat Numpy", fontsize=16)
plt.show()

```



2.6 Vectorized NumPy Model Implementation (Optional)

The optional lectures described vector and matrix operations that can be used to speed the calculations. Below describes a layer operation that computes the output for all units in a layer on a given input example:

We can demonstrate this using the examples `x` and the `w1,b1` parameters above. We use `np.matmul` to perform the matrix multiply. Note, the dimensions of `x` and `W` must be compatible as shown in the diagram above.

In [31]:

```
x = X[0].reshape(-1,1)           # column vector (400,1)
z1 = np.matmul(x.T,W1) + b1       # (1,400) (400,25) = (1,25)
a1 = sigmoid(z1)
print(a1.shape)

(1, 25)
```

You can take this a step further and compute all the units for all examples in one Matrix-Matrix operation.

Exercise 3

Below, compose a new `my_dense_v` subroutine that performs the layer calculations for a matrix of examples. This will utilize `np.matmul()`.

Note: This function is not graded because it is discussed in the optional lectures on vectorization. If you didn't go through them, feel free to click the hints below the expected code to see the code. You can also submit the notebook even with a blank answer here.

In [32]:

```
# UNQ_C3
# UNGRADED FUNCTION: my_dense_v

def my_dense_v(A_in, W, b, g):
    """
    Computes dense layer
    Args:
        A_in (ndarray (m,n)) : Data, m examples, n features each
        W      (ndarray (n,j)) : Weight matrix, n features per unit, j units
        b      (ndarray (1,j)) : bias vector, j units
        g      activation function (e.g. sigmoid, relu..)
    Returns
        A_out (tf.Tensor or ndarray (m,j)) : m examples, j units
    """
    ### START CODE HERE ###
    Z = np.matmul(A_in, W) + b
    A_out = g(Z)
    ### END CODE HERE ###
    return A_out
```

In [33]:

```
X_tst = 0.1*np.arange(1,9,1).reshape(4,2) # (4 examples, 2 features)
W_tst = 0.1*np.arange(1,7,1).reshape(2,3) # (2 input features, 3 output fea
tures)
b_tst = 0.1*np.arange(1,4,1).reshape(1,3) # (1,3 features)
A_tst = my_dense_v(X_tst, W_tst, b_tst, sigmoid)
print(A_tst)

[[0.54735762 0.57932425 0.61063923]
 [0.57199613 0.61301418 0.65248946]
 [0.5962827  0.64565631 0.6921095 ]
 [0.62010643 0.67699586 0.72908792]]
```

Expected Output

```
[[0.54735762 0.57932425 0.61063923]
 [0.57199613 0.61301418 0.65248946]
```

```
[0.5962827  0.64565631 0.6921095 ]
[0.62010643 0.67699586 0.72908792]]
```

Click for hints

In [34]:

```
# UNIT TESTS
```

```
test_c3(my_dense_v)
```

```
All tests passed!
```

The following cell builds a three-layer neural network utilizing the `my_dense_v` subroutine above.

In [35]:

```
def my_sequential_v(X, W1, b1, W2, b2, W3, b3):
    A1 = my_dense_v(X, W1, b1, sigmoid)
    A2 = my_dense_v(A1, W2, b2, sigmoid)
    A3 = my_dense_v(A2, W3, b3, sigmoid)
    return(A3)
```

We can again copy trained weights and biases from Tensorflow.

In [36]:

```
W1_tmp,b1_tmp = layer1.get_weights()
W2_tmp,b2_tmp = layer2.get_weights()
W3_tmp,b3_tmp = layer3.get_weights()
```

Let's make a prediction with the new model. This will make a prediction on *all of the examples at once*. Note the shape of the output.

In [37]:

```
Prediction = my_sequential_v(X, W1_tmp, b1_tmp, W2_tmp, b2_tmp, W3_tmp, b3_
tmp )
Prediction.shape
```

Out[37]:

```
(1000, 1)
```

We'll apply a threshold of 0.5 as before, but to all predictions at once.

In [38]:

```
Yhat = (Prediction >= 0.5).astype(int)
print("predict a zero: ",Yhat[0], "predict a one: ", Yhat[500])
predict a zero:  [0] predict a one:  [1]
```

Run the following cell to see predictions. This will use the predictions we just calculated above. This takes a moment to run.

In [39]:

```
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
# You do not need to modify anything in this cell

m, n = X.shape

fig, axes = plt.subplots(8, 8, figsize=(8, 8))
```

```

fig.tight_layout(pad=0.1, rect=[0, 0.03, 1, 0.92]) #[left, bottom, right, top]

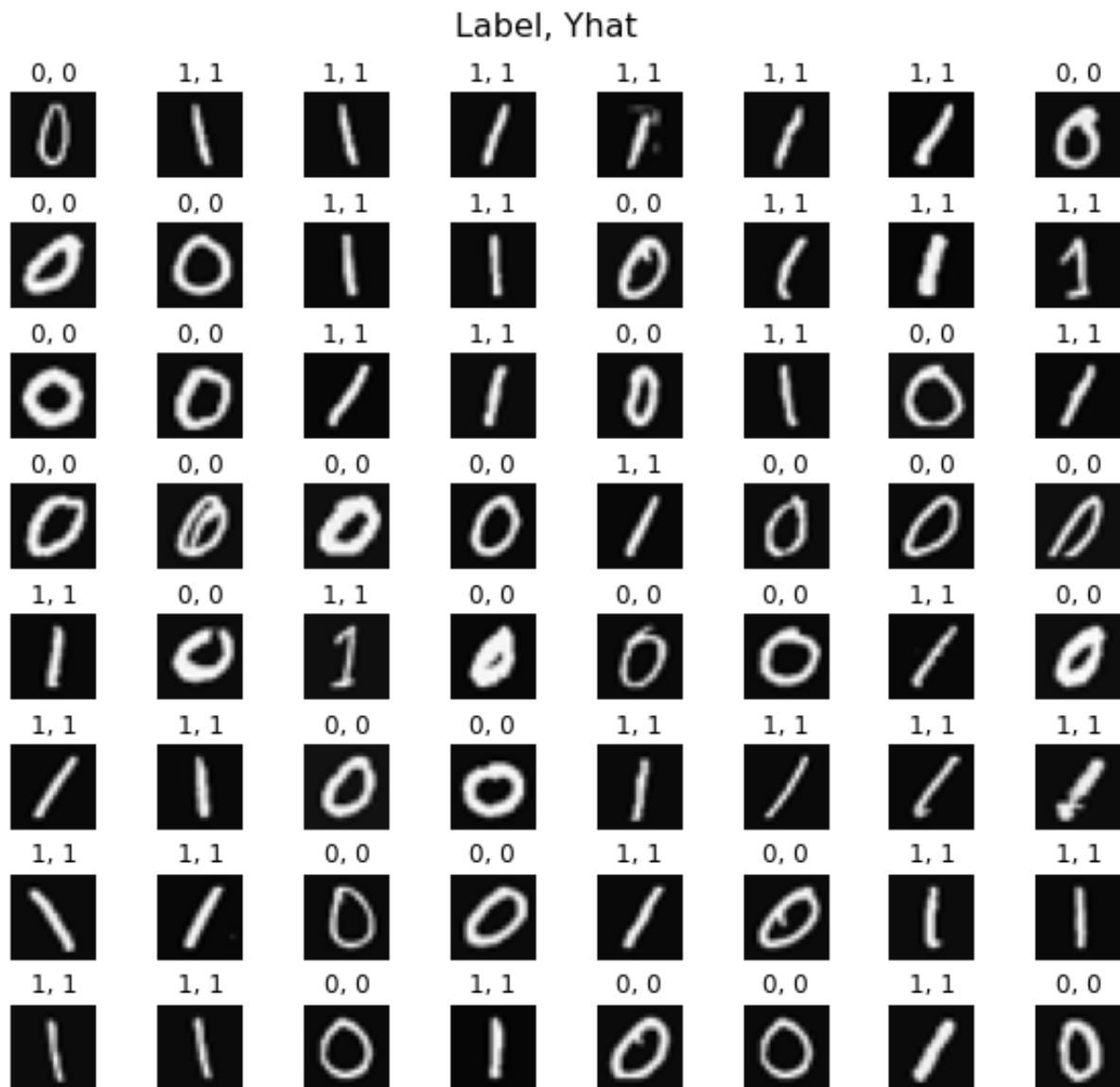
for i, ax in enumerate(axes.flat):
    # Select random indices
    random_index = np.random.randint(m)

    # Select rows corresponding to the random indices and
    # reshape the image
    X_random_resaped = X[random_index].reshape((20, 20)).T

    # Display the image
    ax.imshow(X_random_resaped, cmap='gray')

    # Display the label above the image
    ax.set_title(f"{y[random_index,0]}, {Yhat[random_index, 0]}")
    ax.set_axis_off()
fig.suptitle("Label, Yhat", fontsize=16)
plt.show()

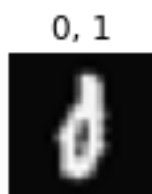
```



You can see how one of the misclassified images looks.

In [40]:

```
fig = plt.figure(figsize=(1, 1))
errors = np.where(y != Yhat)
random_index = errors[0][0]
X_random_resaped = X[random_index].reshape((20, 20)).T
plt.imshow(X_random_resaped, cmap='gray')
plt.title(f"{y[random_index,0]}, {Yhat[random_index, 0]}")
plt.axis('off')
plt.show()
```



Artificial Intelligence

Lab: 13

Lab 13

Neural Networks for Handwritten Digit Recognition, Multiclass

In this exercise, you will use a neural network to recognize the hand-written digits 0-9.

Outline

- [1 - Packages](#)
- [2 - ReLU Activation](#)
- [3 - Softmax Function](#)
 - [Exercise 1](#)
- [4 - Neural Networks](#)
 - [4.1 Problem Statement](#)
 - [4.2 Dataset](#)
 - [4.3 Model representation](#)
 - [4.4 Tensorflow Model Implementation](#)
 - [4.5 Softmax placement](#)
 - [Exercise 2](#)

NOTE: To prevent errors from the autograder, you are not allowed to edit or delete non-graded cells in this notebook . Please also refrain from adding any new cells. **Once you have passed this assignment** and want to experiment with any of the non-graded code, you may follow the instructions at the bottom of this notebook.

1 - Packages

First, let's run the cell below to import all the packages that you will need during this assignment.

- [numpy](#) is the fundamental package for scientific computing with Python.
- [matplotlib](#) is a popular library to plot graphs in Python.
- [tensorflow](#) a popular platform for machine learning.

In [1]:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.activations import linear, relu, sigmoid
%matplotlib widget
import matplotlib.pyplot as plt
plt.style.use('./deeplearning.mplstyle')
```

```
import logging
logging.getLogger("tensorflow").setLevel(logging.ERROR)
tf.autograph.set_verbosity(0)

from public_tests import *

from autils import *
from lab_utils_softmax import plt_softmax
np.set_printoptions(precision=2)
```

2 - ReLU Activation

This week, a new activation was introduced, the Rectified Linear Unit (ReLU).

In [2]:

```
plt_act_trio()
```

The example from the lecture on the right shows an application of the ReLU. In this example, the derived "awareness" feature is not binary but has a continuous range of values. The sigmoid is best for on/off or binary situations. The ReLU provides a continuous linear relationship. Additionally it has an 'off' range where the output is zero.

The "off" feature makes the ReLU a Non-Linear activation. Why is this needed? This enables multiple units to contribute to the resulting function without interfering. This is examined more in the supporting optional lab.

3 - Softmax Function

A multiclass neural network generates N outputs. One output is selected as the predicted answer. In the output layer, a vector $\mathbf{z}$ is generated by a linear function which is fed into a softmax function. The softmax function converts $\mathbf{z}$ into a probability distribution as described below. After applying softmax, each output will be between 0 and 1 and the outputs will sum to 1. They can be interpreted as probabilities. The larger inputs to the softmax will correspond to larger output probabilities.

Exercise 1

Let's create a NumPy implementation:

In [3]:

```
# UNQ_C1
# GRADED CELL: my_softmax

def my_softmax(z):
    """ Softmax converts a vector of values to a probability distribution.
    Args:
        z (ndarray (N,)) : input data, N features
    Returns:
        a (ndarray (N,)) : softmax of z
    """
    ### START CODE HERE ###
```

```

# 1. Exponentiate each element in the input vector z
exp_z = np.exp(z)

# 2. Calculate the sum of all exponentiated elements
sum_exp_z = np.sum(exp_z)

# 3. Normalize each exponentiated element by the sum to obtain probabilities
a = exp_z / sum_exp_z

### END CODE HERE ###
return a

```

In [4]:

```

z = np.array([1., 2., 3., 4.])
a = my_softmax(z)
atf = tf.nn.softmax(z)
print(f"my_softmax(z): {a}")
print(f"tensorflow softmax(z): {atf}")

# BEGIN UNIT TEST
test_my_softmax(my_softmax)
# END UNIT TEST

my_softmax(z): [0.03 0.09 0.24 0.64]
tensorflow softmax(z): [0.03 0.09 0.24 0.64]
All tests passed.

```

Click for hints

Below, vary the values of the z inputs. Note in particular how the exponential in the numerator magnifies small differences in the values. Note as well that the output values sum to one.

In [5]:

```

plt.close("all")
plt_softmax(my_softmax)

```

4 - Neural Networks

In last weeks assignment, you implemented a neural network to do binary classification. This week you will extend that to multiclass classification. This will utilize the softmax activation.

4.1 Problem Statement

In this exercise, you will use a neural network to recognize ten handwritten digits, 0-9. This is a multiclass classification task where one of n choices is selected. Automated handwritten digit recognition is widely used today - from recognizing zip codes (postal codes) on mail envelopes to recognizing amounts written on bank checks.

4.2 Dataset

You will start by loading the dataset for this task.

- The `load_data()` function shown below loads the data into variables `X` and `y`
- The data set contains 5000 training examples of handwritten digits 11.
 - Each training example is a 20-pixel x 20-pixel grayscale image of the digit.
 - Each pixel is represented by a floating-point number indicating the grayscale intensity at that location.
 - The 20 by 20 grid of pixels is “unrolled” into a 400-dimensional vector.
 - Each training examples becomes a single row in our data matrix `X`.
 - This gives us a 5000 x 400 matrix `X` where every row is a training example of a handwritten digit image.
 - The second part of the training set is a 5000 x 1 dimensional vector `y` that contains labels for the training set
 - `y = 0` if the image is of the digit 0, `y = 4` if the image is of the digit 4 and so on.

This is a subset of the MNIST handwritten digit dataset (<http://yann.lecun.com/exdb/mnist/>)

In [6]:

```
# load dataset
X, y = load_data()
```

4.2.1 View the variables

Let's get more familiar with your dataset.

- A good place to start is to print out each variable and see what it contains.

The code below prints the first element in the variables `X` and `y`.

In [7]:

```
print ('The first element of X is: ', X[0])
```

In [8]:

```
print ('The first element of y is: ', y[0,0])
print ('The last element of y is: ', y[-1,0])
```

```
The first element of y is:  0
The last element of y is:  9
```

4.2.2 Check the dimensions of your variables

Another way to get familiar with your data is to view its dimensions. Please print the shape of `X` and `y` and see how many training examples you have in your dataset.

In [9]:

```
print ('The shape of X is: ' + str(X.shape))
print ('The shape of y is: ' + str(y.shape))
```

```
The shape of X is: (5000, 400)
```

The shape of `y` is: (5000, 1)

4.2.3 Visualizing the Data

You will begin by visualizing a subset of the training set.

- In the cell below, the code randomly selects 64 rows from `X`, maps each row back to a 20 pixel by 20 pixel grayscale image and displays the images together.
- The label for each image is displayed above the image

In [10]:

```
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
# You do not need to modify anything in this cell

m, n = X.shape

fig, axes = plt.subplots(8,8, figsize=(5,5))
fig.tight_layout(pad=0.13,rect=[0, 0.03, 1, 0.91]) #[left, bottom, right, top]

#fig.tight_layout(pad=0.5)
widgvis(fig)
for i,ax in enumerate(axes.flat):
    # Select random indices
    random_index = np.random.randint(m)

    # Select rows corresponding to the random indices and
    # reshape the image
    X_random_reshaped = X[random_index].reshape((20,20)).T

    # Display the image
    ax.imshow(X_random_reshaped, cmap='gray')

    # Display the label above the image
    ax.set_title(y[random_index,0])
    ax.set_axis_off()
    fig.suptitle("Label, image", fontsize=14)
```

4.3 Model representation

The neural network you will use in this assignment is shown in the figure below.

- This has two dense layers with ReLU activations followed by an output layer with a linear activation.
 - Recall that our inputs are pixel values of digit images.
 - Since the images are of size 20×20 , this gives us 400 inputs

- The parameters have dimensions that are sized for a neural network with 2525 units in layer 1, 1515 units in layer 2 and 1010 output units in layer 3, one for each digit.

4.4 Tensorflow Model Implementation

Tensorflow models are built layer by layer. A layer's input dimensions (Sin $\diamond\diamond\diamond$ above) are calculated for you. You specify a layer's *output dimensions* and this determines the next layer's input dimension. The input dimension of the first layer is derived from the size of the input data specified in the `model.fit` statement below.

Note: It is also possible to add an input layer that specifies the input dimension of the first layer. For example:

```
tf.keras.Input(shape=(400,)), #specify input shape
```

We will include that here to illuminate some model sizing.

4.5 Softmax placement

As described in the lecture and the optional softmax lab, numerical stability is improved if the softmax is grouped with the loss function rather than the output layer during training. This has implications when *building* the model and *using* the model.

Building:

- The final Dense layer should use a 'linear' activation. This is effectively no activation.
- The `model.compile` statement will indicate this by including `from_logits=True, loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)`
- This does not impact the form of the target. In the case of `SparseCategoricalCrossentropy`, the target is the expected digit, 0-9.

Using the model:

- The outputs are not probabilities. If output probabilities are desired, apply a softmax function.

Exercise 2

Below, using Keras [Sequential model](#) and [Dense Layer](#) with a ReLU activation to construct the three layer network described above.

In [11]:

```
# UNQ_C2
# GRADED CELL: Sequential model
tf.random.set_seed(1234) # for consistent results
model = Sequential(
    [
        ### START CODE HERE ###

        # Input layer (specifying input shape)
```

```

tf.keras.Input(shape=(400,)), # 400 input features for 20x20 image
s

# First dense layer with 25 units and ReLU activation
Dense(25, activation='relu'),

# Second dense layer with 15 units and ReLU activation
Dense(15, activation='relu'),

# Output layer with 10 units (one for each digit) and linear activation
Dense(10, activation='linear'),

### END CODE HERE ###
], name = "my_model"
)

```

In [12]:

```
model.summary()
```

```
Model: "my_model"
```

Layer (type)	Output Shape	Param #
=====		
dense (Dense)	(None, 25)	10025
dense_1 (Dense)	(None, 15)	390
dense_2 (Dense)	(None, 10)	160
=====		
Total params: 10,575		
Trainable params: 10,575		
Non-trainable params: 0		

In [13]:

```

# BEGIN UNIT TEST
test_model(model, 10, 400)
# END UNIT TEST

```

All tests passed!

The parameter counts shown in the summary correspond to the number of elements in the weight and bias arrays as shown below.

Let's further examine the weights to verify that tensorflow produced the same dimensions as we calculated above.

In [14]:

```
[layer1, layer2, layer3] = model.layers
```

In [15]:

```
#### Examine Weights shapes
```

```

W1,b1 = layer1.get_weights()
W2,b2 = layer2.get_weights()
W3,b3 = layer3.get_weights()
print(f"W1 shape = {W1.shape}, b1 shape = {b1.shape}")
print(f"W2 shape = {W2.shape}, b2 shape = {b2.shape}")
print(f"W3 shape = {W3.shape}, b3 shape = {b3.shape}")

W1 shape = (400, 25), b1 shape = (25,)
W2 shape = (25, 15), b2 shape = (15,)
W3 shape = (15, 10), b3 shape = (10,)

```

Expected Output

```

W1 shape = (400, 25), b1 shape = (25,)

W2 shape = (25, 15), b2 shape = (15,)

W3 shape = (15, 10), b3 shape = (10,)

```

The following code:

- defines a loss function, `SparseCategoricalCrossentropy` and indicates the softmax should be included with the loss calculation by adding `from_logits=True`)
- defines an optimizer. A popular choice is Adaptive Moment (Adam) which was described in lecture.

In [16]:

```

model.compile(
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
)

history = model.fit(
    X,y,
    epochs=40
)

```

Epochs and batches

In the `fit` statement above, the number of `epochs` was set to 40. This specifies that the entire data set should be applied during training 40 times. During training, you see output describing the progress of training that looks like this:

```

Epoch 1/40

157/157 [=====] - 0s 1ms/step - loss: 2
.2770

```

The first line, `Epoch 1/40`, describes which epoch the model is currently running. For efficiency, the training data set is broken into 'batches'. The default size of a batch in Tensorflow is 32.

There are 5000 examples in our data set or roughly 157 batches. The notation on the 2nd line `157/157 [====` is describing which batch has been executed.

Loss (cost)

In course 1, we learned to track the progress of gradient descent by monitoring the cost. Ideally, the cost will decrease as the number of iterations of the algorithm increases. Tensorflow refers to the cost as `loss`. Above, you saw the loss displayed each epoch as `model.fit` was executing. The `.fit` method returns a variety of metrics including the loss. This is captured in the `history` variable above. This can be used to examine the loss in a plot as shown below.

In [17]:

```
plot_loss_tf(history)
```

Prediction

To make a prediction, use Keras `predict`. Below, `X[1015]` contains an image of a two.

In [18]:

```
image_of_two = X[1015]
display_digit(image_of_two)
```

```
prediction = model.predict(image_of_two.reshape(1,400)) # prediction
```

```
print(f" predicting a Two: \n{prediction}")
```

```
print(f" Largest Prediction index: {np.argmax(prediction)}")
```

```
    predicting a Two:
```

```
[[ -7.99  -2.23   0.77  -2.41 -11.66 -11.15  -9.53  -3.36  -4.42  -7.17]]
```

```
    Largest Prediction index: 2
```

The largest output is `prediction[2]`, indicating the predicted digit is a '2'. If the problem only requires a selection, that is sufficient. Use NumPy [argmax](#) to select it. If the problem requires a probability, a softmax is required:

In [19]:

```
prediction_p = tf.nn.softmax(prediction)
```

```
print(f" predicting a Two. Probability vector: \n{prediction_p}")
```

```
print(f"Total of predictions: {np.sum(prediction_p):0.3f}")
```

```
    predicting a Two. Probability vector:
```

```
[[1.42e-04 4.49e-02 8.98e-01 3.76e-02 3.61e-06 5.97e-06 3.03e-05 1.44e-02
```

```
   5.03e-03 3.22e-04]]
```

```
Total of predictions: 1.000
```

To return an integer representing the predicted target, you want the index of the largest probability. This is accomplished with the Numpy [argmax](#) function.

In [20]:

```
yhat = np.argmax(prediction_p)
```

```
print(f"np.argmax(prediction_p): {yhat}")
```

```
np.argmax(prediction_p): 2
```

Let's compare the predictions vs the labels for a random sample of 64 digits. This takes a moment to run.

In [21]:

```
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
# You do not need to modify anything in this cell

m, n = X.shape

fig, axes = plt.subplots(8,8, figsize=(5,5))
fig.tight_layout(pad=0.13,rect=[0, 0.03, 1, 0.91]) #[left, bottom, right, top]
widgvis(fig)
for i,ax in enumerate(axes.flat):
    # Select random indices
    random_index = np.random.randint(m)

    # Select rows corresponding to the random indices and
    # reshape the image
    X_random_reshaped = X[random_index].reshape((20,20)).T

    # Display the image
    ax.imshow(X_random_reshaped, cmap='gray')

    # Predict using the Neural Network
    prediction = model.predict(X[random_index].reshape(1,400))
    prediction_p = tf.nn.softmax(prediction)
    yhat = np.argmax(prediction_p)

    # Display the label above the image
    ax.set_title(f"{y[random_index,0]}, {yhat}", fontsize=10)
    ax.set_axis_off()
fig.suptitle("Label, yhat", fontsize=14)
plt.show()
```

Let's look at some of the errors.

Note: increasing the number of training epochs can eliminate the errors on this data set.

In [22]:

```
print( f"{display_errors(model,X,y)} errors out of {len(X)} images")
15 errors out of 5000 images
```

Artificial Intelligence

Lab: 14

Lab 14

Advice for Applying Machine Learning

In this lab, you will explore techniques to evaluate and improve your machine learning models.

Outline

- [1 - Packages](#)
- [2 - Evaluating a Learning Algorithm \(Polynomial Regression\)](#)
 - [2.1 Splitting your data set](#)
 - [2.2 Error calculation for model evaluation, linear regression](#)
 - [Exercise 1](#)
 - [2.3 Compare performance on training and test data](#)
- [3 - Bias and Variance](#)
 - [3.1 Plot Train, Cross-Validation, Test](#)
 - [3.2 Finding the optimal degree](#)
 - [3.3 Tuning Regularization.](#)
 - [3.4 Getting more data: Increasing Training Set Size \(m\)](#)
- [4 - Evaluating a Learning Algorithm \(Neural Network\)](#)
 - [4.1 Data Set](#)
 - [4.2 Evaluating categorical model by calculating classification error](#)
 - [Exercise 2](#)
- [5 - Model Complexity](#)
 - [Exercise 3](#)
 - [5.1 Simple model](#)
 - [Exercise 4](#)
- [6 - Regularization](#)
 - [Exercise 5](#)
- [7 - Iterate to find optimal regularization value](#)
 - [7.1 Test](#)

NOTE: To prevent errors from the autograder, you are not allowed to edit or delete non-graded cells in this notebook . Please also refrain from adding any new cells. **Once you have passed this assignment** and want to experiment with any of the non-graded code, you may follow the instructions at the bottom of this notebook.

1 - Packages

First, let's run the cell below to import all the packages that you will need during this assignment.

- [numpy](#) is the fundamental package for scientific computing Python.
- [matplotlib](#) is a popular library to plot graphs in Python.
- [scikitlearn](#) is a basic library for data mining

- [tensorflow](#) a popular platform for machine learning.

In [1]:

```
import numpy as np
%matplotlib widget
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.activations import relu, linear
from tensorflow.keras.losses import SparseCategoricalCrossentropy
from tensorflow.keras.optimizers import Adam

import logging
logging.getLogger("tensorflow").setLevel(logging.ERROR)

from public_tests_a1 import *

tf.keras.backend.set_floatx('float64')
from assignment_utils import *

tf.autograph.set_verbosity(0)
```

2 - Evaluating a Learning Algorithm (Polynomial Regression)

Let's say you have created a machine learning model and you find it *fits* your training data very well. You're done? Not quite. The goal of creating the model was to be able to predict values for **new** examples.

How can you test your model's performance on new data before deploying it?
The answer has two parts:

- Split your original data set into "Training" and "Test" sets.
 - Use the training data to fit the parameters of the model
 - Use the test data to evaluate the model on *new* data
- Develop an error function to evaluate your model.

2.1 Splitting your data set

Lectures advised reserving 20-40% of your data set for testing. Let's use an `sklearn` function [train_test_split](#) to perform the split. Double-check the shapes after running the following cell.

In [2]:

```
# Generate some data
X,y,x_ideal,y_ideal = gen_data(18, 2, 0.7)
print("X.shape", X.shape, "y.shape", y.shape)

#split the data using sklearn routine
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.33, random_state=1)
print("X_train.shape", X_train.shape, "y_train.shape", y_train.shape)
print("X_test.shape", X_test.shape, "y_test.shape", y_test.shape)

X.shape (18,) y.shape (18,)
X_train.shape (12,) y_train.shape (12,)
X_test.shape (6,) y_test.shape (6,)
```

2.1.1 Plot Train, Test sets

You can see below the data points that will be part of training (in red) are intermixed with those that the model is not trained on (test). This particular data set is a quadratic function with noise added. The "ideal" curve is shown for reference.

In [3]:

```
fig, ax = plt.subplots(1,1,figsize=(4,4))
ax.plot(x_ideal, y_ideal, "--", color = "orangered", label="y_ideal", lw=1)
ax.set_title("Training, Test",fontsize = 14)
ax.set_xlabel("x")
ax.set_ylabel("y")

ax.scatter(X_train, y_train, color = "red", label="train")
ax.scatter(X_test, y_test, color = "d1blue", label="test")
ax.legend(loc='upper left')
plt.show()
```

2.2 Error calculation for model evaluation, linear regression

When *evaluating* a linear regression model, you average the squared error difference of the predicted values and the target values.

Exercise 1

Below, create a function to evaluate the error on a data set for a linear regression model.

In [12]:

```
# UNQ_C1
# GRADED CELL: eval_mse
def eval_mse(y, yhat):
    """
    Calculate the mean squared error on a data set.
    Args:
        y : (ndarray Shape (m,) or (m,1)) target value of each example
```

```

    yhat : (ndarray Shape (m,) or (m,1)) predicted value of each example

Returns:
    err: (scalar)
"""
m = len(y)
err = 0.0
for i in range(m):
    ### START CODE HERE ###

    err += ((y[i] - yhat[i]) ** 2)

err *= (1/(2*m))
    ### END CODE HERE ###

return(err)

```

In [13]:

```

y_hat = np.array([2.4, 4.2])
y_tmp = np.array([2.3, 4.1])
eval_mse(y_hat, y_tmp)

# BEGIN UNIT TEST
test_eval_mse(eval_mse)
# END UNIT TEST

All tests passed.

```

[Click for hints](#)

2.3 Compare performance on training and test data

Let's build a high degree polynomial model to minimize training error. This will use the linear_regression functions from `sklearn`. The code is in the imported utility file if you would like to see the details. The steps below are:

- create and fit the model. ('fit' is another name for training or running gradient descent).
- compute the error on the training data.
- compute the error on the test data.

In [14]:

```

# create a model in sklearn, train on training data
degree = 10
lmodel = lin_model(degree)
lmodel.fit(X_train, y_train)

# predict on training data, find training error
yhat = lmodel.predict(X_train)
err_train = lmodel.mse(y_train, yhat)

```

```
# predict on test data, find error
yhat = lmodel.predict(X_test)
err_test = lmodel.mse(y_test, yhat)
```

The computed error on the training set is substantially less than that of the test set.

In [15]:

```
print(f"training err {err_train:0.2f}, test err {err_test:0.2f}")
training err 58.01, test err 171215.01
```

The following plot shows why this is. The model fits the training data very well. To do so, it has created a complex function. The test data was not part of the training and the model does a poor job of predicting on this data.

This model would be described as 1) is overfitting, 2) has high variance 3) 'generalizes' poorly.

In [16]:

```
# plot predictions over data range
x = np.linspace(0,int(X.max()),100) # predict values for plot
y_pred = lmodel.predict(x).reshape(-1,1)

plt_train_test(X_train, y_train, X_test, y_test, x, y_pred, x_ideal, y_ideal, degree)
```

The test set error shows this model will not work well on new data. If you use the test error to guide improvements in the model, then the model will perform well on the test data... but the test data was meant to represent *new* data. You need yet another set of data to test new data performance.

The proposal made during lecture is to separate data into three groups. The distribution of training, cross-validation and test sets shown in the below table is a typical distribution, but can be varied depending on the amount of data available.

data	% of total	Description
training	60	Data used to tune model parameters w and b in training or fitting
cross-validation	20	Data used to tune other model parameters like degree of polynomial, regularization or the architecture of a neural network.
test	20	Data used to test the model after tuning to gauge performance on new data

Let's generate three data sets below. We'll once again use `train_test_split` from `sklearn` but will call it twice to get three splits:

In [17]:

```
# Generate data
X,y, x_ideal,y_ideal = gen_data(40, 5, 0.7)
print("X.shape", X.shape, "y.shape", y.shape)

#split the data using sklearn routine
```

```

X_train, X_, y_train, y_ = train_test_split(X,y,test_size=0.40, random_state=1)
X_cv, X_test, y_cv, y_test = train_test_split(X_,y_,test_size=0.50, random_state=1)
print("X_train.shape", X_train.shape, "y_train.shape", y_train.shape)
print("X_cv.shape", X_cv.shape, "y_cv.shape", y_cv.shape)
print("X_test.shape", X_test.shape, "y_test.shape", y_test.shape)

X.shape (40,) y.shape (40,)
X_train.shape (24,) y_train.shape (24,)
X_cv.shape (8,) y_cv.shape (8,)
X_test.shape (8,) y_test.shape (8,)

```

3 - Bias and Variance

Above, it was clear the degree of the polynomial model was too high. How can you choose a good value? It turns out, as shown in the diagram, the training and cross-validation performance can provide guidance. By trying a range of degree values, the training and cross-validation performance can be evaluated. As the degree becomes too large, the cross-validation performance will start to degrade relative to the training performance. Let's try this on our example.

3.1 Plot Train, Cross-Validation, Test

You can see below the datapoints that will be part of training (in red) are intermixed with those that the model is not trained on (test and cv).

In [18]:

```

fig, ax = plt.subplots(1,1,figsize=(4,4))
ax.plot(x_ideal, y_ideal, "--", color = "orangered", label="y_ideal", lw=1)
ax.set_title("Training, CV, Test",fontsize = 14)
ax.set_xlabel("x")
ax.set_ylabel("y")

ax.scatter(X_train, y_train, color = "red", label="train")
ax.scatter(X_cv, y_cv, color = dlc["dlorange"], label="cv")
ax.scatter(X_test, y_test, color = dlc["dlblue"], label="test")
ax.legend(loc='upper left')
plt.show()

```

3.2 Finding the optimal degree

In previous labs, you found that you could create a model capable of fitting complex curves by utilizing a polynomial (See Course1, Week2 Feature Engineering and Polynomial Regression Lab). Further, you demonstrated that by increasing the *degree* of the polynomial, you could *create* overfitting. (See Course 1, Week3, Over-Fitting Lab). Let's use that knowledge here to test our ability to tell the difference between over-fitting and under-fitting.

Let's train the model repeatedly, increasing the degree of the polynomial each iteration. Here, we're going to use the [scikit-learn](#) linear regression model for speed and simplicity.

In [19]:

```
max_degree = 9
err_train = np.zeros(max_degree)
err_cv = np.zeros(max_degree)
x = np.linspace(0, int(X.max()), 100)
y_pred = np.zeros((100, max_degree)) #columns are lines to plot

for degree in range(max_degree):
    lmodel = lin_model(degree+1)
    lmodel.fit(X_train, y_train)
    yhat = lmodel.predict(X_train)
    err_train[degree] = lmodel.mse(y_train, yhat)
    yhat = lmodel.predict(X_cv)
    err_cv[degree] = lmodel.mse(y_cv, yhat)
    y_pred[:, degree] = lmodel.predict(x)

optimal_degree = np.argmin(err_cv)+1
```

Let's plot the result:

In [20]:

```
plt.close("all")
plt_optimal_degree(X_train, y_train, X_cv, y_cv, x, y_pred, x_ideal, y_ideal,
                  err_train, err_cv, optimal_degree, max_degree)
```

The plot above demonstrates that separating data into two groups, data the model is trained on and data the model has not been trained on, can be used to determine if the model is underfitting or overfitting. In our example, we created a variety of models varying from underfitting to overfitting by increasing the degree of the polynomial used.

- On the left plot, the solid lines represent the predictions from these models. A polynomial model with degree 1 produces a straight line that intersects very few data points, while the maximum degree hews very closely to every data point.
- on the right:
 - the error on the trained data (blue) decreases as the model complexity increases as expected
 - the error of the cross-validation data decreases initially as the model starts to conform to the data, but then increases as the model starts to over-fit on the training data (fails to *generalize*).

It's worth noting that the curves in these examples are not as smooth as one might draw for a lecture. It's clear the specific data points assigned to each group can change your results significantly. The general trend is what is important.

3.3 Tuning Regularization.

In previous labs, you have utilized *regularization* to reduce overfitting. Similar to degree, one can use the same methodology to tune the regularization parameter λ .

Let's demonstrate this by starting with a high degree polynomial and varying the regularization parameter.

```
In [21]:
lambda_range = np.array([0.0, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 1e-1, 1, 10, 100])
num_steps = len(lambda_range)
degree = 10
err_train = np.zeros(num_steps)
err_cv = np.zeros(num_steps)
x = np.linspace(0, int(X.max()), 100)
y_pred = np.zeros((100, num_steps)) #columns are lines to plot

for i in range(num_steps):
    lambda_ = lambda_range[i]
    lmodel = lin_model(degree, regularization=True, lambda_=lambda_)
    lmodel.fit(X_train, y_train)
    yhat = lmodel.predict(X_train)
    err_train[i] = lmodel.mse(y_train, yhat)
    yhat = lmodel.predict(X_cv)
    err_cv[i] = lmodel.mse(y_cv, yhat)
    y_pred[:, i] = lmodel.predict(x)

optimal_reg_idx = np.argmin(err_cv)
```

```
In [22]:
plt.close("all")
plt_tune_regularization(X_train, y_train, X_cv, y_cv, x, y_pred, err_train,
err_cv, optimal_reg_idx, lambda_range)
```

Above, the plots show that as regularization increases, the model moves from a high variance (overfitting) model to a high bias (underfitting) model. The vertical line in the right plot shows the optimal value of λ . In this example, the polynomial degree was set to 10.

3.4 Getting more data: Increasing Training Set Size (m)

When a model is overfitting (high variance), collecting additional data can improve performance. Let's try that here.

```
In [23]:
X_train, y_train, X_cv, y_cv, x, y_pred, err_train, err_cv, m_range, degree
= tune_m()
plt_tune_m(X_train, y_train, X_cv, y_cv, x, y_pred, err_train, err_cv, m_range, degree)
```

The above plots show that when a model has high variance and is overfitting, adding more examples improves performance. Note the curves on the left plot. The final curve with the highest value of m is a smooth curve that is in the center of the data. On the right, as the number of examples increases, the performance of the training set and cross-validation set converge to

similar values. Note that the curves are not as smooth as one might see in a lecture. That is to be expected. The trend remains clear: more data improves generalization. Note that adding more examples when the model has high bias (underfitting) does not improve performance.

4 - Evaluating a Learning Algorithm (Neural Network)

Above, you tuned aspects of a polynomial regression model. Here, you will work with a neural network model. Let's start by creating a classification data set.

4.1 Data Set

Run the cell below to generate a data set and split it into training, cross-validation (CV) and test sets. In this example, we're increasing the percentage of cross-validation data points for emphasis.

In [24]:

```
# Generate and split data set
X, y, centers, classes, std = gen_blobs()

# split the data. Large CV population for demonstration
X_train, X_, y_train, y_ = train_test_split(X,y,test_size=0.50, random_state=1)
X_cv, X_test, y_cv, y_test = train_test_split(X_,y,test_size=0.20, random_state=1)
print("X_train.shape:", X_train.shape, "X_cv.shape:", X_cv.shape, "X_test.shape:", X_test.shape)

X_train.shape: (400, 2) X_cv.shape: (320, 2) X_test.shape: (80, 2)
```

In [25]:

```
plt_train_eq_dist(X_train, y_train, classes, X_cv, y_cv, centers, std)
```

Above, you can see the data on the left. There are six clusters identified by color. Both training points (dots) and cross-validation points (triangles) are shown. The interesting points are those that fall in ambiguous locations where either cluster might consider them members. What would you expect a neural network model to do? What would be an example of overfitting? underfitting? On the right is an example of an 'ideal' model, or a model one might create knowing the source of the data. The lines represent 'equal distance' boundaries where the distance between center points is equal. It's worth noting that this model would "misclassify" roughly 8% of the total data set.

4.2 Evaluating categorical model by calculating classification error

The evaluation function for categorical models used here is simply the fraction of incorrect predictions:

Exercise 2

Below, complete the routine to calculate classification error. Note, in this lab, target values are the index of the category and are not [one-hot encoded](#).

In [34]:

```
# UNQ_C2
# GRADED CELL: eval_cat_err
def eval_cat_err(y, yhat):
    """
    Calculate the categorization error

    Args:
        y      : (ndarray Shape (m,) or (m,1))  target value of each example
        yhat   : (ndarray Shape (m,) or (m,1))  predicted value of each example

    Returns:
        cerr: (scalar)

    """
    m = len(y)
    incorrect = 0
    cerr = 0
    for i in range(m):
        ### START CODE HERE ###

        cerr += 1 if y[i] != yhat[i] else 0

    cerr /= m
    ### END CODE HERE ###

    return(cerr)
```

In [35]:

```
y_hat = np.array([1, 2, 0])
y_tmp = np.array([1, 2, 3])
print(f"categorization error {np.squeeze(eval_cat_err(y_hat, y_tmp)):0.3f},
expected:0.333" )
y_hat = np.array([[1], [2], [0], [3]])
y_tmp = np.array([[1], [2], [1], [3]])
print(f"categorization error {np.squeeze(eval_cat_err(y_hat, y_tmp)):0.3f},
expected:0.250" )

# BEGIN UNIT TEST
test_eval_cat_err(eval_cat_err)
# END UNIT TEST

categorization error 0.333, expected:0.333
categorization error 0.250, expected:0.250
All tests passed.
```

Click for hints

5 - Model Complexity

Below, you will build two models. A complex model and a simple model. You will evaluate the models to determine if they are likely to overfit or underfit.

5.1 Complex model

Exercise 3

Below, compose a three-layer model:

- Dense layer with 120 units, relu activation
 - Dense layer with 40 units, relu activation
 - Dense layer with 6 units and a linear activation (not softmax)
- Compile using
- loss with `SparseCategoricalCrossentropy`, remember to use `from_logits=True`
 - Adam optimizer with learning rate of 0.01.

In [38]:

```
# UNQ_C3
# GRADED CELL: model
import logging
logging.getLogger("tensorflow").setLevel(logging.ERROR)

tf.random.set_seed(1234)
model = Sequential(
    [
        ### START CODE HERE ###

        Dense(120, activation='relu'), # Assuming you've defined input_shape
        Dense(40, activation='relu'),
        Dense(6, activation='linear')

        ### END CODE HERE ###

    ], name="Complex"
)
model.compile(
    ### START CODE HERE ###
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
    ### END CODE HERE ###
)
```

In [39]:

```
# BEGIN UNIT TEST
```

```

model.fit(
    X_train, y_train,
    epochs=1000
)
# END UNIT TEST

```

Out[39]:

```
<keras.callbacks.History at 0x7faf577e2fd0>
```

In [40]:

```

# BEGIN UNIT TEST
model.summary()

model_test(model, classes, X_train.shape[1])
# END UNIT TEST

Model: "Complex"

```

Layer (type)	Output Shape	Param #
=====		
dense (Dense)	(None, 120)	360
dense_1 (Dense)	(None, 40)	4840
dense_2 (Dense)	(None, 6)	246
=====		
Total params: 5,446		
Trainable params: 5,446		
Non-trainable params: 0		

All tests passed!

Click for hints

In [41]:

```

#make a model for plotting routines to call
model_predict = lambda Xl: np.argmax(tf.nn.softmax(model.predict(Xl)).numpy
(),axis=1)
plt_nn(model_predict,X_train,y_train, classes, X_cv, y_cv, suptitle="Comple
x Model")

```

This model has worked very hard to capture outliers of each category. As a result, it has miscategorized some of the cross-validation data. Let's calculate the classification error.

In [42]:

```

training_cerr_complex = eval_cat_err(y_train, model_predict(X_train))
cv_cerr_complex = eval_cat_err(y_cv, model_predict(X_cv))
print(f"categorization error, training, complex model: {training_cerr_compl
ex:0.3f}")
print(f"categorization error, cv,          complex model: {cv_cerr_complex:0.3
f}")

categorization error, training, complex model: 0.003

```

categorization error, cv, complex model: 0.122

5.1 Simple model

Now, let's try a simple model

Exercise 4

Below, compose a two-layer model:

- Dense layer with 6 units, relu activation
- Dense layer with 6 units and a linear activation. Compile using
- loss with `SparseCategoricalCrossentropy`, remember to use `from_logits=True`
- Adam optimizer with learning rate of 0.01.

In [44]:

```
# UNQ_C4
# GRADED CELL: model_s

tf.random.set_seed(1234)
model_s = Sequential(
    [
        ### START CODE HERE ###
        Dense(6, activation='relu'),
        Dense(6, activation='linear')
        ### END CODE HERE ###
    ], name = "Simple"
)
model_s.compile(
    ### START CODE HERE ###
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
    ### START CODE HERE ###
)
```

In [45]:

```
import logging
logging.getLogger("tensorflow").setLevel(logging.ERROR)

# BEGIN UNIT TEST
model_s.fit(
    X_train,y_train,
    epochs=1000
)
# END UNIT TEST
```

Out[45]:

```
<keras.callbacks.History at 0x7faf5437fe50>
```

In [46]:

```
# BEGIN UNIT TEST
model_s.summary()

model_s_test(model_s, classes, X_train.shape[1])
# END UNIT TEST

Model: "Simple"
```

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 6)	18
dense_4 (Dense)	(None, 6)	42

=====
Total params: 60
Trainable params: 60
Non-trainable params: 0
=====

All tests passed!

In [47]:

```
#make a model for plotting routines to call
model_predict_s = lambda Xl: np.argmax(tf.nn.softmax(model_s.predict(Xl)).n
umpy(),axis=1)
plt_nn(model_predict_s,X_train,y_train, classes, X_cv, y_cv, suptitle="Simple Model")
```

This simple models does pretty well. Let's calculate the classification error.

In [48]:

```
training_cerr_simple = eval_cat_err(y_train, model_predict_s(X_train))
cv_cerr_simple = eval_cat_err(y_cv, model_predict_s(X_cv))
print(f"categorization error, training, simple model, {training_cerr_simple:0.3f}, complex model: {training_cerr_complex:0.3f}" )
print(f"categorization error, cv, simple model, {cv_cerr_simple:0.3f}, complex model: {cv_cerr_complex:0.3f}" )

categorization error, training, simple model, 0.062, complex model: 0.003
categorization error, cv, simple model, 0.087, complex model: 0.122
Our simple model has a little higher classification error on training data but does better on cross-validation data than the more complex model.
```

6 - Regularization

As in the case of polynomial regression, one can apply regularization to moderate the impact of a more complex model. Let's try this below.

Exercise 5

Reconstruct your complex model, but this time include regularization. Below, compose a three-layer model:

- Dense layer with 120 units, relu activation, `kernel_regularizer=tf.keras.regularizers.l2(0.1)`
- Dense layer with 40 units, relu activation, `kernel_regularizer=tf.keras.regularizers.l2(0.1)`
- Dense layer with 6 units and a linear activation. Compile using
- loss with `SparseCategoricalCrossentropy`, remember to use `from_logits=True`
- Adam optimizer with learning rate of 0.01.

In [49]:

```
# UNQ_C5
# GRADED CELL: model_r

tf.random.set_seed(1234)
model_r = Sequential(
    [
        ### START CODE HERE ###
        Dense(120, activation='relu', kernel_regularizer=tf.keras.regulariz
ers.l2(0.1)),
        Dense(40, activation='relu', kernel_regularizer=tf.keras.regularize
rs.l2(0.1)),
        Dense(6, activation='linear')
        ### START CODE HERE ###
    ], name= None
)
model_r.compile(
    ### START CODE HERE ###
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
    ### START CODE HERE ###
)
```

In [50]:

```
# BEGIN UNIT TEST
model_r.fit(
    X_train, y_train,
    epochs=1000
)
# END UNIT TEST
```

Out[50]:

```
<keras.callbacks.History at 0x7faf54085f10>
```

In [51]:

```
# BEGIN UNIT TEST
model_r.summary()

model_r_test(model_r, classes, X_train.shape[1])
```

```
# END UNIT TEST
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense_5 (Dense)	(None, 120)	360
dense_6 (Dense)	(None, 40)	4840
dense_7 (Dense)	(None, 6)	246

```
Total params: 5,446
```

```
Trainable params: 5,446
```

```
Non-trainable params: 0
```

```
ddd
```

```
All tests passed!
```

Click for hints

In [52]:

```
#make a model for plotting routines to call
```

```
model_predict_r = lambda Xl: np.argmax(tf.nn.softmax(model_r.predict(Xl)).n  
umpy(),axis=1)
```

```
plt_nn(model_predict_r, X_train,y_train, classes, X_cv, y_cv, suptitle="Reg  
ularized")
```

The results look very similar to the 'ideal' model. Let's check classification error.

In [53]:

```
training_cerr_reg = eval_cat_err(y_train, model_predict_r(X_train))  
cv_cerr_reg = eval_cat_err(y_cv, model_predict_r(X_cv))  
test_cerr_reg = eval_cat_err(y_test, model_predict_r(X_test))  
print(f"categorization error, training, regularized: {training_cerr_reg:0.3  
f}, simple model, {training_cerr_simple:0.3f}, complex model: {training_cer  
r_complex:0.3f}" )  
print(f"categorization error, cv,          regularized: {cv_cerr_reg:0.3f}, si  
mple model, {cv_cerr_simple:0.3f}, complex model: {cv_cerr_complex:0.3f}" )  
categorization error, training, regularized: 0.072, simple model, 0.062, co  
mplex model: 0.003  
categorization error, cv,          regularized: 0.066, simple model, 0.087, co  
mplex model: 0.122
```

The simple model is a bit better in the training set than the regularized model but worse in the cross validation set.

7 - Iterate to find optimal regularization value

As you did in linear regression, you can try many regularization values. This code takes several minutes to run. If you have time, you can run it and check the results. If not, you have completed the graded parts of the assignment!

In [54]:

```
tf.random.set_seed(1234)
lambdas = [0.0, 0.001, 0.01, 0.05, 0.1, 0.2, 0.3]
models=[None] * len(lambdas)

for i in range(len(lambdas)):
    lambda_ = lambdas[i]
    models[i] = Sequential(
        [
            Dense(120, activation = 'relu', kernel_regularizer=tf.keras.regularizers.l2(lambda_)),
            Dense(40, activation = 'relu', kernel_regularizer=tf.keras.regularizers.l2(lambda_)),
            Dense(classes, activation = 'linear')
        ]
    )
    models[i].compile(
        loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
        optimizer=tf.keras.optimizers.Adam(0.01),
    )

    models[i].fit(
        X_train,y_train,
        epochs=1000
    )
    print(f"Finished lambda = {lambda_}")

plot_iterate(lambdas, models, X_train, y_train, X_cv, y_cv)
```

As regularization is increased, the performance of the model on the training and cross-validation data sets converge. For this data set and model, $\lambda > 0.01$ seems to be a reasonable choice.

7.1 Test

Let's try our optimized models on the test set and compare them to 'ideal' performance.

In [56]:

```
plt_compare(X_test,y_test, classes, model_predict_s, model_predict_r, centers)
```

Our test set is small and seems to have a number of outliers so classification error is high. However, the performance of our optimized models is comparable to ideal performance.

Artificial Intelligence

Lab: 15

Lab 15

Decision Trees

In this exercise, you will implement a decision tree from scratch and apply it to the task of classifying whether a mushroom is edible or poisonous.

Outline

- [1 - Packages](#)
- [2 - Problem Statement](#)
- [3 - Dataset](#)
 - [3.1 One hot encoded dataset](#)
- [4 - Decision Tree Refresher](#)
 - [4.1 Calculate entropy](#)
 - [Exercise 1](#)
 - [4.2 Split dataset](#)
 - [Exercise 2](#)
 - [4.3 Calculate information gain](#)
 - [Exercise 3](#)
 - [4.4 Get best split](#)
 - [Exercise 4](#)
- [5 - Building the tree](#)

NOTE: To prevent errors from the autograder, you are not allowed to edit or delete non-graded cells in this notebook . Please also refrain from adding any new cells. **Once you have passed this assignment** and want to experiment with any of the non-graded code, you may follow the instructions at the bottom of this notebook.

1 - Packages

First, let's run the cell below to import all the packages that you will need during this assignment.

- [numpy](#) is the fundamental package for working with matrices in Python.
- [matplotlib](#) is a famous library to plot graphs in Python.
- `utils.py` contains helper functions for this assignment. You do not need to modify code in this file.

In [4]:

```
import numpy as np
import matplotlib.pyplot as plt
from public_tests import *
from utils import *

%matplotlib inline
```

2 - Problem Statement

Suppose you are starting a company that grows and sells wild mushrooms.

- Since not all mushrooms are edible, you'd like to be able to tell whether a given mushroom is edible or poisonous based on it's physical attributes
- You have some existing data that you can use for this task.

Can you use the data to help you identify which mushrooms can be sold safely?

Note: The dataset used is for illustrative purposes only. It is not meant to be a guide on identifying edible mushrooms.

3 - Dataset

You will start by loading the dataset for this task. The dataset you have collected is as follows:

Cap Color	Stalk Shape	Solitary	Edible
Brown	Tapering	Yes	1
Brown	Enlarging	Yes	1
Brown	Enlarging	No	0
Brown	Enlarging	No	0
Brown	Tapering	Yes	1
Red	Tapering	Yes	0

Cap Color	Stalk Shape	Solitary	Edible
Red	Enlarging	No	0

Brown	Enlarging	Yes	1
-------	-----------	-----	---

Red	Tapering	No	1
-----	----------	----	---

Brown	Enlarging	No	0
-------	-----------	----	---

- You have 10 examples of mushrooms. For each example, you have
 - Three features
 - Cap Color (Brown or Red),
 - Stalk Shape (Tapering (as in \/) or Enlarging (as in /\)), and
 - Solitary (Yes or No)
 - Label
 - Edible (1 indicating yes or 0 indicating poisonous)

3.1 One hot encoded dataset

For ease of implementation, we have one-hot encoded the features (turned them into 0 or 1 valued features)

Brown Cap	Tapering Stalk Shape	Solitary	Edible
1	1	1	1

1	0	1	1
---	---	---	---

Brown Cap	Tapering Stalk Shape	Solitary	Edible
1	0	0	0
1	0	0	0
1	1	1	1
0	1	1	0
0	0	0	0
1	0	1	1
0	1	0	1
1	0	0	0

Therefore,

- `X_train` contains three features for each example
 - Brown Color (A value of `1` indicates "Brown" cap color and `0` indicates "Red" cap color)
 - Tapering Shape (A value of `1` indicates "Tapering Stalk Shape" and `0` indicates "Enlarging" stalk shape)
 - Solitary (A value of `1` indicates "Yes" and `0` indicates "No")
- `y_train` is whether the mushroom is edible
 - `y = 1` indicates edible
 - `y = 0` indicates poisonous

```
In [5]:
X_train = np.array([[1,1,1],[1,0,1],[1,0,0],[1,0,0],[1,1,1],[0,1,1],[0,0,0],
,[1,0,1],[0,1,0],[1,0,0]])
y_train = np.array([1,1,0,0,1,0,0,1,1,0])
```

View the variables

Let's get more familiar with your dataset.

- A good place to start is to just print out each variable and see what it contains.

The code below prints the first few elements of `X_train` and the type of the variable.

```
In [6]:
print("First few elements of X_train:\n", X_train[:5])
print("Type of X_train:",type(X_train))

First few elements of X_train:
[[1 1 1]
 [1 0 1]
 [1 0 0]
 [1 0 0]
 [1 1 1]]
Type of X_train: <class 'numpy.ndarray'>
```

Now, let's do the same for `y_train`

```
In [7]:
print("First few elements of y_train:", y_train[:5])
print("Type of y_train:",type(y_train))

First few elements of y_train: [1 1 0 0 1]
Type of y_train: <class 'numpy.ndarray'>
```

Check the dimensions of your variables

Another useful way to get familiar with your data is to view its dimensions.

Please print the shape of `X_train` and `y_train` and see how many training examples you have in your dataset.

```
In [8]:
print ('The shape of X_train is:', X_train.shape)
print ('The shape of y_train is: ', y_train.shape)
print ('Number of training examples (m):', len(X_train))

The shape of X_train is: (10, 3)
The shape of y_train is: (10,)
Number of training examples (m): 10
```

4 - Decision Tree Refresher

In this practice lab, you will build a decision tree based on the dataset provided.

- Recall that the steps for building a decision tree are as follows:
 - Start with all examples at the root node
 - Calculate information gain for splitting on all possible features, and pick the one with the highest information gain
 - Split dataset according to the selected feature, and create left and right branches of the tree
 - Keep repeating splitting process until stopping criteria is met
- In this lab, you'll implement the following functions, which will let you split a node into left and right branches using the feature with the highest information gain
 - Calculate the entropy at a node
 - Split the dataset at a node into left and right branches based on a given feature
 - Calculate the information gain from splitting on a given feature
 - Choose the feature that maximizes information gain
- We'll then use the helper functions you've implemented to build a decision tree by repeating the splitting process until the stopping criteria is met
 - For this lab, the stopping criteria we've chosen is setting a maximum depth of 2

4.1 Calculate entropy

First, you'll write a helper function called `compute_entropy` that computes the entropy (measure of impurity) at a node.

- The function takes in a numpy array (`y`) that indicates whether the examples in that node are edible (1) or poisonous(0)

Complete the `compute_entropy()` function.

Exercise 1

Please complete the `compute_entropy()` function using the previous instructions.

If you get stuck, you can check out the hints presented after the cell below to help you with the implementation.

In [9]:

```
# UNQ_C1
# GRADED FUNCTION: compute_entropy

def compute_entropy(y):
    """
    Computes the entropy for

    Args:
        y (ndarray): Numpy array indicating whether each example at a node is
                     edible ('1') or poisonous ('0')
```

```

Returns:
    entropy (float): Entropy at that node

"""
# You need to return the following variables correctly
entropy = 0.

### START CODE HERE ###

n_examples = len(y)
p = np.sum(y==1) / n_examples
if p not in [0., 1.]:
    entropy = (-p * np.log2(p)) + (-(1 - p) * np.log2(1 - p))

### END CODE HERE ###

return entropy

```

Click for hints

You can check if your implementation was correct by running the following test code:

In [10]:

```

# Compute entropy at the root node (i.e. with all examples)
# Since we have 5 edible and 5 non-edible mushrooms, the entropy should be 1"

print("Entropy at root node: ", compute_entropy(y_train))

# UNIT TESTS
compute_entropy_test(compute_entropy)

Entropy at root node: 1.0
All tests passed.

```

Expected Output:

```
Entropy at root node: 1.0
```

4.2 Split dataset

Next, you'll write a helper function called `split_dataset` that takes in the data at a node and a feature to split on and splits it into left and right branches. Later in the lab, you'll implement code to calculate how good the split is.

- The function takes in the training data, the list of indices of data points at that node, along with the feature to split on.
- It splits the data and returns the subset of indices at the left and the right branch.
- For example, say we're starting at the root node (so `node_indices = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`), and we chose to split on feature 0, which is whether or not the example has a brown cap.

- The output of the function is then, `left_indices = [0,1,2,3,4,7,9]` (data points with brown cap) and `right_indices = [5,6,8]` (data points without a brown cap)

	Brown Cap	Tapering Stalk Shape	Solitary	Edible
0	1	1	1	1
1	1	0	1	1
2	1	0	0	0
3	1	0	0	0
4	1	1	1	1
5	0	1	1	0
6	0	0	0	0
7	1	0	1	1
8	0	1	0	1

	Brown Cap	Tapering Stalk Shape	Solitary	Edible
9	1	0	0	0

Exercise 2

Please complete the `split_dataset()` function shown below

- For each index in `node_indices`
 - If the value of `x` at that index for that feature is `1`, add the index to `left_indices`
 - If the value of `x` at that index for that feature is `0`, add the index to `right_indices`

If you get stuck, you can check out the hints presented after the cell below to help you with the implementation.

In [11]:

```
# UNQ_C2
# GRADED FUNCTION: split_dataset

def split_dataset(X, node_indices, feature):
    """
    Splits the data at the given node into
    left and right branches

    Args:
        X (ndarray): Data matrix of shape(n_samples, n_features)
        node_indices (list): List containing the active indices. I.e, the
            samples being considered at this step.
        feature (int): Index of feature to split on

    Returns:
        left_indices (list): Indices with feature value == 1
        right_indices (list): Indices with feature value == 0
    """

    # You need to return the following variables correctly
    left_indices = []
    right_indices = []

    ### START CODE HERE ###
```

```

    for index in node_indices:
        if X[index, feature] == 1:
            left_indices.append(index)
        else:
            right_indices.append(index)

    ### END CODE HERE ###

    return left_indices, right_indices

```

In [12]:

```

# Case 1

root_indices = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Feel free to play around with these variables
# The dataset only has three features, so this value can be 0 (Brown Cap),
# 1 (Tapering Stalk Shape) or 2 (Solitary)
feature = 0

left_indices, right_indices = split_dataset(X_train, root_indices, feature)

print("CASE 1:")
print("Left indices: ", left_indices)
print("Right indices: ", right_indices)

# Visualize the split
generate_split_viz(root_indices, left_indices, right_indices, feature)

print()

# Case 2

root_indices_subset = [0, 2, 4, 6, 8]
left_indices, right_indices = split_dataset(X_train, root_indices_subset, feature)

print("CASE 2:")
print("Left indices: ", left_indices)
print("Right indices: ", right_indices)

# Visualize the split
generate_split_viz(root_indices_subset, left_indices, right_indices, feature)

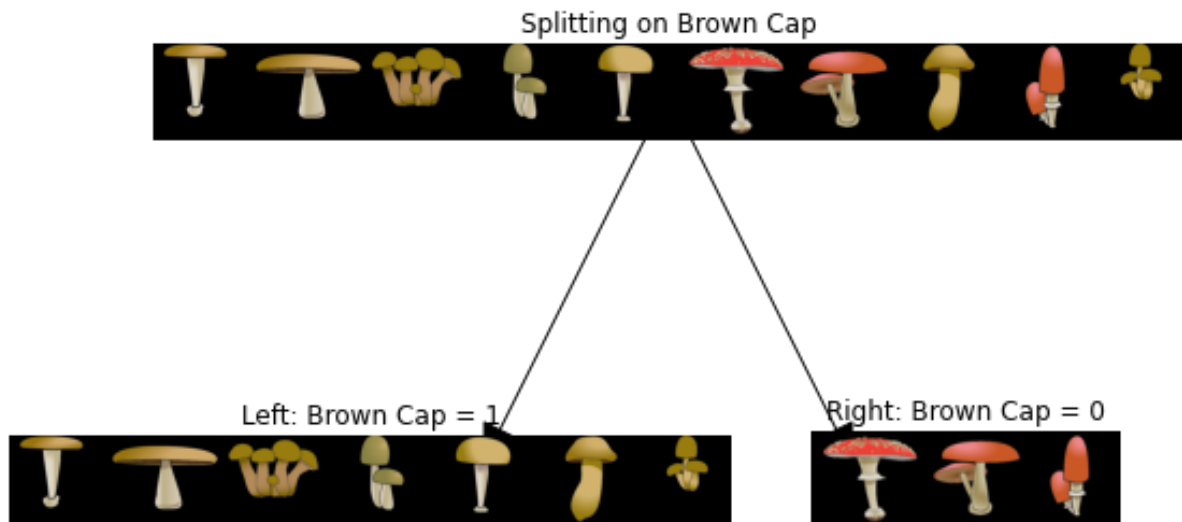
# UNIT TESTS
split_dataset_test(split_dataset)

CASE 1:

```

Left indices: [0, 1, 2, 3, 4, 7, 9]

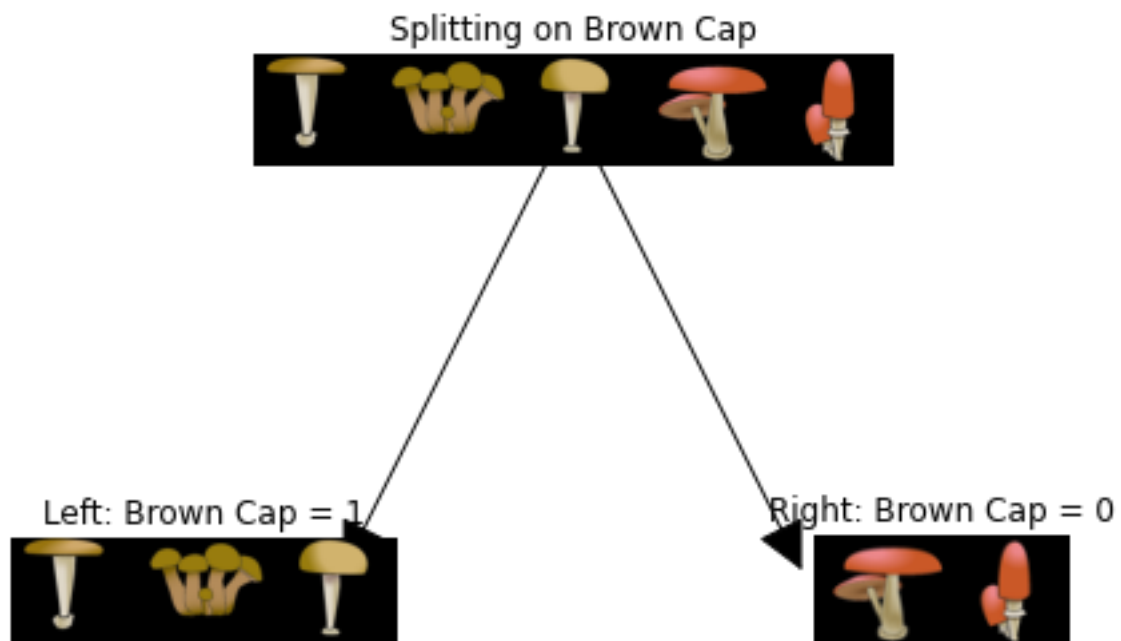
Right indices: [5, 6, 8]



CASE 2:

Left indices: [0, 2, 4]

Right indices: [6, 8]



All tests passed.

Expected Output:

CASE 1:

Left indices: [0, 1, 2, 3, 4, 7, 9]

Right indices: [5, 6, 8]

CASE 2:

Left indices: [0, 2, 4]

Right indices: [6, 8]

4.3 Calculate information gain

Next, you'll write a function called `information_gain` that takes in the training data, the indices at a node and a feature to split on and returns the information gain from the split.

Exercise 3

Please complete the `compute_information_gain()` function shown below to compute

$$\text{Information Gain} = H(p_{\text{node}}) - (w_{\text{left}}H(p_{\text{left}}) + w_{\text{right}}H(p_{\text{right}}))$$
$$\text{Gain} = H(p_{\text{node}}) - (w_{\text{left}}H(p_{\text{left}}) + w_{\text{right}}H(p_{\text{right}}))$$

where

- $H(p_{\text{node}})$ is entropy at the node
- $H(p_{\text{left}})$ and $H(p_{\text{right}})$ are the entropies at the left and the right branches resulting from the split
- w_{left} and w_{right} are the proportion of examples at the left and right branch, respectively

Note:

- You can use the `compute_entropy()` function that you implemented above to calculate the entropy
- We've provided some starter code that uses the `split_dataset()` function you implemented above to split the dataset

If you get stuck, you can check out the hints presented after the cell below to help you with the implementation.

In [15]:

```
# UNQ_C3
# GRADED FUNCTION: compute_information_gain

def compute_information_gain(X, y, node_indices, feature):

    """
    Compute the information of splitting the node on a given feature

    Args:
        X (ndarray): Data matrix of shape (n_samples, n_features)
        y (array like): list or ndarray with n_samples containing the target variable
        node_indices (ndarray): List containing the active indices. I.e, the samples being considered in this step.
```

Returns:

cost (float): Cost computed

```
"""
# Split dataset
left_indices, right_indices = split_dataset(X, node_indices, feature)

# Some useful variables
X_node, y_node = X[node_indices], y[node_indices]
X_left, y_left = X[left_indices], y[left_indices]
X_right, y_right = X[right_indices], y[right_indices]

# You need to return the following variables correctly
information_gain = 0

### START CODE HERE ###

# Calculate proportions of examples in left and right branches
w_left = len(left_indices) / len(node_indices)
w_right = len(right_indices) / len(node_indices)

# Calculate entropies
entropy_node = compute_entropy(y_node)
entropy_left = compute_entropy(y_left)
entropy_right = compute_entropy(y_right)

# Calculate weighted average entropy of the branches
weighted_average_entropy = w_left * entropy_left + w_right * entropy_right

# Finally, calculate the information gain
information_gain = entropy_node - weighted_average_entropy

### END CODE HERE ###
import math
information_gain = information_gain if not math.isnan(information_gain)
else 0.
return information_gain
```

Click for hints

You can now check your implementation using the cell below and calculate what the information gain would be from splitting on each of the features

In [16]:

```
info_gain0 = compute_information_gain(X_train, y_train, root_indices, feature=0)
```

```

print("Information Gain from splitting the root on brown cap: ", info_gain0
)

info_gain1 = compute_information_gain(X_train, y_train, root_indices, featu
re=1)
print("Information Gain from splitting the root on tapering stalk shape: ",
info_gain1)

info_gain2 = compute_information_gain(X_train, y_train, root_indices, featu
re=2)
print("Information Gain from splitting the root on solitary: ", info_gain2)

# UNIT TESTS
compute_information_gain_test(compute_information_gain)
Information Gain from splitting the root on brown cap:  0.03485155455967703
4
Information Gain from splitting the root on tapering stalk shape:  0.124511
24978365313
Information Gain from splitting the root on solitary:  0.2780719051126377
All tests passed.

```

Expected Output:

```

Information Gain from splitting the root on brown cap:  0.034851
554559677034

Information Gain from splitting the root on tapering stalk shape
:  0.12451124978365313

Information Gain from splitting the root on solitary:  0.2780719
051126377

```

Splitting on "Solitary" (feature = 2) at the root node gives the maximum information gain. Therefore, it's the best feature to split on at the root node.

4.4 Get best split

Now let's write a function to get the best feature to split on by computing the information gain from each feature as we did above and returning the feature that gives the maximum information gain

Exercise 4

Please complete the `get_best_split()` function shown below.

- The function takes in the training data, along with the indices of datapoint at that node
- The output of the function is the feature that gives the maximum information gain

- You can use the `compute_information_gain()` function to iterate through the features and calculate the information for each feature. If you get stuck, you can check out the hints presented after the cell below to help you with the implementation.

In [29]:

```
# UNQ_C4
# GRADED FUNCTION: get_best_split

def get_best_split(X, y, node_indices):
    """
    Returns the optimal feature and threshold value
    to split the node data

    Args:
        X (ndarray): Data matrix of shape (n_samples, n_features)
        y (array like): list or ndarray with n_samples containing the
            target variable
        node_indices (ndarray): List containing the active indices. I.e, the
            samples being considered in this step.

    Returns:
        best_feature (int): The index of the best feature to split
    """

    # Some useful variables
    num_features = X.shape[1]

    # You need to return the following variables correctly
    best_feature = -1
    max_information_gain = 0.

    ### START CODE HERE ###
    # Get the feature values for the active indices
    for feature_index in range(num_features):

        # Calculate the information gain for the current feature
        information_gain = compute_information_gain(X, y, node_indices, feature_index)

        # Update the best feature if the current information gain is better
        if information_gain > max_information_gain:
            max_information_gain = information_gain
            best_feature = feature_index

    ### END CODE HERE ###
```

```
return best_feature
```

In [30]:

```
best_feature = get_best_split(X_train, y_train, root_indices)
print("Best feature to split on: %d" % best_feature)
```

```
# UNIT TESTS
```

```
get_best_split_test(get_best_split)
```

```
Best feature to split on: 2
```

```
All tests passed.
```

As we saw above, the function returns that the best feature to split on at the root node is feature 2 ("Solitary")

5 - Building the tree

In this section, we use the functions you implemented above to generate a decision tree by successively picking the best feature to split on until we reach the stopping criteria (maximum depth is 2).

You do not need to implement anything for this part.

In [31]:

```
# Not graded
```

```
tree = []
```

```
def build_tree_recursive(X, y, node_indices, branch_name, max_depth, current_depth):
```

```
    """
```

```
    Build a tree using the recursive algorithm that split the dataset into
    2 subgroups at each node.
```

```
    This function just prints the tree.
```

```
    Args:
```

```
        X (ndarray): Data matrix of shape(n_samples, n_features)
```

```
        y (array like): list or ndarray with n_samples containing the
```

```
        target variable
```

```
        node_indices (ndarray): List containing the active indices. I.e, the
        samples being considered in this step.
```

```
        branch_name (string): Name of the branch. ['Root', 'Left', 'Right']
```

```
    '
```

```
        max_depth (int): Max depth of the resulting tree.
```

```
        current_depth (int): Current depth. Parameter used during recursive
        call.
```

```
    """
```

```
# Maximum depth reached - stop splitting
```

```
if current_depth == max_depth:
```

```
    formatting = " "*current_depth + "-"*current_depth
```

```

        print(formatting, "%s leaf node with indices" % branch_name, node_i
indices)
        return

    # Otherwise, get best split and split the data
    # Get the best feature and threshold at this node
    best_feature = get_best_split(X, y, node_indices)

    formatting = "-"*current_depth
    print("%s Depth %d, %s: Split on feature: %d" % (formatting, current_de
pth, branch_name, best_feature))

    # Split the dataset at the best feature
    left_indices, right_indices = split_dataset(X, node_indices, best_featu
re)
    tree.append((left_indices, right_indices, best_feature))

    # continue splitting the left and the right child. Increment current de
pth
    build_tree_recursive(X, y, left_indices, "Left", max_depth, current_dep
th+1)
    build_tree_recursive(X, y, right_indices, "Right", max_depth, current_d
epth+1)

```

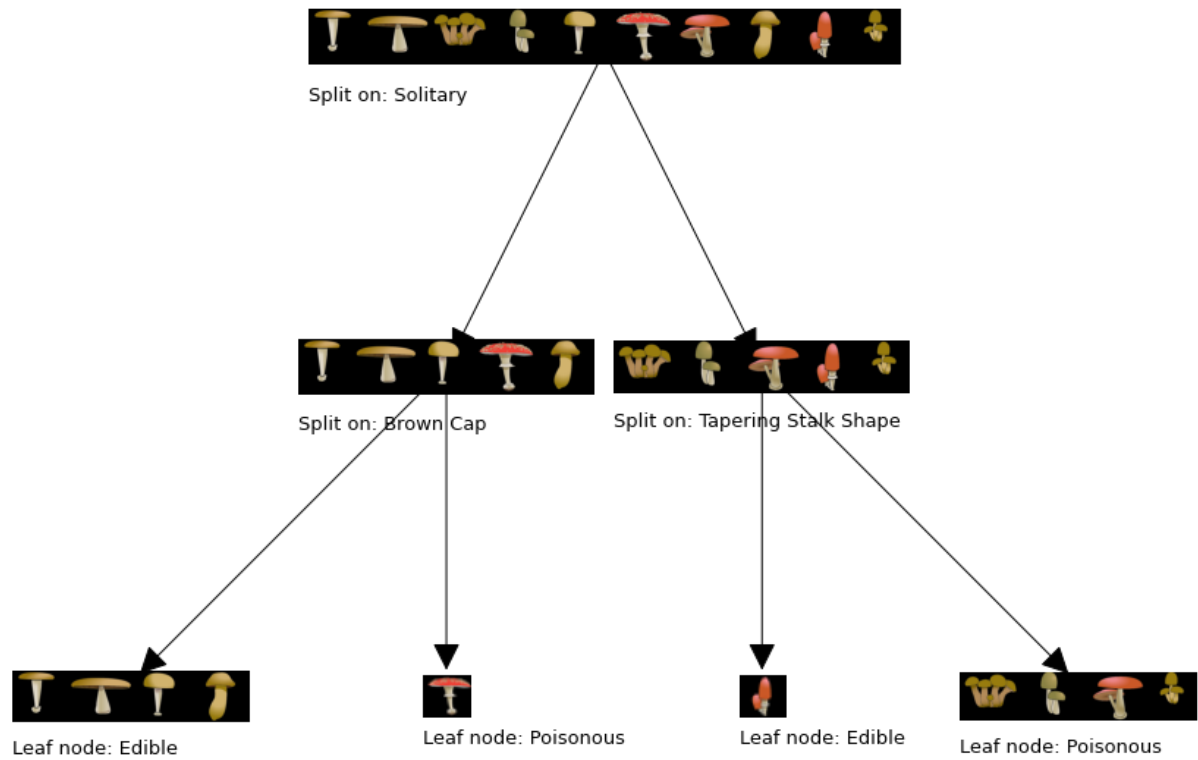
In [32]:

```

build_tree_recursive(X_train, y_train, root_indices, "Root", max_depth=2, c
urrent_depth=0)
generate_tree_viz(root_indices, y_train, tree)

Depth 0, Root: Split on feature: 2
- Depth 1, Left: Split on feature: 0
  -- Left leaf node with indices [0, 1, 4, 7]
  -- Right leaf node with indices [5]
- Depth 1, Right: Split on feature: 1
  -- Left leaf node with indices [8]
  -- Right leaf node with indices [2, 3, 6, 9]

```



Artificial Intelligence

Lab: 16

Lab 16

K-means Clustering

In this exercise, you will implement the K-means algorithm and use it for image compression.

- You will start with a sample dataset that will help you gain an intuition of how the K-means algorithm works.
- After that, you will use the K-means algorithm for image compression by reducing the number of colors that occur in an image to only those that are most common in that image.

Outline

- [1 - Implementing K-means](#)
 - [1.1 Finding closest centroids](#)
 - [Exercise 1](#)
 - [1.2 Computing centroid means](#)
 - [Exercise 2](#)
- [2 - K-means on a sample dataset](#)
- [3 - Random initialization](#)
- [4 - Image compression with K-means](#)
 - [4.1 Dataset](#)
 - [4.2 K-Means on image pixels](#)
 - [4.3 Compress the image](#)

NOTE: To prevent errors from the autograder, you are not allowed to edit or delete non-graded cells in this lab. Please also refrain from adding any new cells. **Once you have passed this assignment** and want to experiment with any of the non-graded code, you may follow the instructions at the bottom of this notebook.

First, run the cell below to import the packages needed in this assignment:

- [numpy](#) is the fundamental package for scientific computing with Python.
- [matplotlib](#) is a popular library to plot graphs in Python.
- `utils.py` contains helper functions for this assignment. You do not need to modify code in this file.

In [1]:

```
import numpy as np
import matplotlib.pyplot as plt
from utils import *

%matplotlib inline
```

1 - Implementing K-means

The K-means algorithm is a method to automatically cluster similar data points together.

- K-means is an iterative procedure that
 - Starts by guessing the initial centroids, and then
 - Refines this guess by
 - Repeatedly assigning examples to their closest centroids, and then
 - Recomputing the centroids based on the assignments.
- In pseudocode, the K-means algorithm is as follows:

```
• # Initialize centroids

• # K is the number of clusters

• centroids = kMeans_init_centroids(X, K)

•

• for iter in range(iterations):

•     # Cluster assignment step:

•     # Assign each data point to the closest centroid.

•     # idx[i] corresponds to the index of the centroid

•     # assigned to example i

•     idx = find_closest_centroids(X, centroids)

•

•     # Move centroid step:

•     # Compute means based on centroid assignments

•     centroids = compute_centroids(X, idx, K)
```

You will implement the two phases of the K-means algorithm separately in the next sections.

- You will start by completing `find_closest_centroid` and then proceed to complete `compute_centroids`.

1.1 Finding closest centroids

Exercise 1

Your task is to complete the code in `find_closest_centroids`.

In [2]:

```
# UNQ_C1
# GRADED FUNCTION: find_closest_centroids

def find_closest_centroids(X, centroids):
    """
    Computes the centroid memberships for every example

    Args:
        X (ndarray): (m, n) Input values
        centroids (ndarray): (K, n) centroids

    Returns:
        idx (array_like): (m,) closest centroids

    """

    # Set K
    K = centroids.shape[0]

    # You need to return the following variables correctly
    idx = np.zeros(X.shape[0], dtype=int)

    ### START CODE HERE ###

    for i in range(X.shape[0]): # Iterate through each example
        distances = np.linalg.norm(X[i] - centroids, axis=1) # Calculate L
        # 2 distances to all centroids
        closest_centroid_index = np.argmin(distances) # Find index of closest centroid
        idx[i] = closest_centroid_index # Assign closest centroid index to idx

    ### END CODE HERE ###

    return idx
```

In [3]:

```
# Load an example dataset that we will be using
X = load_data()
```

The code below prints the first five elements in the variable `X` and the dimensions of the variable

In [4]:

```
print("First five elements of X are:\n", X[:5])
print('The shape of X is:', X.shape)
```

First five elements of X are:

```
[[1.84207953 4.6075716 ]
```

```
[5.65858312 4.79996405]
[6.35257892 3.2908545 ]
[2.90401653 4.61220411]
[3.23197916 4.93989405]]
The shape of X is: (300, 2)
```

In [5]:

```
# Select an initial set of centroids (3 Centroids)
initial_centroids = np.array([[3,3], [6,2], [8,5]])

# Find closest centroids using initial_centroids
idx = find_closest_centroids(X, initial_centroids)

# Print closest centroids for the first three elements
print("First three elements in idx are:", idx[:3])

# UNIT TEST
from public_tests import *

find_closest_centroids_test(find_closest_centroids)

First three elements in idx are: [0 2 1]
All tests passed!
```

Expected Output:

```
First three elements in idx are [0 2 1]
```

1.2 Computing centroid means

Given assignments of every point to a centroid, the second phase of the algorithm recomputes, for each centroid, the mean of the points that were assigned to it.

Exercise 2

Please complete the `compute_centroids` below to recompute the value for each centroid.

In [6]:

```
# UNQ_C2
# GRADED FUNCTION: compute_centroids

def compute_centroids(X, idx, K):
    """
    Returns the new centroids by computing the means of the
    data points assigned to each centroid.

    Args:
        X (ndarray): (m, n) Data points
        idx (ndarray): (m,) Array containing index of closest centroid for
        each
```

```

of                                     example in X. Concretely, idx[i] contains the index
                                     the centroid closest to example i
K (int):                             number of centroids

Returns:
    centroids (ndarray): (K, n) New centroids computed
    """

    # Useful variables
    m, n = X.shape

    # You need to return the following variables correctly
    centroids = np.zeros((K, n))

    ### START CODE HERE ###

    for k in range(K):
        assigned_points = X[idx == k] # Get points assigned to centroid k
        centroids[k] = np.mean(assigned_points, axis=0) # Calculate mean o
        f assigned points

    ### END CODE HERE ##

    return centroids

```

In [7]:

```

K = 3
centroids = compute_centroids(X, idx, K)

print("The centroids are:", centroids)

# UNIT TEST
compute_centroids_test(compute_centroids)
The centroids are: [[2.42830111 3.15792418]
 [5.81350331 2.63365645]
 [7.11938687 3.6166844 ]]
All tests passed!

```

Expected Output:

2.42830111 3.15792418

5.81350331 2.63365645

7.11938687 3.6166844

2 - K-means on a sample dataset

After you have completed the two functions

(`find_closest_centroids` and `compute_centroids`) above, the next step is to run the K-means algorithm on a toy 2D dataset to help you understand how K-means works.

- We encourage you to take a look at the function (`run_kMeans`) below to understand how it works.
- Notice that the code calls the two functions you implemented in a loop.

When you run the code below, it will produce a visualization that steps through the progress of the algorithm at each iteration.

- At the end, your figure should look like the one displayed in Figure 1.
- The final centroids are the black X-marks in the middle of the colored clusters.
- You can see how these centroids got to their final location by looking at the other X-marks connected to it.

Note: You do not need to implement anything for this part. Simply run the code provided below

In [8]:

```
# You do not need to implement anything for this part
```

```
def run_kMeans(X, initial_centroids, max_iters=10, plot_progress=False):
```

```

"""
Runs the K-Means algorithm on data matrix X, where each row of X
is a single example
"""

# Initialize values
m, n = X.shape
K = initial_centroids.shape[0]
centroids = initial_centroids
previous_centroids = centroids
idx = np.zeros(m)
plt.figure(figsize=(8, 6))

# Run K-Means
for i in range(max_iters):

    #Output progress
    print("K-Means iteration %d/%d" % (i, max_iters-1))

    # For each example in X, assign it to the closest centroid
    idx = find_closest_centroids(X, centroids)

    # Optionally plot progress
    if plot_progress:
        plot_progress_kMeans(X, centroids, previous_centroids, idx, K,
i)
        previous_centroids = centroids

    # Given the memberships, compute new centroids
    centroids = compute_centroids(X, idx, K)
plt.show()
return centroids, idx

In [9]:

# Load an example dataset
X = load_data()

# Set initial centroids
initial_centroids = np.array([[3,3],[6,2],[8,5]])

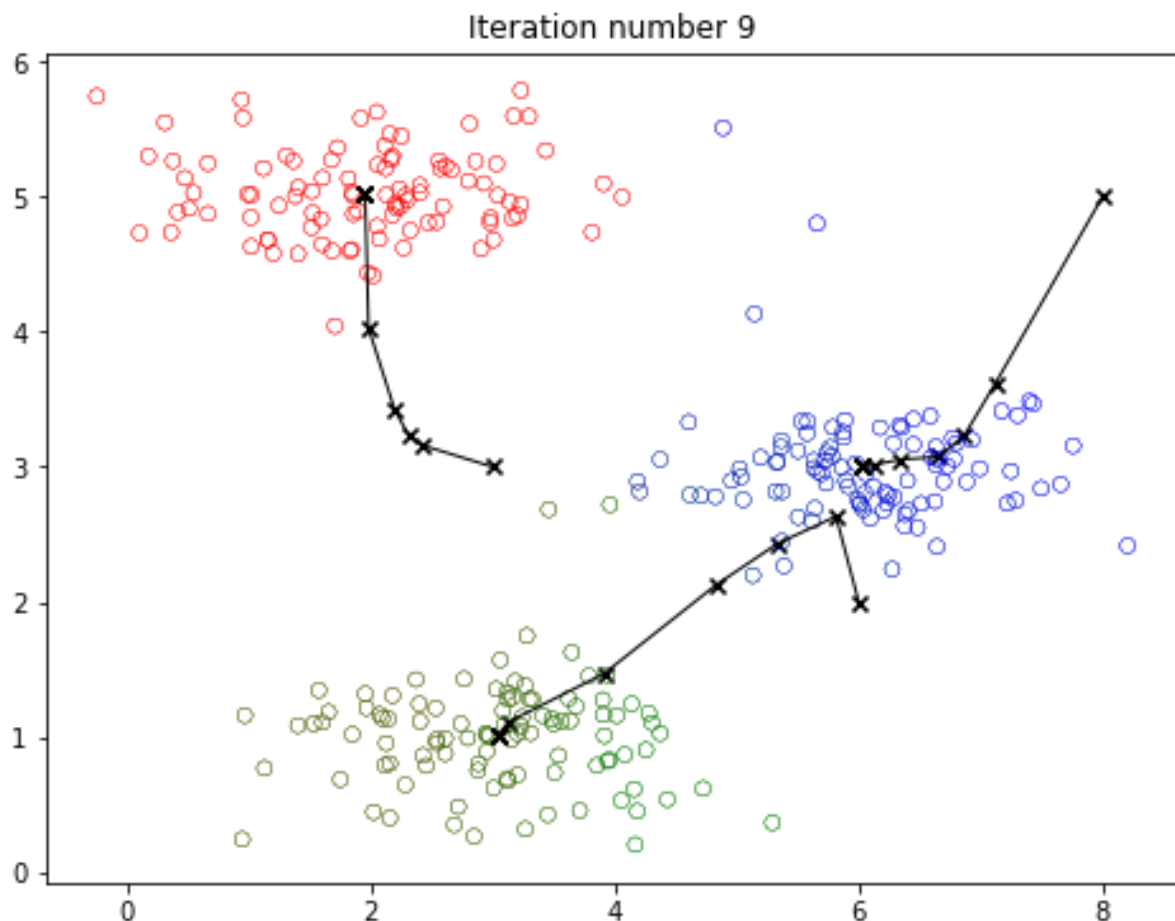
# Number of iterations
max_iters = 10

# Run K-Means
centroids, idx = run_kMeans(X, initial_centroids, max_iters, plot_progress=
True)

K-Means iteration 0/9
K-Means iteration 1/9

```

K-Means iteration 2/9
K-Means iteration 3/9
K-Means iteration 4/9
K-Means iteration 5/9
K-Means iteration 6/9
K-Means iteration 7/9
K-Means iteration 8/9
K-Means iteration 9/9



3 - Random initialization

The initial assignments of centroids for the example dataset was designed so that you will see the same figure as in Figure 1. In practice, a good strategy for initializing the centroids is to select random examples from the training set.

In this part of the exercise, you should understand how the function `kMeans_init_centroids` is implemented.

- The code first randomly shuffles the indices of the examples (using `np.random.permutation()`).
- Then, it selects the first K examples based on the random permutation of the indices.
- This allows the examples to be selected at random without the risk of selecting the same example twice.

Note: You do not need to implement anything for this part of the exercise.

In [10]:

```
# You do not need to modify this part

def kMeans_init_centroids(X, K):
    """
    This function initializes K centroids that are to be
    used in K-Means on the dataset X

    Args:
        X (ndarray): Data points
        K (int):      number of centroids/clusters

    Returns:
        centroids (ndarray): Initialized centroids
    """

    # Randomly reorder the indices of examples
    randidx = np.random.permutation(X.shape[0])

    # Take the first K examples as centroids
    centroids = X[randidx[:K]]

    return centroids
```

You can run K-Means again but this time with random initial centroids. Run the cell below several times and observe how different clusters are created based on the initial points chosen.

In [11]:

```
# Run this cell repeatedly to see different outcomes.

# Set number of centroids and max number of iterations
K = 3
max_iters = 10

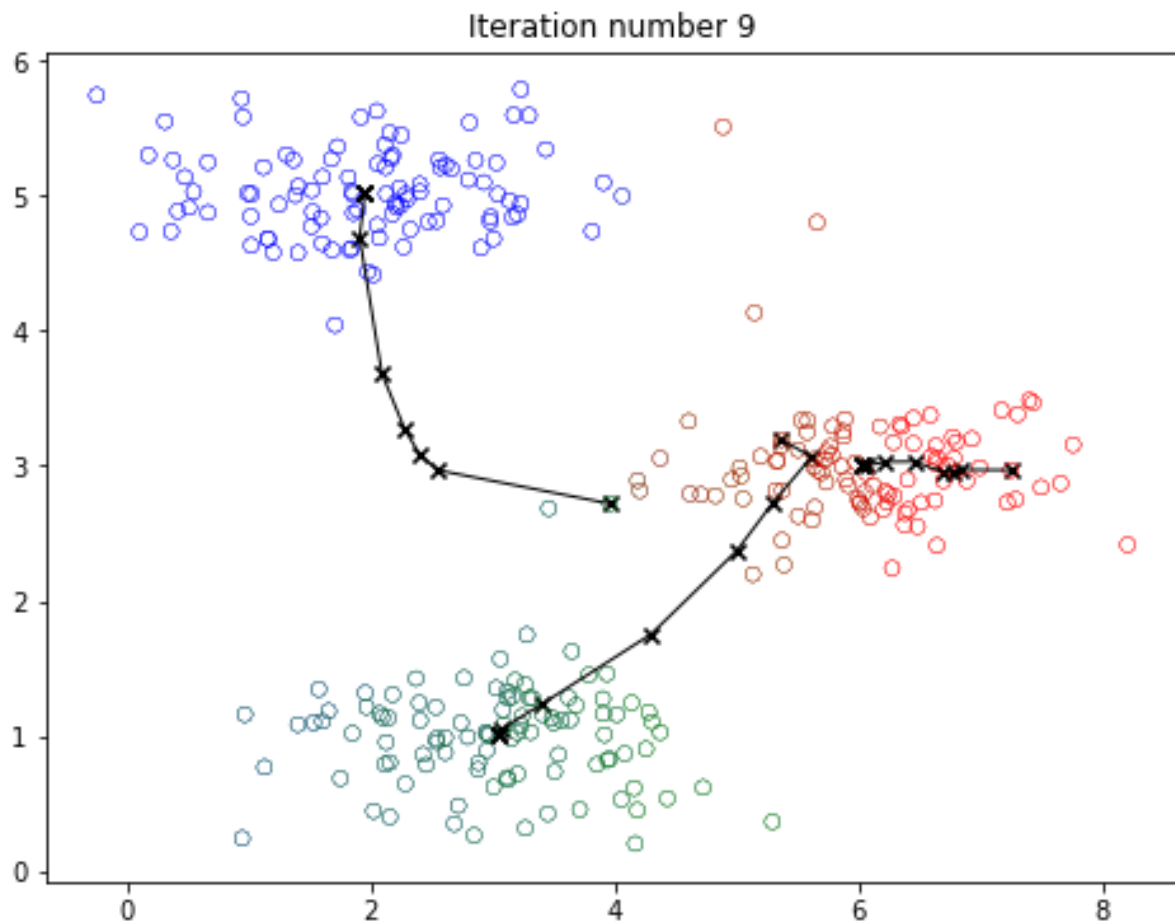
# Set initial centroids by picking random examples from the dataset
initial_centroids = kMeans_init_centroids(X, K)

# Run K-Means
centroids, idx = run_kMeans(X, initial_centroids, max_iters, plot_progress=
True)

K-Means iteration 0/9
K-Means iteration 1/9
K-Means iteration 2/9
K-Means iteration 3/9
K-Means iteration 4/9
K-Means iteration 5/9
K-Means iteration 6/9
K-Means iteration 7/9
```

K-Means iteration 8/9

K-Means iteration 9/9



4 - Image compression with K-means

In this exercise, you will apply K-means to image compression.

- In a straightforward 24-bit color representation of an image, each pixel is represented as three 8-bit unsigned integers (ranging from 0 to 255) that specify the red, green and blue intensity values. This encoding is often referred to as the RGB encoding.
- Our image contains thousands of colors, and in this part of the exercise, you will reduce the number of colors to 16 colors.
- By making this reduction, it is possible to represent (compress) the photo in an efficient way.
- Specifically, you only need to store the RGB values of the 16 selected colors, and for each pixel in the image you now need to only store the index of the color at that location (where only 4 bits are necessary to represent 16 possibilities).

In this part, you will use the K-means algorithm to select the 16 colors that will be used to represent the compressed image.

- Concretely, you will treat every pixel in the original image as a data example and use the K-means algorithm to find the 16 colors that best group (cluster) the pixels in the 3-dimensional RGB space.
- Once you have computed the cluster centroids on the image, you will then use the 16 colors to replace the pixels in the original image.

4.1 Dataset

Load image

First, you will use `matplotlib` to read in the original image, as shown below.

In [12]:

```
# Load an image of a bird
original_img = plt.imread('bird_small.png')
```

Visualize image

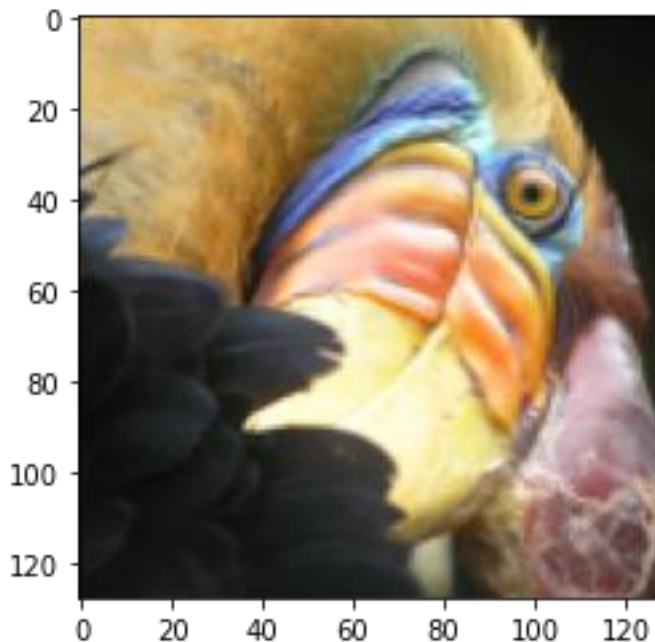
You can visualize the image that was just loaded using the code below.

In [13]:

```
# Visualizing the image
plt.imshow(original_img)
```

Out[13]:

<matplotlib.image.AxesImage at 0x7f7200998a10>



Check the dimension of the variable

As always, you will print out the shape of your variable to get more familiar with the data.

In [14]:

```
print("Shape of original_img is:", original_img.shape)
```

Shape of original_img is: (128, 128, 3)

As you can see, this creates a three-dimensional matrix `original_img` where

- the first two indices identify a pixel position, and
- the third index represents red, green, or blue.

For example, `original_img[50, 33, 2]` gives the blue intensity of the pixel at row 50 and column 33.

Processing data

To call the `run_kMeans`, you need to first transform the matrix `original_img` into a two-dimensional matrix.

Note: If you'll try this exercise later on a JPG file, you first need to divide the pixel values by 255 so it will be in the range 0 to 1. This is not necessary for PNG files (e.g. `bird_small.png`) because it is already loaded in the required range (as mentioned in the [plt.imread\(\)](#) [documentation](#)). We commented a line below for this so you can just uncomment it later in case you want to try a different file.

In [15]:

```
# Divide by 255 so that all values are in the range 0 - 1 (not needed for PNG files)
# original_img = original_img / 255

# Reshape the image into an m x 3 matrix where m = number of pixels
# (in this case m = 128 x 128 = 16384)
# Each row will contain the Red, Green and Blue pixel values
# This gives us our dataset matrix X_img that we will use K-Means on.

X_img = np.reshape(original_img, (original_img.shape[0] * original_img.shape[1], 3))
```

4.2 K-Means on image pixels

Now, run the cell below to run K-Means on the pre-processed image.

In [16]:

```
# Run your K-Means algorithm on this data
# You should try different values of K and max_iters here
K = 16
max_iters = 10

# Using the function you have implemented above.
initial_centroids = kMeans_init_centroids(X_img, K)

# Run K-Means - this can take a couple of minutes depending on K and max_iters
centroids, idx = run_kMeans(X_img, initial_centroids, max_iters)

K-Means iteration 0/9
K-Means iteration 1/9
```

```
K-Means iteration 2/9
K-Means iteration 3/9
K-Means iteration 4/9
K-Means iteration 5/9
K-Means iteration 6/9
K-Means iteration 7/9
K-Means iteration 8/9
K-Means iteration 9/9
<Figure size 576x432 with 0 Axes>
```

In [17]:

```
print("Shape of idx:", idx.shape)
print("Closest centroid for the first five elements:", idx[:5])
```

```
Shape of idx: (16384,)
```

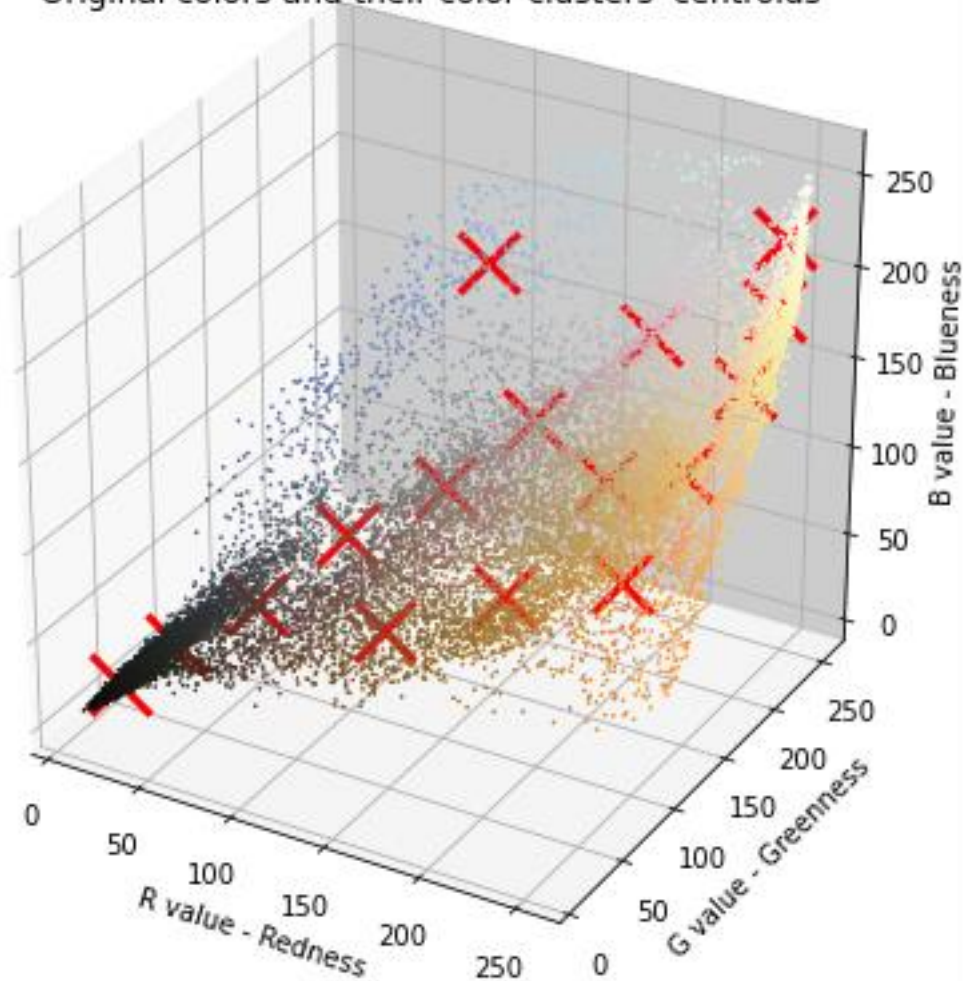
```
Closest centroid for the first five elements: [10 10 10 10 10]
```

The code below will plot all the colors found in the original image. As mentioned earlier, the color of each pixel is represented by RGB values so the plot should have 3 axes -- R, G, and B. You'll notice a lot of dots below representing thousands of colors in the original image. The red markers represent the centroids after running K-means. These will be the 16 colors that you will use to compress the image.

In [18]:

```
# Plot the colors of the image and mark the centroids
plot_kMeans_RGB(X_img, centroids, idx, K)
```

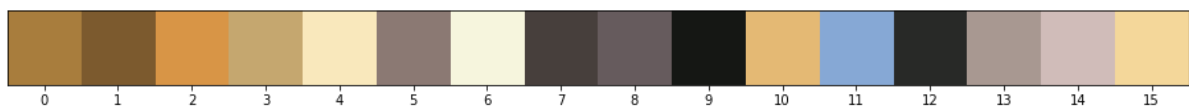
Original colors and their color clusters' centroids



You can visualize the colors at each of the red markers (i.e. the centroids) above with the function below. You will only see these colors when you generate the new image in the next section. The number below each color is its index and these are the numbers you see in the `idx` array.

In [19]:

```
# Visualize the 16 colors selected
show_centroid_colors(centroids)
```



4.3 Compress the image

After finding the top $K=16$ colors to represent the image, you can now assign each pixel position to its closest centroid using the `find_closest_centroids` function.

- This allows you to represent the original image using the centroid assignments of each pixel.

- Notice that you have significantly reduced the number of bits that are required to describe the image.
 - The original image required 24 bits (i.e. 8 bits x 3 channels in RGB encoding) for each one of the $128 \times 128 \times 128$ pixel locations, resulting in total size of $128 \times 128 \times 24 = 393,216$ bits.
 - The new representation requires some overhead storage in form of a dictionary of 16 colors, each of which require 24 bits, but the image itself then only requires 4 bits per pixel location.
 - The final number of bits used is therefore $16 \times 24 + 128 \times 128 \times 4 = 65,920$ bits, which corresponds to compressing the original image by about a factor of 6.

In [20]:

```
# Find the closest centroid of each pixel
idx = find_closest_centroids(X_img, centroids)

# Replace each pixel with the color of the closest centroid
X_recovered = centroids[idx, :]

# Reshape image into proper dimensions
X_recovered = np.reshape(X_recovered, original_img.shape)
```

Finally, you can view the effects of the compression by reconstructing the image based only on the centroid assignments.

- Specifically, you replaced each pixel with the value of the centroid assigned to it.
- Figure 3 shows a sample reconstruction. Even though the resulting image retains most of the characteristics of the original, you will also see some compression artifacts because of the fewer colors.
- Run the code below to see how the image is reconstructed using the 16 colors selected earlier.

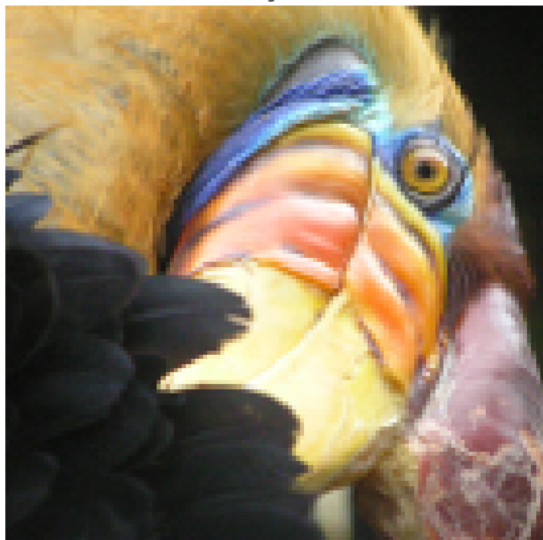
In [21]:

```
# Display original image
fig, ax = plt.subplots(1,2, figsize=(16,16))
plt.axis('off')

ax[0].imshow(original_img)
ax[0].set_title('Original')
ax[0].set_axis_off()

# Display compressed image
ax[1].imshow(X_recovered)
ax[1].set_title('Compressed with %d colours'%K)
ax[1].set_axis_off()
```

Original



Compressed with 16 colours

